Workflow router and benchmark app

handoff/workflow-router-app/app/api/exports/benchmark/route.ts · 20 lines

```
import { NextResponse } from 'next/server';
import { requireUser } from '@/lib/supabase/server';
import { aggregateGrades } from '@/lib/benchmark/scoring';
import { toCsv } from '@/lib/exports/csv';
import { benchmarkMarkdown } from '@/lib/exports/markdown';
export async function GET(req: Request) {
  const url = new URL(req.url); const suiteId = url.searchParams.get('suite'); const format = url.searchParams.get('format')
=== 'md' ? 'md' : 'csv';
  const { supabase, user } = await requireUser(); if (!user) return NextResponse.json({ error: 'Sign in' }, { status: 401 });
  if (!suiteId) return NextResponse.json({ error: 'suite is required' }, { status: 422 });
  const { data: tasks, error } = await supabase.from('benchmark_tasks').select('id, number, weights, benchmark_runs(id,
platform, status, benchmark_grades(*))').eq('suite_id', suiteId).order('number'); // RLS scopes this
  if (error) return NextResponse.json({ error: error.message }, { status: 500 });
  const rows = (tasks ?? []).flatMap((t: any) => (t.benchmark_runs ?? []).map((r: any) => {
    const sheets = (r.benchmark_grades ?? []).map((g: any) => ({ correctness: g.correctness, codeQuality: g.code_quality,
rendering: g.rendering, stateManagement: g.state_management, maintainability: g.maintainability, handoff: g.handoff,
sandboxFit: g.sandbox_fit, persistence: g.persistence }));
    const s = aggregateGrades(sheets, t.weights ?? undefined);
    return { workflow: t.number, platform: r.platform, status: r.status, outOf5: s.outOf5, outOf100: s.outOf100, graders:
s.graders, missing: s.missing.join(' ') };
  }));
  const body = format === 'md' ? benchmarkMarkdown(rows) : toCsv(rows, ['workflow', 'platform', 'status', 'outOf5', 'outOf100',
'graders', 'missing']);
  return new NextResponse(body, { headers: { 'content-type': format === 'md' ? 'text/markdown; charset=utf-8' : 'text/csv;
charset=utf-8', 'content-disposition': 'attachment; filename="semester-benchmark.' + format + '"' } });
}
```

handoff/workflow-router-app/app/api/exports/recommendation/route.ts · 15 lines

```
import { NextResponse } from 'next/server';
import { RouterAnswersSchema } from '@/lib/validation/workflow-router';
import { recommend } from '@/lib/workflow-router/engine';
import { buildPrompt } from '@/lib/workflow-router/prompts';
import { recommendationMarkdown } from '@/lib/exports/markdown';
import { toJson } from '@/lib/exports/json';
export async function POST(req: Request) {
  const format = new URL(req.url).searchParams.get('format') === 'json' ? 'json' : 'md';
  const parsed = RouterAnswersSchema.safeParse(await req.json().catch(() => null));
  if (!parsed.success) return NextResponse.json({ error: parsed.error.flatten() }, { status: 422 });
  const r = recommend(parsed.data); const prompt = buildPrompt(parsed.data, r);
  const body = format === 'json' ? toJson({ answers: parsed.data, recommendation: r, prompt }) :
recommendationMarkdown(parsed.data, r, prompt);
  return new NextResponse(body, { headers: { 'content-type': format === 'json' ? 'application/json' : 'text/markdown;
charset=utf-8', 'content-disposition': 'attachment; filename="semester-recommendation.' + format + '"' } });
}
```

handoff/workflow-router-app/app/auth/confirm/route.ts · 11 lines

```
import { type EmailOtpType } from '@supabase/supabase-js';
import { NextResponse, type NextRequest } from 'next/server';
import { createClient } from '@/lib/supabase/server';
export async function GET(request: NextRequest) {
  const url = new URL(request.url);
  const token_hash = url.searchParams.get('token_hash'); const type = url.searchParams.get('type') as EmailOtpType | null;
  const next = url.searchParams.get('next')?.startsWith('/') ? url.searchParams.get('next')! : '/workflow-router';
  if (token_hash && type) { const supabase = await createClient(); const { error } = await supabase.auth.verifyOtp({ type,
token_hash }); if (!error) return NextResponse.redirect(new URL(next, url.origin)); }
  return NextResponse.redirect(new URL('/auth/sign-in?error=link', url.origin));
}
```

handoff/workflow-router-app/app/auth/sign-in/page.tsx · 17 lines

```
'use client';
import { useState } from 'react';
import { createClient } from '@/lib/supabase/client';
export default function SignIn() {
  const [email, setEmail] = useState(''); const [state, setState] = useState<'idle' | 'sending' | 'sent' | 'error'>('idle');
const [msg, setMsg] = useState('');
  async function send(e: React.FormEvent) {
    e.preventDefault(); setState('sending');
    const { error } = await createClient().auth.signInWithOtp({ email, options: { emailRedirectTo: location.origin +
'/auth/confirm?next=/workflow-router' } });
    if (error) { setState('error'); setMsg(error.message); } else setState('sent');
  }
  return (<main className="mx-auto max-w-sm p-8"><h1 className="text-2xl font-semibold">Sign in</h1>
    <p className="mt-2 text-sm text-neutral-600">We'll email you a sign-in link. Without signing in, the router works in
prototype mode and nothing is saved.</p>
    <form onSubmit={send} className="mt-6 grid gap-3"><label className="grid gap-1 text-sm">School or work email<input required
type="email" value={email} onChange={(e) => setEmail(e.target.value)} className="min-h-11 rounded border px-3" /></label>
      <button disabled={state === 'sending'} className="min-h-11 rounded bg-slate-800 px-4 font-medium text-white
disabled:opacity-50">{state === 'sending' ? 'Sending…' : 'Email me a link'}</button></form>
    <p role="status" className="mt-4 text-sm">{state === 'sent' ? 'Check your inbox — the link signs you in on this device.' :
state === 'error' ? msg : ''}</p></main>);
}
```

handoff/workflow-router-app/app/auth/sign-out/route.ts · 4 lines

```
import { NextResponse } from 'next/server';
import { createClient } from '@/lib/supabase/server';
export async function POST(request: Request) { const supabase = await createClient(); await supabase.auth.signOut(); return
NextResponse.redirect(new URL('/', request.url), { status: 303 }); }
```

handoff/workflow-router-app/app/benchmark/-suiteId-/page.tsx · 16 lines

```
import { notFound } from 'next/navigation';
import { requireUser } from '@/lib/supabase/server';
import { aggregateGrades, leaderboard } from '@/lib/benchmark/scoring';
import { PLATFORM_LABEL, PLATFORMS, type Platform } from '@/lib/benchmark/benchmark.config';
import { Leaderboard } from '@/components/benchmark/leaderboard';
export default async function Page({ params }: { params: Promise<{ suiteId: string }> }) {
  const { suiteId } = await params; const { supabase, user } = await requireUser(); if (!user) notFound();
  const { data: suite } = await supabase.from('benchmark_suites').select('id, name').eq('id', suiteId).single(); if (!suite)
notFound(); // RLS hides other users' suites
  const { data: tasks } = await supabase.from('benchmark_tasks').select('id, number, title, weights, benchmark_runs(id,
platform, status, benchmark_grades(*))').eq('suite_id', suiteId).order('number');
  const cells = (tasks ?? []).flatMap((t: any) => PLATFORMS.map((p) => { const run = (t.benchmark_runs ?? []).find((r: any) =>
r.platform === p); const sheets = (run?.benchmark_grades ?? []).map((g: any) => ({ correctness: g.correctness, codeQuality:
g.code_quality, rendering: g.rendering, stateManagement: g.state_management, maintainability: g.maintainability, handoff:
g.handoff, sandboxFit: g.sandbox_fit, persistence: g.persistence })); return { task: t, platform: p as Platform, run, s:
aggregateGrades(sheets, t.weights ?? undefined) }; }));
  return (<div className="grid gap-8"><div className="flex items-end justify-between"><h1 className="text-2xl font-semibold">
{suite.name}</h1><div className="flex gap-2 text-sm"><a className="rounded border px-3 py-2" href={'/api/exports/benchmark?
suite=' + suiteId + '&format=csv'}>CSV</a><a className="rounded border px-3 py-2" href={'/api/exports/benchmark?suite=' +
suiteId + '&format=md'}>Markdown report</a></div></div>
    <Leaderboard title="Aggregate platform leaderboard" rows={leaderboard(cells.map((c) => ({ key: PLATFORM_LABEL[c.platform],
score: c.s.outOf5 })))} />
    <div className="overflow-x-auto rounded border"><table className="w-full text-sm"><thead className="bg-neutral-100"><tr><th
className="px-3 py-2 text-left">#</th><th className="px-3 py-2 text-left">Workflow</th>{PLATFORMS.map((p) => <th key={p}
className="px-3 py-2 text-left">{PLATFORM_LABEL[p]}</th>)}</tr></thead>
      <tbody>{(tasks ?? []).map((t: any) => <tr key={t.id} className="border-t"><td className="px-3 py-2 tabular-nums">
{t.number}</td><td className="px-3 py-2">{t.title}</td>{PLATFORMS.map((p) => { const c = cells.find((x) => x.task.id === t.id
&& x.platform === p)!; return <td key={p} className="px-3 py-2 tabular-nums">{!c.run ? 'No run' : c.run.status !== 'completed'
? c.run.status : c.s.outOf100 === null ? 'Not graded' : c.s.outOf100 + ' · ' + c.s.graders + ' grader' + (c.s.graders === 1 ?
'' : 's')}</td>; })}</tr>)}</tbody></table></div></div>);
}
```

handoff/workflow-router-app/app/benchmark/actions.ts · 21 lines

```
'use server';
import { revalidatePath } from 'next/cache';
import { requireUser } from '@/lib/supabase/server';
```

```
import { GradeSchema, RunSchema } from '@/lib/validation/benchmark';
export async function recordRun(input: unknown) {
  const p = RunSchema.safeParse(input); if (!p.success) return { ok: false as const, error: p.error.flatten() };
  const { supabase, user } = await requireUser(); if (!user) return { ok: false as const, error: 'Sign in to record runs.' };
  const d = p.data;
  const { data, error } = await supabase.from('benchmark_runs').insert({ task_id: d.taskId, platform: d.platform, created_by:
user.id, status: d.status, status_reason: d.statusReason ?? null, executed_prompt: d.executedPrompt, model_name: d.modelName ??
null, model_version: d.modelVersion ?? null, output_url: d.outputUrl || null, source_manifest: d.sourceManifest, runtime_notes:
d.runtimeNotes ?? null, console_notes: d.consoleNotes ?? null, duration_ms: d.durationMs ?? null, persistence_tested:
d.persistenceTested, persistence_survived_reload: d.persistenceSurvivedReload, persistence_mode: d.persistenceMode,
persistence_evidence: d.persistenceEvidence ?? null, persistence_failure_notes: d.persistenceFailureNotes ?? null
}).select('id').single();
  if (error) return { ok: false as const, error: error.message };
  revalidatePath('/benchmark'); return { ok: true as const, id: data.id };
}
export async function gradeRun(input: unknown) {
  const p = GradeSchema.safeParse(input); if (!p.success) return { ok: false as const, error: p.error.flatten() };
  const { supabase, user } = await requireUser(); if (!user) return { ok: false as const, error: 'Sign in to grade.' };
  const g = p.data;
  const { error } = await supabase.from('benchmark_grades').upsert({ run_id: g.runId, grader_id: user.id, correctness:
g.correctness, code_quality: g.codeQuality, rendering: g.rendering, state_management: g.stateManagement, maintainability:
g.maintainability, handoff: g.handoff, sandbox_fit: g.sandboxFit, persistence: g.persistence, evidence: g.evidence ?? null,
notes: g.notes ?? null }, { onConflict: 'run_id,grader_id' });
  if (error) return { ok: false as const, error: error.message };
  revalidatePath('/benchmark'); return { ok: true as const };
}
```

handoff/workflow-router-app/app/benchmark/page.tsx · 10 lines

```
import Link from 'next/link';
import { requireUser } from '@/lib/supabase/server';
export default async function Page() {
  const { supabase, user } = await requireUser();
  if (!user) return (<div className="grid gap-3"><h1 className="text-2xl font-semibold">Benchmark lab</h1><p>Sign in to create
suites, record runs and grade them. Benchmark data is private to you or your organization.</p><Link className="w-fit rounded
bg-slate-800 px-4 py-2 text-white" href="/auth/sign-in">Sign in</Link></div>);
  const { data: suites, error } = await supabase.from('benchmark_suites').select('id, name, description,
created_at').order('created_at', { ascending: false });
  return (<div className="grid gap-4"><h1 className="text-2xl font-semibold">Benchmark suites</h1>{error && <p
role="alert">Could not load suites: {error.message}</p>}
    {!suites?.length ? <p className="rounded border border-dashed p-6">No suites yet. Run <code>supabase/seed.sql</code> to
create the canonical 12.</p> : <ul className="grid gap-2">{suites.map((s) => <li key={s.id}><Link href={'/benchmark/' + s.id}
className="block rounded border bg-white p-4 hover:bg-neutral-50"><strong>{s.name}</strong><p className="text-sm text-neutral-
600">{s.description}</p></Link></li>)}</ul>}</div>);
}
```

handoff/workflow-router-app/app/globals.css · 4 lines

```
@import "tailwindcss";
:focus-visible { outline: 2px solid #597ea3; outline-offset: 2px; }
@media (prefers-reduced-motion: reduce) { * { animation: none !important; transition: none !important; } }
```

handoff/workflow-router-app/app/layout.tsx · 10 lines

```
import './globals.css';
import Link from 'next/link';
import { AuthButton } from '@/components/auth/auth-button';
export const metadata = { title: 'Semester AI Workflow Router & Benchmark Lab' };
export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (<html lang="en"><body className="bg-[#f4f1ea] text-[#1b1a17] antialiased"><a href="#main" className="sr-only
focus:not-sr-only">Skip to content</a>
    <header className="flex items-center justify-between bg-[#090a0e] px-6 py-3 text-[#eceef2]"><nav className="flex gap-5
text-sm" aria-label="Main"><Link href="/" className="font-semibold tracking-[.22em]">SEMESTER</Link><Link href="/workflow-
router">Workflow router</Link><Link href="/benchmark">Benchmark lab</Link></nav><AuthButton /></header>
    <main id="main" className="mx-auto max-w-[1280px] p-6">{children}</main></body></html>);
}
```

handoff/workflow-router-app/app/page.tsx · 6 lines

```
import Link from 'next/link';
export default function Home() {
  return (<div className="grid gap-6"><h1 className="text-3xl font-semibold">AI Workflow Router &amp;amp; Benchmark Lab</h1><p
className="max-w-[62ch] text-neutral-700">Route a Semester workflow to Claude Artifacts, ChatGPT Canvas or v0, compare their
limits, and record graded benchmark runs with evidence.</p>
    <div className="flex gap-3"><Link className="min-h-11 rounded bg-slate-800 px-4 py-3 text-white" href="/workflow-
router">Open the router</Link><Link className="min-h-11 rounded border px-4 py-3" href="/benchmark">Benchmark lab</Link></div>
</div>);
}
```

handoff/workflow-router-app/app/workflow-router/actions.ts · 17 lines

```
'use server';
import { revalidatePath } from 'next/cache';
import { requireUser } from '@/lib/supabase/server';
import { SaveRecommendationSchema } from '@/lib/validation/workflow-router';
import { recommend } from '@/lib/workflow-router/engine';
export async function saveRecommendation(input: unknown) {
  const parsed = SaveRecommendationSchema.safeParse(input);
  if (!parsed.success) return { ok: false as const, error: parsed.error.flatten() };
  const { supabase, user } = await requireUser();
  if (!user) return { ok: false as const, error: 'Sign in to save recommendations.' };
  const r = recommend(parsed.data.answers); // recomputed on the server; the client's result is never trusted
  const { error, data } = await supabase.from('workflow_recommendations').insert({ owner_id: user.id, organization_id:
parsed.data.organizationId, visibility: parsed.data.visibility, title: parsed.data.title, answers: parsed.data.answers,
primary_platform: r.primary, secondary_platform: r.secondary, confidence: r.confidence, result: r }).select('id').single();
  if (error) return { ok: false as const, error: error.message };
  revalidatePath('/workflow-router');
  return { ok: true as const, id: data.id };
}
```

handoff/workflow-router-app/app/workflow-router/page.tsx · 10 lines

```
import { requireUser } from '@/lib/supabase/server';
import { WorkflowRouter } from '@/components/workflow-router/workflow-router';
import { FeatureMatrix } from '@/components/workflow-router/feature-matrix';
import { PlatformMatrix } from '@/components/workflow-router/platform-matrix';
export default async function Page() {
  const { user } = await requireUser();
  return (<div className="grid gap-10"><header><p className="text-xs uppercase tracking-wider text-neutral-600">Workflow
router</p><h1 className="text-2xl font-semibold">Where should this be built?</h1></header>
    <WorkflowRouter signedIn={!!user} /><section><h2 className="mb-3 text-lg font-semibold">Feature matrix</h2><FeatureMatrix
/></section><section><h2 className="mb-3 text-lg font-semibold">Twelve canonical workflows</h2><PlatformMatrix /></section>
</div>);
}
```

handoff/workflow-router-app/components/auth/auth-button.tsx · 8 lines

```
import Link from 'next/link';
import { requireUser } from '@/lib/supabase/server';
export async function AuthButton() {
  const { user } = await requireUser();
  return user ? (<form action="/auth/sign-out" method="post" className="flex items-center gap-3 text-sm"><span>{user.email}
</span><button className="min-h-9 rounded border px-3">Sign out</button></form>)
    : (<div className="flex items-center gap-3 text-sm"><span className="rounded border border-dashed px-2 py-1">Prototype mode
— nothing is saved</span><Link href="/auth/sign-in" className="min-h-9 rounded bg-slate-800 px-3 py-2 text-white">Sign
in</Link></div>);
}
```

handoff/workflow-router-app/components/benchmark/benchmark-dashboard.tsx · 2 lines

```
export { Leaderboard as BenchmarkDashboard } from './leaderboard';
```

handoff/workflow-router-app/components/benchmark/benchmark-run-detail.tsx · 11 lines

```
import { PLATFORM_LABEL, type Platform, type Weights } from '@/lib/benchmark/benchmark.config';
import { persistenceCeiling } from '@/lib/benchmark/probes/persistence';
```

```
import { GradingForm } from './grading-form';
export function BenchmarkRunDetail({ run, weights, requiresPersistence }: { run: any; weights?: Weights; requiresPersistence:
boolean }) {
  const ceiling = persistenceCeiling({ tested: run.persistence_tested, survivedReload: run.persistence_survived_reload, mode:
run.persistence_mode, evidence: run.persistence_evidence ?? undefined }, requiresPersistence);
  return (<article className="grid gap-4 rounded border bg-white p-5"><h3 className="text-lg font-semibold">
{PLATFORM_LABEL[run.platform as Platform]} · {run.status}</h3>
    <dl className="grid grid-cols-[160px_1fr] gap-2 text-sm"><dt>Model</dt><dd>{run.model_name ?? '—'}</dd><dt>Output</dt><dd>
{run.output_url ? <a href={run.output_url} className="underline">{run.output_url}</a> : '—'}</dd><dt>Duration</dt><dd
className="tabular-nums">{run.duration_ms ?? '—'} ms</dd><dt>Persistence</dt><dd>{run.persistence_tested ?
(run.persistence_survived_reload ? 'Survived reload' : 'Lost on reload') + ' · ' + run.persistence_mode : 'Not tested'}</dd>
<dt>Status reason</dt><dd>{run.status_reason ?? '—'}</dd></dl>
    <details><summary className="cursor-pointer text-sm">Exact prompt</summary><pre className="whitespace-pre-wrap font-mono
text-xs">{run.executed_prompt}</pre></details>
    <GradingForm runId={run.id} weights={weights} persistenceCeiling={ceiling} /></article>);
}
```

handoff/workflow-router-app/components/benchmark/grading-form.tsx · 19 lines

```
'use client';
import { useState, useTransition } from 'react';
import { CRITERIA, type Weights } from '@/lib/benchmark/benchmark.config';
import { weightedScore, type Grades } from '@/lib/benchmark/scoring';
import { gradeRun } from '@/app/benchmark/actions';
const LABEL: Record<string, string> = { correctness: 'Correctness', codeQuality: 'Code quality', rendering: 'Rendering',
stateManagement: 'State management', maintainability: 'Maintainability', handoff: 'Handoff/export', sandboxFit: 'Sandbox fit',
persistence: 'Persistence' };
export function GradingForm({ runId, weights, persistenceCeiling }: { runId: string; weights?: Weights; persistenceCeiling:
number }) {
  const [g, setG] = useState<Grades>({}); const [evidence, setEvidence] = useState(''); const [notes, setNotes] = useState('');
const [said, setSaid] = useState(''); const [pending, start] = useTransition();
  const s = weightedScore(g, weights);
  return (<form className="grid gap-3" onSubmit={(e) => { e.preventDefault(); start(async () => { const r = await gradeRun({
runId, ...Object.fromEntries(CRITERIA.map((c) => [c, g[c] ?? null])), evidence, notes }); setSaid(r.ok ? 'Grade saved.' : 'Not
saved — check the fields.'); }); }}>
    {CRITERIA.map((c) => (<fieldset key={c} className="flex items-center gap-3"><legend className="w-40 text-sm">{LABEL[c]}
</legend>
      {[0, 1, 2, 3, 4, 5].map((n) => { const capped = c === 'persistence' && n > persistenceCeiling; return (<label key={n}
className={'inline-flex min-h-9 min-w-9 items-center justify-center rounded border text-sm ' + (g[c] === n ? 'bg-slate-800
text-white' : '') + (capped ? ' opacity-40' : '')}><input type="radio" className="sr-only" name={c} disabled={capped} checked=
{g[c] === n} onChange={() => setG({ ...g, [c]: n })} />{n}</label>); })}</fieldset>))}
    <p className="text-xs text-neutral-600">Persistence is capped at {persistenceCeiling} by the recorded reload test.</p>
    <label className="grid gap-1 text-sm">Evidence<textarea required value={evidence} onChange={(e) =>
setEvidence(e.target.value)} className="min-h-20 rounded border p-2" /></label>
    <label className="grid gap-1 text-sm">Notes<textarea value={notes} onChange={(e) => setNotes(e.target.value)}
className="min-h-16 rounded border p-2" /></label>
    <p className="tabular-nums text-sm">Weighted: {s.outOf5 ?? '—'} / 5 · {s.outOf100 ?? '—'} / 100 {s.missing.length ? '·
missing ' + s.missing.join(', ') : ''}</p>
    <button disabled={pending} className="min-h-10 rounded bg-slate-800 px-4 text-sm text-white">Save grade</button><p
role="status" className="text-sm">{said}</p></form>);
}
```

handoff/workflow-router-app/components/benchmark/leaderboard.tsx · 6 lines

```
import type { LeaderRow } from '@/lib/benchmark/scoring';
export function Leaderboard({ title, rows }: { title: string; rows: LeaderRow[] }) {
  return (<section><h2 className="mb-2 text-lg font-semibold">{title}</h2><table className="w-full text-sm"><thead><tr
className="text-left text-xs uppercase tracking-wider text-neutral-600"><th>#</th><th>Platform</th><th>Score /5</th>
<th>Graded</th></tr></thead>
    <tbody>{rows.map((r, i) => <tr key={r.key} className="border-t"><td className="py-2 tabular-nums">{i + 1}</td><td>{r.key}
</td><td className="tabular-nums">{r.score ?? 'Not graded'}</td><td className="tabular-nums">{r.graded} / {r.tasks}</td></tr>)}
</tbody></table></section>);
}
```

handoff/workflow-router-app/components/benchmark/run-form.tsx · 29 lines

```
'use client';
import { useState, useTransition } from 'react';
import { PERSISTENCE_MODES, PLATFORMS, PLATFORM_LABEL, type WorkflowSpec } from '@/lib/benchmark/benchmark.config';
import { recordRun } from '@/app/benchmark/actions';
import { capabilityGuard } from '@/lib/benchmark/adapters/capability-guard';
```

```
export function RunForm({ taskId, task }: { taskId: string; task: WorkflowSpec }) {
  const [platform, setPlatform] = useState<(typeof PLATFORMS)[number]>(task.recommended); const guard =
capabilityGuard(platform, task);
  const [said, setSaid] = useState(''); const [pending, start] = useTransition();
  return (<form className="grid gap-3" onSubmit={(e) => { e.preventDefault(); const f = new FormData(e.currentTarget);
start(async () => {
      const r = await recordRun({ taskId, platform, executedPrompt: f.get('prompt'), modelName: f.get('model') || undefined,
status: f.get('status'), statusReason: f.get('reason') || undefined, outputUrl: f.get('url') || '', runtimeNotes:
f.get('runtime') || undefined, consoleNotes: f.get('console') || undefined, durationMs: f.get('ms') ? Number(f.get('ms')) :
undefined, sourceManifest: String(f.get('manifest') || '').split('\n').filter(Boolean).map((path) => ({ path })),
persistenceTested: f.get('ptested') === 'on', persistenceSurvivedReload: f.get('psurvived') === 'yes' ? true :
f.get('psurvived') === 'no' ? false : null, persistenceMode: f.get('pmode'), persistenceEvidence: f.get('pevidence') ||
undefined });
      setSaid(r.ok ? 'Run recorded.' : 'Not recorded: ' + JSON.stringify(r.error)); }); }}>
    <label className="grid gap-1 text-sm">Platform<select value={platform} onChange={(e) => setPlatform(e.target.value as
typeof platform)} className="min-h-10 rounded border px-2">{PLATFORMS.map((p) => <option key={p} value={p}>{PLATFORM_LABEL[p]}
</option>)}</select></label>
    {!guard.ok && <p role="status" className="rounded border border-amber-600/50 bg-amber-50 p-2 text-sm">Unsupported:
{guard.reason}</p>}
    <label className="grid gap-1 text-sm">Exact executed prompt<textarea name="prompt" required defaultValue={task.prompt}
className="min-h-24 rounded border p-2 font-mono text-xs" /></label>
    <div className="grid grid-cols-2 gap-3"><label className="grid gap-1 text-sm">Model name/version<input name="model"
className="min-h-10 rounded border px-2" /></label>
      <label className="grid gap-1 text-sm">Status<select name="status" defaultValue={guard.ok ? 'manual-review-required' :
'unsupported'} className="min-h-10 rounded border px-2">{['completed', 'manual-review-required', 'unsupported',
'failed'].map((s) => <option key={s}>{s}</option>)}</select></label></div>
    <label className="grid gap-1 text-sm">Status reason<input name="reason" defaultValue={guard.ok ? '' : guard.reason}
className="min-h-10 rounded border px-2" /></label>
    <label className="grid gap-1 text-sm">Output URL<input name="url" type="url" className="min-h-10 rounded border px-2" />
</label>
    <label className="grid gap-1 text-sm">Source/file manifest (one path per line)<textarea name="manifest" className="min-h-16
rounded border p-2 font-mono text-xs" /></label>
    <div className="grid grid-cols-3 gap-3"><label className="grid gap-1 text-sm">Duration (ms)<input name="ms" type="number"
min={0} className="min-h-10 rounded border px-2" /></label><label className="col-span-2 grid gap-1 text-sm">Runtime/sandbox
notes<input name="runtime" className="min-h-10 rounded border px-2" /></label></div>
    <label className="grid gap-1 text-sm">Console/log notes<textarea name="console" className="min-h-16 rounded border p-2
font-mono text-xs" /></label>
    <fieldset className="grid gap-2 rounded border p-3"><legend className="px-1 text-sm font-medium">Persistence
test{task.requiresPersistence ? ' (required)' : ''}</legend>
      <label className="text-sm"><input type="checkbox" name="ptested" defaultChecked={task.requiresPersistence} /> Tested
reload/refresh</label>
      <label className="grid gap-1 text-sm">Survived reload?<select name="psurvived" className="min-h-10 rounded border px-2">
<option value="">Not recorded</option><option value="yes">Yes</option><option value="no">No</option></select></label>
      <label className="grid gap-1 text-sm">Storage mode<select name="pmode" className="min-h-10 rounded border px-2">
{PERSISTENCE_MODES.map((m) => <option key={m}>{m}</option>)}</select></label>
      <label className="grid gap-1 text-sm">Evidence (URL, row id, screenshot or log)<input name="pevidence" className="min-h-
10 rounded border px-2" /></label></fieldset>
    <button disabled={pending} className="min-h-10 rounded bg-slate-800 px-4 text-sm text-white">Record run</button><p
role="status" className="text-sm">{said}</p></form>);
}
```

handoff/workflow-router-app/components/benchmark/workflow-matrix.tsx · 2 lines

```
export { PlatformMatrix as WorkflowMatrix } from '../workflow-router/platform-matrix';
```

handoff/workflow-router-app/components/workflow-router/decision-tree.tsx · 18 lines

```
'use client';
import type { RouterAnswers } from '@/lib/validation/workflow-router';
export const QUESTIONS: { key: keyof RouterAnswers; label: string; options: [string, string][] }[] = [
  { key: 'deliverable', label: 'Deliverable', options: [['interactive-tool', 'Interactive tool'], ['prototype', 'Product
prototype'], ['marketing-page', 'Marketing page'], ['diagram', 'Diagram'], ['document', 'Document'], ['presentation',
'Presentation'], ['data-analysis', 'Data analysis'], ['react-component', 'React component'], ['full-stack', 'Full-stack
feature']] },
  { key: 'interaction', label: 'Interaction', options: [['none', 'None'], ['simple-local', 'Simple local interaction'],
['complex-local', 'Complex local state'], ['multi-user', 'Multi-user collaboration']] },
  { key: 'backend', label: 'Backend', options: [['none', 'None'], ['file-export', 'File export'], ['database', 'Database'],
['server-api', 'Server API'], ['auth-secrets', 'Auth/secrets']] },
  { key: 'technology', label: 'Technology target', options: [['none', 'None'], ['html', 'HTML/CSS/JavaScript'], ['react',
'React'], ['nextjs', 'Next.js'], ['nextjs-postgres', 'Next.js + Postgres'], ['nextjs-supabase', 'Next.js + Supabase/Auth']] },
  { key: 'persistence', label: 'Persistence/access', options: [['none', 'No persistence'], ['database', 'Database'], ['auth',
'Auth'], ['storage', 'Storage'], ['realtime', 'Realtime'], ['multi-tenant', 'Multi-tenant organization workspace']] },
  { key: 'handoff', label: 'Handoff', options: [['share', 'Share immediately'], ['document-export', 'Document/source export'],
```

```
['github', 'GitHub'], ['vercel', 'Deploy to Vercel']] },
];
export function DecisionTree({ value, onChange }: { value: RouterAnswers; onChange: (v: RouterAnswers) => void }) {
  return (<div className="grid gap-5">{QUESTIONS.map((q) => (
    <fieldset key={q.key}><legend className="mb-2 text-xs font-medium uppercase tracking-wider text-neutral-600">{q.label}
</legend>
      <div className="flex flex-wrap gap-2">{q.options.map(([v, l]) => (
        <label key={v} className={'cursor-pointer rounded border px-3 py-2 text-sm min-h-10 inline-flex items-center focus-
within:ring-2 focus-within:ring-slate-500 ' + (value[q.key] === v ? 'border-slate-700 bg-slate-800 text-white' : 'border-
neutral-300 bg-white hover:bg-neutral-50')}>
          <input type="radio" className="sr-only" name={q.key} value={v} checked={value[q.key] === v} onChange={() =>
onChange({ ...value, [q.key]: v } as RouterAnswers)} />{l}</label>))}</div></fieldset>))}</div>);
}
```

handoff/workflow-router-app/components/workflow-router/feature-matrix.tsx · 19 lines

```
'use client';
import { useMemo, useState } from 'react';
export const FEATURES = [
  { platform: 'Claude Artifacts', primaryUse: 'Self-contained interactive artifacts, diagrams, prototypes', execution: 'Client-
side sandbox beside the chat', sandbox: 'Approved CDNs only; localStorage/fetch may be restricted', react: 4, clientState: 4,
persistent: 1, multiFile: 1, server: 0, database: 0, auth: 0, docExport: 3, codeExport: 4, github: 1, deploy: 1, semester:
'Decision tools, artifact catalog, architecture diagrams' },
  { platform: 'ChatGPT Canvas', primaryUse: 'Revision-centred writing and code iteration', execution: 'Isolated React/HTML
preview; Python execution', sandbox: 'Isolated preview; not a hosting environment', react: 3, clientState: 3, persistent: 1,
multiFile: 2, server: 1, database: 0, auth: 0, docExport: 5, codeExport: 3, github: 1, deploy: 0, semester: 'PRDs, TDDs, legal
first drafts, CSV analysis' },
  { platform: 'v0', primaryUse: 'Multi-file Next.js apps with server logic', execution: 'VM-backed Node.js sandbox', sandbox:
'Isolated VM; runs server code, API routes, DB connections', react: 5, clientState: 5, persistent: 4, multiFile: 5, server: 5,
database: 4, auth: 4, docExport: 1, codeExport: 5, github: 5, deploy: 5, semester: 'Authenticated, database-backed Semester
features' },
] as const;
type Row = (typeof FEATURES)[number]; type Key = keyof Row;
const COLS: [Key, string][] = [['platform', 'Platform'], ['primaryUse', 'Primary use'], ['execution', 'Execution model'],
['sandbox', 'Sandbox limits'], ['react', 'React'], ['clientState', 'Client state'], ['persistent', 'Persistent state'],
['multiFile', 'Multi-file'], ['server', 'Server/API'], ['database', 'Database'], ['auth', 'Auth'], ['docExport', 'Doc export'],
['codeExport', 'Code export'], ['github', 'GitHub'], ['deploy', 'Deploy'], ['semester', 'Best Semester use']];
const word = (n: number) => ['None', 'Minimal', 'Partial', 'Good', 'Strong', 'Native'][n];
export function FeatureMatrix() {
  const [sort, setSort] = useState<{ key: Key; dir: 1 | -1 }>({ key: 'platform', dir: 1 }); const [q, setQ] = useState('');
  const rows = useMemo(() => [...FEATURES].filter((r) => JSON.stringify(r).toLowerCase().includes(q.toLowerCase())).sort((a, b)
=> (a[sort.key] > b[sort.key] ? 1 : a[sort.key] < b[sort.key] ? -1 : 0) * sort.dir), [sort, q]);
  return (<div><label className="mb-2 block text-sm">Search <input value={q} onChange={(e) => setQ(e.target.value)}
className="ml-2 min-h-9 rounded border px-2" /></label>
    <div className="overflow-x-auto rounded-md border" role="region" aria-label="Feature matrix" tabIndex={0}><table
className="min-w-[1400px] text-sm"><thead className="bg-neutral-100"><tr>{COLS.map(([k, l]) => (
      <th key={k} scope="col" aria-sort={sort.key === k ? (sort.dir === 1 ? 'ascending' : 'descending') : 'none'}
className="px-3 py-2 text-left text-xs uppercase tracking-wider"><button type="button" onClick={() => setSort((s) => ({ key: k,
dir: s.key === k ? (s.dir === 1 ? -1 : 1) : 1 }))}>{l}{sort.key === k ? (sort.dir === 1 ? ' ↑' : ' ↓') : ''}</button></th>))}
</tr></thead>
      <tbody>{rows.map((r) => <tr key={r.platform} className="border-t">{COLS.map(([k]) => <td key={k} className="px-3 py-2
align-top">{typeof r[k] === 'number' ? <span className="tabular-nums">{word(r[k] as number)} ({r[k]}/5)</span> : String(r[k])}
</td>)}</tr>)}</tbody></table></div></div>);
}
```

handoff/workflow-router-app/components/workflow-router/platform-matrix.tsx · 7 lines

```
'use client';
import { WORKFLOWS, PLATFORM_LABEL } from '@/lib/benchmark/benchmark.config';
export function PlatformMatrix() {
  return (<div className="overflow-x-auto rounded-md border"><table className="w-full text-sm"><thead className="bg-neutral-
100"><tr><th className="px-3 py-2 text-left">#</th><th className="px-3 py-2 text-left">Workflow</th><th className="px-3 py-2
text-left">Category</th><th className="px-3 py-2 text-left">Recommended</th><th className="px-3 py-2 text-
left">Persistence</th></tr></thead>
    <tbody>{WORKFLOWS.map((w) => <tr key={w.number} className="border-t"><td className="px-3 py-2 tabular-nums">{w.number}</td>
<td className="px-3 py-2">{w.title}</td><td className="px-3 py-2">{w.category}</td><td className="px-3 py-2">
{PLATFORM_LABEL[w.recommended]}</td><td className="px-3 py-2">{w.requiresPersistence ? 'Required' : '—'}</td></tr>)}</tbody>
</table></div>);
}
```

handoff/workflow-router-app/components/workflow-router/prompt-builder.tsx · 9 lines

```
'use client';
import { useState } from 'react';
export function CopyBlock({ label, text }: { label: string; text: string }) {
  const [said, setSaid] = useState('');
  return (<div className="rounded-md border border-neutral-300 bg-white"><div className="flex items-center justify-between
border-b px-4 py-2"><h3 className="text-sm font-semibold">{label}</h3>
    <button type="button" className="min-h-9 rounded border px-3 text-sm" onClick={async () => { try { await
navigator.clipboard.writeText(text); setSaid('Copied'); } catch { setSaid('Copy failed — select the text'); } setTimeout(() =>
setSaid(''), 1500); }}>Copy</button></div>
    <pre className="max-h-72 overflow-auto whitespace-pre-wrap p-4 font-mono text-xs">{text}</pre><span role="status"
className="sr-only">{said}</span>{said && <p className="px-4 pb-2 text-xs">{said}</p>}</div>);
}
```

handoff/workflow-router-app/components/workflow-router/recommendation-panel.tsx · 15 lines

```
'use client';
import { PLATFORM_LABEL } from '@/lib/benchmark/benchmark.config';
import type { Recommendation } from '@/lib/workflow-router/engine';
export function RecommendationPanel({ r }: { r: Recommendation }) {
  return (<section aria-labelledby="rec-h" className="rounded-md border border-neutral-300 bg-white p-5">
    <p className="text-xs uppercase tracking-wider text-neutral-600">Recommendation · confidence {Math.round(r.confidence *
100)}%</p>
    <h2 id="rec-h" className="mt-1 text-2xl font-semibold">{PLATFORM_LABEL[r.primary]} <span className="text-base font-normal
text-neutral-600">then {PLATFORM_LABEL[r.secondary]}</span></h2>
    {r.escalation && <p role="status" className="mt-3 rounded border border-amber-600/50 bg-amber-50 px-3 py-2 text-sm text-
amber-900"><strong>! Production escalation:</strong> {r.escalationReasons.join('; ')}</p>}
    <h3 className="mt-4 text-sm font-semibold">Why</h3><ul className="mt-1 list-disc pl-5 text-sm">
{r.reasons[r.primary].map((x) => <li key={x}>{x}</li>)}</ul>
    <h3 className="mt-4 text-sm font-semibold">Limitations</h3><ul className="mt-1 list-disc pl-5 text-sm text-neutral-700">
{r.limitations.map((x) => <li key={x}>{x}</li>)}</ul>
    <p className="mt-4 text-sm"><strong>Handoff:</strong> {r.handoffFormat}</p>
    <div className="mt-3 flex gap-3 text-xs text-neutral-600 tabular-nums">{Object.entries(r.scores).map(([p, s]) => <span key=
{p}>{PLATFORM_LABEL[p as keyof typeof PLATFORM_LABEL]} {s}</span>)}</div>
  </section>);
}
```

handoff/workflow-router-app/components/workflow-router/workflow-router.tsx · 27 lines

```
'use client';
import { useMemo, useState, useTransition } from 'react';
import type { RouterAnswers } from '@/lib/validation/workflow-router';
import { recommend } from '@/lib/workflow-router/engine';
import { buildPrompt, githubPlan, supabasePrompt } from '@/lib/workflow-router/prompts';
import { recommendationMarkdown } from '@/lib/exports/markdown';
import { toJson } from '@/lib/exports/json';
import { DecisionTree } from './decision-tree';
import { RecommendationPanel } from './recommendation-panel';
import { CopyBlock } from './prompt-builder';
import { saveRecommendation } from '@/app/workflow-router/actions';
const DEFAULT: RouterAnswers = { deliverable: 'interactive-tool', interaction: 'complex-local', backend: 'file-export',
technology: 'react', persistence: 'none', handoff: 'share' };
const download = (name: string, type: string, body: string) => { const a = document.createElement('a'); a.href =
URL.createObjectURL(new Blob([body], { type })); a.download = name; a.click(); URL.revokeObjectURL(a.href); };
export function WorkflowRouter({ signedIn }: { signedIn: boolean }) {
  const [a, setA] = useState<RouterAnswers>(DEFAULT); const r = useMemo(() => recommend(a), [a]); const prompt = buildPrompt(a,
r);
  const [said, setSaid] = useState(''); const [pending, start] = useTransition();
  return (<div className="grid gap-6 lg:grid-cols-[minmax(0,1fr)_minmax(0,1.1fr)]">
    <DecisionTree value={a} onChange={setA} />
    <div className="grid content-start gap-4"><RecommendationPanel r={r} />
      <div className="flex flex-wrap gap-2"><button className="min-h-10 rounded border px-3 text-sm" onClick={() =>
download('semester-recommendation.md', 'text/markdown', recommendationMarkdown(a, r, prompt))}>Download Markdown</button>
        <button className="min-h-10 rounded border px-3 text-sm" onClick={() => download('semester-recommendation.json',
'application/json', toJson({ answers: a, recommendation: r, prompt }))}>Download JSON</button>
        <button disabled={!signedIn || pending} title={signedIn ? '' : 'Sign in to save'} className="min-h-10 rounded bg-slate-
800 px-3 text-sm text-white disabled:opacity-50" onClick={() => start(async () => { const res = await saveRecommendation({
title: a.deliverable + ' → ' + r.primary, organizationId: null, answers: a, visibility: 'private' }); setSaid(res.ok ? 'Saved
to your recommendations.' : typeof res.error === 'string' ? res.error : 'Check the inputs.'); })}>{signedIn ? 'Save
recommendation' : 'Sign in to save'}</button></div>
      <p role="status" className="text-sm">{said}</p>
      <CopyBlock label="Build prompt" text={prompt} /><CopyBlock label="GitHub handoff plan" text={githubPlan(r).map((x, i) =>
i + 1 + '. ' + x).join('\n')} />
```

```
      {(a.technology === 'nextjs-supabase' || a.persistence !== 'none') && <CopyBlock label="Supabase implementation prompt"
text={supabasePrompt()} />}</div></div>);
}
```

handoff/workflow-router-app/install.sh · 6 lines

```
#!/bin/sh
set -e
find . -name "*.txt" ! -name "LICENSE.txt" | while read -r f; do mv "$f" "${f%.txt}"; done
[ -d app/benchmark/-suiteId- ] && mv app/benchmark/-suiteId- "app/benchmark/[suiteId]"
echo "Restored. Next: npm install && npm run typecheck && npm test && npm run build"
```

handoff/workflow-router-app/lib/benchmark/adapters/capability-guard.ts · 7 lines

```
import type { Platform, WorkflowSpec } from '../benchmark.config';
/** Honest boundaries: what each platform can be asked to do at all. Execution itself is always manual — no approved automation
API is assumed. */
export function capabilityGuard(platform: Platform, task: WorkflowSpec): { ok: true } | { ok: false; status: 'unsupported';
reason: string } {
  if (platform !== 'v0' && task.requiresPersistence) return { ok: false, status: 'unsupported', reason: (platform === 'claude-
artifacts' ? 'Claude Artifacts' : 'ChatGPT Canvas') + ' has no durable backend, auth or secret store; this task needs ' +
task.requirements.join(', ') + '. Record a run only to document the gap.' };
  return { ok: true };
}
```

handoff/workflow-router-app/lib/benchmark/adapters/chatgpt-canvas.adapter.ts · 12 lines

```
import type { BenchmarkAdapter, AdapterRun } from './types';
import { capabilityGuard } from './capability-guard';
const MANUAL = 'No approved programmatic integration for ChatGPT Canvas is assumed. Run the prompt in Canvas, export the
document/code, and attach the file manifest and a share link.';
export const chatgptCanvasAdapter: BenchmarkAdapter = {
  platform: 'chatgpt-canvas',
  canRun(task) { const g = capabilityGuard('chatgpt-canvas', task); return g.ok ? { ok: false, status: 'manual-review-
required', reason: MANUAL } : g; },
  async run(task) { const g = this.canRun(task); return { platform: 'chatgpt-canvas', status: g.ok ? 'completed' : g.status,
reason: g.ok ? undefined : g.reason, executedPrompt: task.prompt }; },
  collectEvidence(run: AdapterRun) { return { outputUrl: run.outputUrl, manifest: run.manifest ?? [], notes: [run.consoleNotes
?? '', run.reason ?? ''].filter(Boolean) }; },
  evaluatePersistence() { return { tested: false, survivedReload: null, mode: 'local-preview' }; },
  reportUnsupported(reason) { return { platform: 'chatgpt-canvas', status: 'unsupported', reason, executedPrompt: '' }; },
};
```

handoff/workflow-router-app/lib/benchmark/adapters/claude-artifacts.adapter.ts · 12 lines

```
import type { BenchmarkAdapter, AdapterRun } from './types';
import { capabilityGuard } from './capability-guard';
const MANUAL = 'No approved programmatic integration for creating Claude Artifacts is assumed. Paste the exact prompt into
claude.ai, publish the artifact, and record its URL, source and console notes.';
export const claudeArtifactsAdapter: BenchmarkAdapter = {
  platform: 'claude-artifacts',
  canRun(task) { const g = capabilityGuard('claude-artifacts', task); return g.ok ? { ok: false, status: 'manual-review-
required', reason: MANUAL } : g; },
  async run(task) { const g = this.canRun(task); return { platform: 'claude-artifacts', status: g.ok ? 'completed' : g.status,
reason: g.ok ? undefined : g.reason, executedPrompt: task.prompt }; },
  collectEvidence(run: AdapterRun) { return { outputUrl: run.outputUrl, manifest: run.manifest ?? [], notes: [run.consoleNotes
?? '', run.reason ?? ''].filter(Boolean) }; },
  evaluatePersistence() { return { tested: false, survivedReload: null, mode: 'local-preview' }; },
  reportUnsupported(reason) { return { platform: 'claude-artifacts', status: 'unsupported', reason, executedPrompt: '' }; },
};
```

handoff/workflow-router-app/lib/benchmark/adapters/types.ts · 14 lines

```
import type { Platform, WorkflowSpec } from '../benchmark.config';
import type { PersistenceObservation } from '../probes/persistence';
export type RunStatus = 'completed' | 'manual-review-required' | 'unsupported' | 'failed';
export interface AdapterRun { platform: Platform; status: RunStatus; reason?: string; executedPrompt: string; outputUrl?:
string; manifest?: { path: string; bytes?: number }[]; consoleNotes?: string; durationMs?: number }
```

```
export interface Evidence { outputUrl?: string; manifest: { path: string }[]; notes: string[] }
export interface BenchmarkAdapter {
  platform: Platform;
  canRun(task: WorkflowSpec): { ok: true } | { ok: false; status: 'manual-review-required' | 'unsupported'; reason: string };
  run(task: WorkflowSpec): Promise<AdapterRun>;
  collectEvidence(run: AdapterRun): Evidence;
  evaluatePersistence(run: AdapterRun): PersistenceObservation;
  reportUnsupported(reason: string): AdapterRun;
}
```

handoff/workflow-router-app/lib/benchmark/adapters/v0.adapter.ts · 12 lines

```
import type { BenchmarkAdapter, AdapterRun } from './types';
import { capabilityGuard } from './capability-guard';
const MANUAL = 'Run the prompt in v0, push to GitHub, and record the repo/deployment URL, file manifest and test log.
Persistence is graded only from a reload test with a row id or log.';
export const v0Adapter: BenchmarkAdapter = {
  platform: 'v0',
  canRun(task) { const g = capabilityGuard('v0', task); return g.ok ? { ok: false, status: 'manual-review-required', reason:
MANUAL } : g; },
  async run(task) { const g = this.canRun(task); return { platform: 'v0', status: g.ok ? 'completed' : g.status, reason: g.ok ?
undefined : g.reason, executedPrompt: task.prompt }; },
  collectEvidence(run: AdapterRun) { return { outputUrl: run.outputUrl, manifest: run.manifest ?? [], notes: [run.consoleNotes
?? '', run.reason ?? ''].filter(Boolean) }; },
  evaluatePersistence() { return { tested: false, survivedReload: null, mode: 'project-backed' }; },
  reportUnsupported(reason) { return { platform: 'v0', status: 'unsupported', reason, executedPrompt: '' }; },
};
```

handoff/workflow-router-app/lib/benchmark/benchmark.config.ts · 56 lines

```
export const PLATFORMS = ['claude-artifacts', 'chatgpt-canvas', 'v0'] as const;
export type Platform = (typeof PLATFORMS)[number];
export const PLATFORM_LABEL: Record<Platform, string> = { 'claude-artifacts': 'Claude Artifacts', 'chatgpt-canvas': 'ChatGPT
Canvas', v0: 'v0' };

export const CRITERIA = ['correctness', 'codeQuality', 'rendering', 'stateManagement', 'maintainability', 'handoff',
'sandboxFit', 'persistence'] as const;
export type Criterion = (typeof CRITERIA)[number];
export type Weights = Partial<Record<Criterion, number>>;

/** Default weights (sum 1.0). Persistence is graded but carries 0 weight unless a task overrides it. */
export const DEFAULT_WEIGHTS: Weights = { correctness: 0.25, codeQuality: 0.2, rendering: 0.15, stateManagement: 0.15,
maintainability: 0.1, handoff: 0.1, sandboxFit: 0.05, persistence: 0 };

export const PERSISTENCE_MODES = ['none', 'local-preview', 'browser-local', 'session', 'database', 'project-backed'] as const;
export type PersistenceMode = (typeof PERSISTENCE_MODES)[number];

export interface WorkflowSpec { number: number; slug: string; title: string; category: string; prompt: string; requirements:
string[]; acceptance: string[]; recommended: Platform; requiresPersistence: boolean; weights?: Weights }

const P = (w: Weights) => w;
export const WORKFLOWS: WorkflowSpec[] = [
  { number: 1, slug: 'artifact-tracker', title: 'Sortable artifact tracker', category: 'Interactive tool', recommended:
'claude-artifacts', requiresPersistence: false,
    prompt: 'Build a single-file React tracker of every Semester artifact with search, category/format/priority filters,
sortable columns and CSV export. No external services.',
    requirements: ['Search', 'Multi-filter', 'Sortable columns', 'CSV export'], acceptance: ['Sort is stable per column',
'Filters combine with AND', 'CSV reflects current view'] },
  { number: 2, slug: 'onboarding-prototype', title: 'Interactive onboarding prototype', category: 'Product prototype',
recommended: 'claude-artifacts', requiresPersistence: false,
    prompt: 'Prototype Semester student onboarding: add a syllabus, connect a calendar, or use sample data; consent one
decision at a time; first next step with its source label.',
    requirements: ['3 entry paths', 'Consent steps', 'Back/forward', 'Keyboard reachable'], acceptance: ['Every step reachable
by keyboard', 'Decline is equal weight'] },
  { number: 3, slug: 'b2b-landing', title: 'B2B SaaS landing page', category: 'Marketing page', recommended: 'claude-
artifacts', requiresPersistence: false,
    prompt: 'Design a Semester institution landing page: one clear next step, status of every capability (use now / pilot /
later), no unapproved claims.',
    requirements: ['Hero', 'Capability status map', 'Trust section', 'Responsive'], acceptance: ['No prohibited claims',
'Passes 320px reflow'] },
  { number: 4, slug: 'architecture-diagram', title: 'Multi-tenant system architecture diagram', category: 'Diagram',
recommended: 'claude-artifacts', requiresPersistence: false,
    prompt: 'Draw Semester multi-tenant architecture as Mermaid: tenants, Supabase Postgres with RLS, edge functions, SIS/LMS
```

```
connectors, audit log.',
    requirements: ['Mermaid source', 'Tenant boundary', 'Data flows labelled'], acceptance: ['Renders without errors', 'RLS
boundary explicit'] },
  { number: 5, slug: 'tech-design-doc', title: 'Technical design document', category: 'Document', recommended: 'chatgpt-
canvas', requiresPersistence: false,
    prompt: 'Write a technical design document for Semester degree-audit sync: context, goals, non-goals, design, data model,
risks, rollout, open questions.',
    requirements: ['Standard TDD sections', 'Data model', 'Risks'], acceptance: ['Exports to DOCX/PDF/Markdown', 'Revision
history usable'], weights: P({ correctness: 0.3, codeQuality: 0.05, rendering: 0.05, stateManagement: 0, maintainability: 0.2,
handoff: 0.35, sandboxFit: 0.05 }) },
  { number: 6, slug: 'launch-presentation', title: 'Product launch presentation', category: 'Presentation', recommended:
'chatgpt-canvas', requiresPersistence: false,
    prompt: 'Draft a 10-slide Semester pilot launch presentation for a registrar audience: scope, status, what stays official,
evidence, next step.',
    requirements: ['10 slides', 'Speaker notes', 'Status honesty'], acceptance: ['Exports a usable deck or outline'], weights:
P({ correctness: 0.3, codeQuality: 0, rendering: 0.25, stateManagement: 0, maintainability: 0.1, handoff: 0.3, sandboxFit: 0.05
}) },
  { number: 7, slug: 'subscription-csv', title: 'Subscription CSV analysis', category: 'Data analysis', recommended: 'chatgpt-
canvas', requiresPersistence: false,
    prompt: 'Analyse the attached subscription CSV: MRR by month, churn, cohort retention; state assumptions; export cleaned
CSV and a chart.',
    requirements: ['Python execution', 'Cohort table', 'Chart', 'Cleaned export'], acceptance: ['Numbers reproducible from
code', 'Assumptions listed'] },
  { number: 8, slug: 'metric-dashboard', title: 'Reusable React metric dashboard', category: 'React component', recommended:
'claude-artifacts', requiresPersistence: false,
    prompt: 'Build a reusable <MetricDashboard> React component: typed props, metric tiles with source and age, provenance-
encoded bars, empty/loading/error states.',
    requirements: ['Typed props', 'States', 'Accessible chart table'], acceptance: ['Component is self-contained', 'All states
render'] },
  { number: 9, slug: 'auth-route-handler', title: 'Authenticated Next.js route handler', category: 'Full-stack feature',
recommended: 'v0', requiresPersistence: true,
    prompt: 'Implement app/api/recommendations/route.ts: Supabase server client, auth.getUser(), Zod validation, insert scoped
to the user, 401/422 responses.',
    requirements: ['@supabase/ssr', 'getUser on server', 'Zod', 'Typed responses'], acceptance: ['Unauthenticated → 401',
'Invalid body → 422', 'Row visible only to owner'] },
  { number: 10, slug: 'supabase-crud', title: 'Supabase artifact CRUD workflow', category: 'Full-stack feature', recommended:
'v0', requiresPersistence: true,
    prompt: 'Create artifact CRUD with Supabase: migration, RLS, server actions for create/update/delete, optimistic UI,
revalidation.',
    requirements: ['Migration', 'RLS', 'Server Actions', 'Revalidate'], acceptance: ['Survives reload', 'RLS denies other
users'] },
  { number: 11, slug: 'cookie-auth', title: 'Cookie-based Supabase Auth', category: 'Full-stack feature', recommended: 'v0',
requiresPersistence: true,
    prompt: 'Implement cookie-based Supabase Auth with @supabase/ssr: browser/server clients, middleware session refresh, magic
link, confirm route, sign-out.',
    requirements: ['Middleware refresh', 'Magic link', 'Confirm route', 'Sign-out'], acceptance: ['Session survives reload',
'No service key in client bundle'] },
  { number: 12, slug: 'multi-tenant-workspace', title: 'Multi-tenant SaaS workspace', category: 'Full-stack feature',
recommended: 'v0', requiresPersistence: true,
    prompt: 'Build organizations, memberships (owner/admin/member/viewer), org switcher, invite flow and org-scoped RLS for
shared records.',
    requirements: ['Org tables', 'Role enum', 'RLS by membership', 'Org switcher'], acceptance: ['Viewer cannot write', 'Non-
member sees nothing'], weights: P({ correctness: 0.25, codeQuality: 0.15, rendering: 0.05, stateManagement: 0.1,
maintainability: 0.1, handoff: 0.1, sandboxFit: 0.05, persistence: 0.2 }) },
];
```

handoff/workflow-router-app/lib/benchmark/probes/persistence.ts · 20 lines

```
import type { PersistenceMode } from '../benchmark.config';
export interface PersistenceObservation { tested: boolean; survivedReload: boolean | null; mode: PersistenceMode; evidence?:
string }
/**
 * The highest persistence grade a run may receive. Local UI state never earns more than 2:
 * "do not award high persistence scores merely because a UI has local state."
 */
export function persistenceCeiling(o: PersistenceObservation, required: boolean): number {
  if (!o.tested) return required ? 0 : 5;
  if (!o.survivedReload) return 0;
  if (!o.evidence) return 1;
  switch (o.mode) {
    case 'none': case 'local-preview': return 1;
    case 'browser-local': case 'session': return 2;
    case 'database': case 'project-backed': return 5;
  }
}
```

```
export function clampPersistenceGrade(grade: number | null, o: PersistenceObservation, required: boolean): number | null {
  return grade === null ? null : Math.min(grade, persistenceCeiling(o, required));
}
```

handoff/workflow-router-app/lib/benchmark/probes/source-structure.ts · 14 lines

```
export interface ManifestEntry { path: string; bytes?: number }
export interface SourceStructure { files: number; multiFile: boolean; hasServerCode: boolean; hasMigrations: boolean; hasTests:
boolean; hasEnvExample: boolean; leaksServiceRole: boolean }
export function probeSourceStructure(manifest: ManifestEntry[], contents: Record<string, string> = {}): SourceStructure {
  const p = manifest.map((m) => m.path);
  return {
    files: p.length, multiFile: p.length > 1,
    hasServerCode: p.some((x) => /(^|\/)(route\.ts|actions\.ts|middleware\.ts|proxy\.ts)$/.test(x) || /\/api\//.test(x)),
    hasMigrations: p.some((x) => /migrations\/.+\.sql$/.test(x)),
    hasTests: p.some((x) => /\.(test|spec)\.(t|j)sx?$/.test(x)),
    hasEnvExample: p.includes('.env.example'),
    leaksServiceRole: Object.entries(contents).some(([path, t]) => /SERVICE_ROLE/.test(t) && (/^(app|components)\//.test(path)
&& /['"]use client['"]/.test(t) || /NEXT_PUBLIC_[A-Z_]*SERVICE/.test(t))),
  };
}
```

handoff/workflow-router-app/lib/benchmark/scoring.ts · 51 lines

```
import { CRITERIA, DEFAULT_WEIGHTS, type Criterion, type Weights } from './benchmark.config';

export type Grades = Partial<Record<Criterion, number | null>>;
export interface WeightedScore { outOf5: number | null; outOf100: number | null; missing: Criterion[]; weightUsed: number }

export function resolveWeights(override?: Weights): Record<Criterion, number> {
  const w = { ...DEFAULT_WEIGHTS, ...(override ?? {}) } as Record<Criterion, number>;
  for (const c of CRITERIA) w[c] = Math.max(0, w[c] ?? 0);
  return w;
}

/** Weighted 0–5 score. Missing grades are excluded and the remaining weights are renormalised; all-missing → null. */
export function weightedScore(grades: Grades, override?: Weights): WeightedScore {
  const w = resolveWeights(override);
  let sum = 0, used = 0; const missing: Criterion[] = [];
  for (const c of CRITERIA) {
    if (w[c] === 0) continue;
    const g = grades[c];
    if (g === null || g === undefined || Number.isNaN(g)) { missing.push(c); continue; }
    if (g < 0 || g > 5) throw new RangeError(c + ' must be 0–5');
    sum += g * w[c]; used += w[c];
  }
  if (used === 0) return { outOf5: null, outOf100: null, missing, weightUsed: 0 };
  const s = sum / used;
  return { outOf5: round(s, 2), outOf100: round((s / 5) * 100, 1), missing, weightUsed: round(used, 3) };
}

/** Average several graders' sheets criterion by criterion, then weight. */
export function aggregateGrades(sheets: Grades[], override?: Weights): WeightedScore & { graders: number } {
  const merged: Grades = {};
  for (const c of CRITERIA) {
    const vals = sheets.map((s) => s[c]).filter((v): v is number => typeof v === 'number');
    merged[c] = vals.length ? vals.reduce((a, b) => a + b, 0) / vals.length : null;
  }
  return { ...weightedScore(merged, override), graders: sheets.length };
}

export interface LeaderRow { key: string; score: number | null; tasks: number; graded: number }
export function leaderboard(entries: { key: string; score: number | null }[]): LeaderRow[] {
  const by = new Map<string, LeaderRow & { total: number }>();
  for (const e of entries) {
    const r = by.get(e.key) ?? { key: e.key, score: null, tasks: 0, graded: 0, total: 0 };
    r.tasks += 1; if (e.score !== null) { r.graded += 1; r.total += e.score; }
    by.set(e.key, r);
  }
  return [...by.values()].map(({ total, ...r }) => ({ ...r, score: r.graded ? round(total / r.graded, 2) : null }))
    .sort((a, b) => (b.score ?? -1) - (a.score ?? -1) || a.key.localeCompare(b.key));
}
```

```
const round = (n: number, d: number) => Math.round(n * 10 ** d) / 10 ** d;
```

handoff/workflow-router-app/lib/exports/csv.ts · 7 lines

```
const cell = (v: unknown) => { const s = v === null || v === undefined ? '' : String(v); return /[",\n\r]/.test(s) || /^[=+\-
@]/.test(s) ? '"' + (/^[=+\-@]/.test(s) ? "'" : '') + s.replace(/"/g, '""') + '"' : s; };
/** RFC 4180 CSV with formula-injection guarding. */
export function toCsv(rows: Record<string, unknown>[], columns?: string[]): string {
  const cols = columns ?? [...new Set(rows.flatMap((r) => Object.keys(r)))];
  return [cols.map(cell).join(','), ...rows.map((r) => cols.map((c) => cell(r[c])).join(','))].join('\r\n') + '\r\n';
}
```

handoff/workflow-router-app/lib/exports/json.ts · 2 lines

```
export const toJson = (x: unknown) => JSON.stringify({ schema: 'semester.workflow-router/1', exportedAt: new
Date().toISOString(), data: x }, null, 2) + '\n';
```

handoff/workflow-router-app/lib/exports/markdown.ts · 12 lines

```
import { PLATFORM_LABEL, WORKFLOWS } from '../benchmark/benchmark.config';
import type { Recommendation } from '../workflow-router/engine';
import type { RouterAnswers } from '../validation/workflow-router';
export function recommendationMarkdown(a: RouterAnswers, r: Recommendation, prompt: string): string {
  return ['# Semester AI workflow recommendation', '', '| | |', '| --- | --- |', '| Primary | ' + PLATFORM_LABEL[r.primary] + '
|', '| Secondary | ' + PLATFORM_LABEL[r.secondary] + ' |', '| Confidence | ' + Math.round(r.confidence * 100) + '% |', '|
Production escalation | ' + (r.escalation ? 'Yes — ' + r.escalationReasons.join('; ') : 'No') + ' |', '| Handoff | ' +
r.handoffFormat + ' |', '',
    '## Requirements', ...Object.entries(a).map(([k, v]) => '- ' + k + ': ' + v), '', '## Why', ...r.reasons[r.primary].map((x)
=> '- ' + x), '', '## Limitations', ...r.limitations.map((x) => '- ' + x), '', '## Prompt', '````', prompt, '````',
''].join('\n');
}
export function benchmarkMarkdown(rows: { workflow: number; platform: string; status: string; outOf100: number | null; graders:
number }[]): string {
  const title = (n: number) => WORKFLOWS.find((w) => w.number === n)?.title ?? String(n);
  return ['# Semester benchmark report', '', '| # | Workflow | Platform | Status | Score /100 | Graders |', '| --- | --- | ---
| --- | --- | --- |', ...rows.map((r) => '| ' + r.workflow + ' | ' + title(r.workflow) + ' | ' + r.platform + ' | ' + r.status
+ ' | ' + (r.outOf100 ?? '—') + ' | ' + r.graders + ' |'), ''].join('\n');
}
```

handoff/workflow-router-app/lib/supabase/client.ts · 4 lines

```
import { createBrowserClient } from '@supabase/ssr';
import type { Database } from '@/types/database';
export const createClient = () => createBrowserClient<Database>(process.env.NEXT_PUBLIC_SUPABASE_URL!,
process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY!);
```

handoff/workflow-router-app/lib/supabase/middleware.ts · 11 lines

```
import { createServerClient } from '@supabase/ssr';
import { NextResponse, type NextRequest } from 'next/server';
export async function updateSession(request: NextRequest) {
  let response = NextResponse.next({ request });
  const supabase = createServerClient(process.env.NEXT_PUBLIC_SUPABASE_URL!, process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY!,
{
    cookies: { getAll: () => request.cookies.getAll(), setAll: (list) => { list.forEach(({ name, value }) =>
request.cookies.set(name, value)); response = NextResponse.next({ request }); list.forEach(({ name, value, options }) =>
response.cookies.set(name, value, options)); } },
  });
  await supabase.auth.getUser(); // refreshes the session cookie
  return response;
}
```

handoff/workflow-router-app/lib/supabase/server.ts · 18 lines

```
import 'server-only';
import { createServerClient } from '@supabase/ssr';
import { cookies } from 'next/headers';
import type { Database } from '@/types/database';
export async function createClient() {
  const store = await cookies();
  return createServerClient<Database>(process.env.NEXT_PUBLIC_SUPABASE_URL!, process.env.NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY!,
{
    cookies: { getAll: () => store.getAll(), setAll: (list) => { try { list.forEach(({ name, value, options }) =>
store.set(name, value, options)); } catch { /* called from a Server Component; middleware refreshes */ } } },
  });
}
/** Server-validated user (never trust getSession on the server). */
export async function requireUser() {
  const supabase = await createClient();
  const { data: { user }, error } = await supabase.auth.getUser();
  if (error || !user) return { supabase, user: null } as const;
  return { supabase, user } as const;
}
```

handoff/workflow-router-app/lib/validation/benchmark.ts · 19 lines

```
import { z } from 'zod';
import { CRITERIA, PERSISTENCE_MODES, PLATFORMS } from '../benchmark/benchmark.config';
const grade = z.number().int().min(0).max(5).nullable();
export const RunSchema = z.object({
  taskId: z.string().uuid(), platform: z.enum(PLATFORMS), executedPrompt: z.string().min(1),
  modelName: z.string().max(120).optional(), modelVersion: z.string().max(120).optional(),
  status: z.enum(['pending', 'running', 'completed', 'manual-review-required', 'unsupported', 'failed']),
  statusReason: z.string().max(2000).optional(), outputUrl: z.string().url().optional().or(z.literal('')),
  sourceManifest: z.array(z.object({ path: z.string(), bytes: z.number().int().nonnegative().optional() })).default([]),
  runtimeNotes: z.string().max(4000).optional(), consoleNotes: z.string().max(8000).optional(), durationMs:
z.number().int().nonnegative().optional(),
  persistenceTested: z.boolean().default(false), persistenceSurvivedReload: z.boolean().nullable().default(null),
  persistenceMode: z.enum(PERSISTENCE_MODES).default('none'), persistenceEvidence: z.string().max(2000).optional(),
persistenceFailureNotes: z.string().max(2000).optional(),
}).superRefine((r, ctx) => {
  if ((r.status === 'manual-review-required' || r.status === 'unsupported') && !r.statusReason) ctx.addIssue({ code: 'custom',
path: ['statusReason'], message: 'Explain why the run needs manual review or is unsupported' });
  if (r.persistenceTested && r.persistenceSurvivedReload === null) ctx.addIssue({ code: 'custom', path:
['persistenceSurvivedReload'], message: 'Record whether state survived reload' });
  if (r.persistenceTested && !r.persistenceEvidence) ctx.addIssue({ code: 'custom', path: ['persistenceEvidence'], message:
'Persistence evidence (URL, row id, screenshot or log) is required' });
});
export const GradeSchema = z.object({ runId: z.string().uuid(), ...Object.fromEntries(CRITERIA.map((c) => [c, grade])) as
Record<(typeof CRITERIA)[number], typeof grade>, evidence: z.string().max(4000).optional(), notes:
z.string().max(4000).optional() });
```

handoff/workflow-router-app/lib/validation/workflow-router.ts · 12 lines

```
import { z } from 'zod';
export const RouterAnswersSchema = z.object({
  deliverable: z.enum(['interactive-tool', 'prototype', 'marketing-page', 'diagram', 'document', 'presentation', 'data-
analysis', 'react-component', 'full-stack']),
  interaction: z.enum(['none', 'simple-local', 'complex-local', 'multi-user']),
  backend: z.enum(['none', 'file-export', 'database', 'server-api', 'auth-secrets']),
  technology: z.enum(['none', 'html', 'react', 'nextjs', 'nextjs-postgres', 'nextjs-supabase']),
  persistence: z.enum(['none', 'database', 'auth', 'storage', 'realtime', 'multi-tenant']),
  handoff: z.enum(['share', 'document-export', 'github', 'vercel']),
});
export type RouterAnswers = z.infer<typeof RouterAnswersSchema>;
export const SaveRecommendationSchema = z.object({ title: z.string().trim().min(1).max(160), organizationId:
z.string().uuid().nullable(), answers: RouterAnswersSchema, visibility: z.enum(['private', 'organization']).default('private')
});
```

handoff/workflow-router-app/lib/workflow-router/engine.ts · 79 lines

```
import type { Platform } from '../benchmark/benchmark.config';
import type { RouterAnswers } from '../validation/workflow-router';

export interface Recommendation { primary: Platform; secondary: Platform; confidence: number; scores: Record<Platform, number>;
```

```
reasons: Record<Platform, string[]>; escalation: boolean; escalationReasons: string[]; limitations: string[]; handoffFormat:
string; forcedV0: boolean }

/** Selections that must route to v0: anything needing a server, persistence, auth, multi-user, or deployment. */
export function v0Triggers(a: RouterAnswers): string[] {
  const t: string[] = [];
  if (a.technology === 'nextjs' || a.technology === 'nextjs-postgres' || a.technology === 'nextjs-supabase') t.push('Next.js
target');
  if (a.technology === 'nextjs-postgres') t.push('Postgres');
  if (a.technology === 'nextjs-supabase') t.push('Supabase / Auth');
  if (a.backend === 'database') t.push('Persistent records');
  if (a.backend === 'server-api') t.push('Server-side APIs or webhooks');
  if (a.backend === 'auth-secrets') t.push('Authentication and private server secrets');
  if (a.persistence === 'database' || a.persistence === 'storage') t.push('Persistent records');
  if (a.persistence === 'auth') t.push('Authentication');
  if (a.persistence === 'realtime') t.push('Realtime / multi-user');
  if (a.persistence === 'multi-tenant') t.push('Organization workspace, roles and Row Level Security');
  if (a.interaction === 'multi-user') t.push('Multi-user collaboration');
  if (a.deliverable === 'full-stack') t.push('Full-stack feature');
  if (a.handoff === 'github') t.push('GitHub-backed deployment');
  if (a.handoff === 'vercel') t.push('Vercel deployment');
  return [...new Set(t)];
}

export function recommend(a: RouterAnswers): Recommendation {
  const s: Record<Platform, number> = { 'claude-artifacts': 0, 'chatgpt-canvas': 0, v0: 0 };
  const r: Record<Platform, string[]> = { 'claude-artifacts': [], 'chatgpt-canvas': [], v0: [] };
  const add = (p: Platform, n: number, why: string) => { s[p] += n; r[p].push(why); };

  switch (a.deliverable) {
    case 'interactive-tool': add('claude-artifacts', 4, 'Self-contained interactive tools are the Artifacts sweet spot');
add('v0', 1, 'v0 can host the tool if it later needs a backend'); break;
    case 'prototype': add('claude-artifacts', 4, 'Fast shareable prototypes render beside the chat'); add('v0', 2, 'v0
prototypes become real projects'); break;
    case 'marketing-page': add('claude-artifacts', 3, 'Single-page HTML is portable'); add('v0', 3, 'v0 ships Next.js marketing
pages to Vercel'); break;
    case 'diagram': add('claude-artifacts', 5, 'Mermaid/SVG diagrams render natively'); add('chatgpt-canvas', 1, 'Canvas can
iterate diagram source text'); break;
    case 'document': add('chatgpt-canvas', 5, 'Revision-centred writing with PDF/DOCX/Markdown export'); add('claude-
artifacts', 2, 'Markdown documents render as artifacts'); break;
    case 'presentation': add('chatgpt-canvas', 3, 'Iterate slide copy and outline'); add('claude-artifacts', 3, 'HTML slide
decks render and share'); break;
    case 'data-analysis': add('chatgpt-canvas', 5, 'Canvas runs Python for data analysis'); add('claude-artifacts', 1, 'Client-
side charts for a finished dataset'); break;
    case 'react-component': add('claude-artifacts', 3, 'Single-component React renders in Artifacts'); add('v0', 3, 'v0 places
components in a real project'); add('chatgpt-canvas', 2, 'Canvas iterates component code'); break;
    case 'full-stack': add('v0', 6, 'Multi-file Next.js with server routes'); break;
  }
  switch (a.interaction) {
    case 'none': add('chatgpt-canvas', 1, 'No interaction needed'); break;
    case 'simple-local': add('claude-artifacts', 2, 'Simple local state is fine in an artifact'); break;
    case 'complex-local': add('claude-artifacts', 2, 'Complex client state in one React file'); add('v0', 1, 'Complex state may
outgrow one file'); break;
    case 'multi-user': add('v0', 5, 'Multi-user needs a server and database'); break;
  }
  if (a.backend === 'file-export') { add('claude-artifacts', 2, 'Browser-generated CSV/PNG/Markdown downloads'); add('chatgpt-
canvas', 2, 'Document/source-file export'); }
  if (['database', 'server-api', 'auth-secrets'].includes(a.backend)) add('v0', 5, 'Server code, API routes and database
connections run in v0');
  if (a.technology === 'html') add('claude-artifacts', 2, 'Plain HTML/CSS/JS is portable');
  if (a.technology === 'react') { add('claude-artifacts', 1, 'React via approved CDNs'); add('v0', 1, 'Native React project');
}
  if (a.technology.startsWith('nextjs')) add('v0', 5, 'Next.js App Router is v0's native target');
  if (a.persistence === 'none') add('claude-artifacts', 1, 'No persistence required');
  if (a.handoff === 'share') add('claude-artifacts', 2, 'Share a published artifact immediately');
  if (a.handoff === 'document-export') add('chatgpt-canvas', 2, 'PDF/DOCX/Markdown export');
  if (a.handoff === 'github' || a.handoff === 'vercel') add('v0', 4, 'GitHub sync and Vercel deployment');

  const triggers = v0Triggers(a);
  const forcedV0 = triggers.length > 0;
  if (forcedV0) add('v0', 10, 'Required for: ' + triggers.join(', '));

  const order = (Object.keys(s) as Platform[]).sort((x, y) => s[y] - s[x]);
  let [primary, secondary] = order;
  if (forcedV0 && primary !== 'v0') { secondary = primary; primary = 'v0'; }
  const total = Object.values(s).reduce((a2, b) => a2 + b, 0) || 1;
  const margin = (s[primary] - s[secondary]) / total;
```

```
  const confidence = Math.min(0.97, Math.round((0.55 + margin) * 100) / 100);

  const escalationReasons = [...triggers];
  if (primary !== 'v0' && (a.persistence !== 'none')) escalationReasons.push('Persistence selected: artifact/canvas state is
not durable storage');
  const limitations = {
    'claude-artifacts': ['No durable backend, secret store or production auth', 'Libraries only from approved CDNs',
'localStorage/fetch may be restricted in the sandbox'],
    'chatgpt-canvas': ['Not a production backend or deployment target', 'React/HTML preview runs in an isolated sandbox'],
    v0: ['Generated code still needs migrations, RLS, tests and environment variables before production', 'Sandbox is isolated;
production secrets live in Vercel/Supabase'],
  }[primary];
  const handoffFormat = { 'claude-artifacts': 'Single-file HTML or React/JSX, Mermaid/SVG, browser CSV/PNG download', 'chatgpt-
canvas': 'PDF, DOCX or Markdown; code as .py/.ts/.sql', v0: 'Next.js project → GitHub repository → Vercel deployment' }
[primary];
  return { primary, secondary, confidence, scores: s, reasons: r, escalation: escalationReasons.length > 0, escalationReasons,
limitations, handoffFormat, forcedV0 };
}
```

handoff/workflow-router-app/lib/workflow-router/prompts.ts · 20 lines

```
import { PLATFORM_LABEL } from '../benchmark/benchmark.config';
import type { Recommendation } from './engine';
import type { RouterAnswers } from '../validation/workflow-router';
export function buildPrompt(a: RouterAnswers, r: Recommendation): string {
  const head = 'You are building a Semester ' + a.deliverable.replace('-', ' ') + ' in ' + PLATFORM_LABEL[r.primary] + '.';
  const rules = r.primary === 'v0'
    ? ['Next.js App Router, TypeScript strict, Tailwind.', 'Supabase via @supabase/ssr: lib/supabase/client.ts, server.ts,
middleware session refresh.', 'Validate every mutation with Zod on the server; call supabase.auth.getUser() server-side.',
'Write SQL migrations with RLS on every exposed table; never expose a service-role key.', 'Add tests and .env.example; do not
call it production-ready until migrations, RLS and tests pass.']
    : r.primary === 'chatgpt-canvas'
      ? ['Work as a revisable document/code canvas.', 'Use clear sections and export to Markdown/DOCX/PDF (or the code file
type).', 'State assumptions and sources.']
      : ['Single self-contained file (HTML or one React component).', 'Libraries only from approved CDNs; no secrets, no fetch
to private APIs.', 'Treat state as local preview only; provide CSV/Markdown download if data must leave.'];
  return [head, '', 'Requirements: interaction=' + a.interaction + ', backend=' + a.backend + ', tech=' + a.technology + ',
persistence=' + a.persistence + ', handoff=' + a.handoff + '.', '', ...rules.map((x) => '- ' + x), '', 'Use Semester tokens:
Ink chrome, Parchment content, Semester blue for action, glyph + word + tone for every status, and label every fact with its
source.'].join('\n');
}
export function githubPlan(r: Recommendation): string[] {
  return r.primary === 'v0' ? ['Create repo from v0 → GitHub', 'Protect main; PRs require typecheck, lint, test, build', 'Add
Supabase env vars in Vercel (never the service role in NEXT_PUBLIC_*)', 'Run migrations via supabase db push in CI', 'Preview
deployments per PR']
    : ['Export the source (' + r.handoffFormat + ')', 'Commit to docs/ or prototypes/ in the Semester repo', 'Open a PR linking
the artifact URL', 'Escalate to v0 if persistence/auth is later required'];
}
export function supabasePrompt(): string {
  return 'Add Supabase to this Next.js app: @supabase/ssr browser/server clients, middleware session refresh, magic-link auth
with /auth/confirm, migrations for organizations, organization_members (owner/admin/member/viewer), and the feature tables with
RLS scoped to auth.uid() and org membership. Use NEXT_PUBLIC_SUPABASE_URL and NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY only.';
}
```

handoff/workflow-router-app/middleware.ts · 6 lines

```
// Next.js 15: middleware.ts. On Next.js 16+, rename to proxy.ts and export `proxy` instead.
import type { NextRequest } from 'next/server';
import { updateSession } from '@/lib/supabase/middleware';
export async function middleware(request: NextRequest) { return updateSession(request); }
export const config = { matcher: ['/((?!_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|jpeg|gif|webp)$).*)'] };
```

handoff/workflow-router-app/next.config.ts · 4 lines

```
import type { NextConfig } from 'next';
const config: NextConfig = { reactStrictMode: true, poweredByHeader: false };
export default config;
```

handoff/workflow-router-app/package.json · 9 lines

```
{
  "name": "semester-workflow-router",
  "private": true,
  "type": "module",
  "scripts": { "dev": "next dev", "build": "next build", "start": "next start", "lint": "next lint", "typecheck": "tsc --
noEmit", "test": "vitest run", "db:types": "supabase gen types typescript --local > types/database.ts" },
  "dependencies": { "@supabase/ssr": "^0.5.2", "@supabase/supabase-js": "^2.45.0", "next": "^15.1.0", "react": "^19.0.0",
"react-dom": "^19.0.0", "server-only": "^0.0.1", "zod": "^3.23.8" },
  "devDependencies": { "@tailwindcss/postcss": "^4.0.0", "@types/node": "^22", "@types/react": "^19", "eslint": "^9", "eslint-
config-next": "^15.1.0", "tailwindcss": "^4.0.0", "typescript": "^5.6.0", "vite-tsconfig-paths": "^5.1.0", "vitest": "^2.1.0" }
}
```

handoff/workflow-router-app/postcss.config.mjs · 2 lines

```
export default { plugins: { '@tailwindcss/postcss': {} } };
```

handoff/workflow-router-app/supabase/migrations/001_core_workflow_router.sql · 69 lines

```
create extension if not exists pgcrypto;
create type public.org_role as enum ('owner', 'admin', 'member', 'viewer');
create type public.visibility as enum ('private', 'organization');

create table public.organizations (
  id uuid primary key default gen_random_uuid(),
  name text not null check (char_length(name) between 1 and 120),
  created_by uuid not null default auth.uid() references auth.users(id),
  created_at timestamptz not null default now()
);
create table public.organization_members (
  organization_id uuid not null references public.organizations(id) on delete cascade,
  user_id uuid not null references auth.users(id) on delete cascade,
  role public.org_role not null default 'member',
  created_at timestamptz not null default now(),
  primary key (organization_id, user_id)
);
create table public.workflow_recommendations (
  id uuid primary key default gen_random_uuid(),
  owner_id uuid not null default auth.uid() references auth.users(id) on delete cascade,
  organization_id uuid references public.organizations(id) on delete cascade,
  visibility public.visibility not null default 'private',
  title text not null check (char_length(title) between 1 and 160),
  answers jsonb not null,
  primary_platform text not null check (primary_platform in ('claude-artifacts','chatgpt-canvas','v0')),
  secondary_platform text not null check (secondary_platform in ('claude-artifacts','chatgpt-canvas','v0')),
  confidence numeric(4,3) not null check (confidence between 0 and 1),
  result jsonb not null,
  created_at timestamptz not null default now(),
  check (visibility = 'private' or organization_id is not null)
);

-- Membership helpers. SECURITY DEFINER avoids recursive RLS on organization_members.
create or replace function public.is_org_member(org uuid) returns boolean language sql stable security definer set search_path
= public as $$
  select exists (select 1 from organization_members m where m.organization_id = org and m.user_id = auth.uid()) $$;
create or replace function public.has_org_role(org uuid, roles public.org_role[]) returns boolean language sql stable security
definer set search_path = public as $$
  select exists (select 1 from organization_members m where m.organization_id = org and m.user_id = auth.uid() and m.role =
any(roles)) $$;
revoke all on function public.is_org_member(uuid), public.has_org_role(uuid, public.org_role[]) from public;
grant execute on function public.is_org_member(uuid), public.has_org_role(uuid, public.org_role[]) to authenticated;

-- Creator becomes owner.
create or replace function public.add_org_owner() returns trigger language plpgsql security definer set search_path = public as
$$
begin insert into organization_members (organization_id, user_id, role) values (new.id, new.created_by, 'owner'); return new;
end $$;
create trigger organizations_owner after insert on public.organizations for each row execute function public.add_org_owner();

alter table public.organizations enable row level security;
alter table public.organization_members enable row level security;
alter table public.workflow_recommendations enable row level security;
```

```
create policy org_select on public.organizations for select to authenticated using (public.is_org_member(id));
create policy org_insert on public.organizations for insert to authenticated with check (created_by = auth.uid());
create policy org_update on public.organizations for update to authenticated using (public.has_org_role(id,
array['owner','admin']::public.org_role[]));
create policy org_delete on public.organizations for delete to authenticated using (public.has_org_role(id,
array['owner']::public.org_role[]));

create policy mem_select on public.organization_members for select to authenticated using
(public.is_org_member(organization_id));
create policy mem_write on public.organization_members for all to authenticated
  using (public.has_org_role(organization_id, array['owner','admin']::public.org_role[]))
  with check (public.has_org_role(organization_id, array['owner','admin']::public.org_role[]) and (role <> 'owner' or
public.has_org_role(organization_id, array['owner']::public.org_role[])));

create policy rec_select on public.workflow_recommendations for select to authenticated
  using (owner_id = auth.uid() or (visibility = 'organization' and public.is_org_member(organization_id)));
create policy rec_insert on public.workflow_recommendations for insert to authenticated
  with check (owner_id = auth.uid() and (organization_id is null or public.has_org_role(organization_id,
array['owner','admin','member']::public.org_role[])));
create policy rec_update on public.workflow_recommendations for update to authenticated using (owner_id = auth.uid()) with
check (owner_id = auth.uid());
create policy rec_delete on public.workflow_recommendations for delete to authenticated
  using (owner_id = auth.uid() or (organization_id is not null and public.has_org_role(organization_id,
array['owner','admin']::public.org_role[])));
create index on public.workflow_recommendations (owner_id, created_at desc);
create index on public.workflow_recommendations (organization_id) where organization_id is not null;
```

handoff/workflow-router-app/supabase/migrations/002_benchmark_grading.sql · 72 lines

```
create table public.benchmark_suites (
  id uuid primary key default gen_random_uuid(),
  owner_id uuid not null default auth.uid() references auth.users(id) on delete cascade,
  organization_id uuid references public.organizations(id) on delete cascade,
  visibility public.visibility not null default 'private',
  name text not null, description text, created_at timestamptz not null default now(),
  check (visibility = 'private' or organization_id is not null)
);
create table public.benchmark_tasks (
  id uuid primary key default gen_random_uuid(),
  suite_id uuid not null references public.benchmark_suites(id) on delete cascade,
  number int not null check (number between 1 and 99), slug text not null, title text not null, category text not null,
  prompt text not null, requirements jsonb not null default '[]', acceptance jsonb not null default '[]',
  recommended_platform text not null check (recommended_platform in ('claude-artifacts','chatgpt-canvas','v0')),
  requires_persistence boolean not null default false, weights jsonb,
  unique (suite_id, number)
);
create table public.benchmark_runs (
  id uuid primary key default gen_random_uuid(),
  task_id uuid not null references public.benchmark_tasks(id) on delete cascade,
  platform text not null check (platform in ('claude-artifacts','chatgpt-canvas','v0')),
  created_by uuid not null default auth.uid() references auth.users(id),
  status text not null check (status in ('pending','running','completed','manual-review-required','unsupported','failed')),
  status_reason text, executed_prompt text not null, model_name text, model_version text,
  output_url text, source_manifest jsonb not null default '[]', runtime_notes text, console_notes text, duration_ms int check
(duration_ms >= 0),
  persistence_tested boolean not null default false, persistence_survived_reload boolean,
  persistence_mode text not null default 'none' check (persistence_mode in ('none','local-preview','browser-
local','session','database','project-backed')),
  persistence_evidence text, persistence_failure_notes text,
  created_at timestamptz not null default now(),
  check (status not in ('manual-review-required','unsupported') or status_reason is not null),
  check (not persistence_tested or (persistence_survived_reload is not null and persistence_evidence is not null))
);
create table public.benchmark_grades (
  id uuid primary key default gen_random_uuid(),
  run_id uuid not null references public.benchmark_runs(id) on delete cascade,
  grader_id uuid not null default auth.uid() references auth.users(id),
  correctness smallint check (correctness between 0 and 5), code_quality smallint check (code_quality between 0 and 5),
  rendering smallint check (rendering between 0 and 5), state_management smallint check (state_management between 0 and 5),
  maintainability smallint check (maintainability between 0 and 5), handoff smallint check (handoff between 0 and 5),
  sandbox_fit smallint check (sandbox_fit between 0 and 5), persistence smallint check (persistence between 0 and 5),
  evidence text, notes text, created_at timestamptz not null default now(),
  unique (run_id, grader_id)
);
create index on public.benchmark_tasks (suite_id);
```

```
create index on public.benchmark_runs (task_id, platform);
create index on public.benchmark_grades (run_id);

-- Access follows the suite.
create or replace function public.can_read_suite(s uuid) returns boolean language sql stable security definer set search_path =
public as $$
  select exists (select 1 from benchmark_suites b where b.id = s and (b.owner_id = auth.uid() or (b.visibility = 'organization'
and is_org_member(b.organization_id)))) $$;
create or replace function public.can_write_suite(s uuid) returns boolean language sql stable security definer set search_path
= public as $$
  select exists (select 1 from benchmark_suites b where b.id = s and (b.owner_id = auth.uid() or (b.visibility = 'organization'
and has_org_role(b.organization_id, array['owner','admin','member']::org_role[])))) $$;
grant execute on function public.can_read_suite(uuid), public.can_write_suite(uuid) to authenticated;

alter table public.benchmark_suites enable row level security;
alter table public.benchmark_tasks enable row level security;
alter table public.benchmark_runs enable row level security;
alter table public.benchmark_grades enable row level security;

create policy suite_select on public.benchmark_suites for select to authenticated using (public.can_read_suite(id));
create policy suite_insert on public.benchmark_suites for insert to authenticated with check (owner_id = auth.uid() and
(organization_id is null or public.has_org_role(organization_id, array['owner','admin','member']::public.org_role[])));
create policy suite_update on public.benchmark_suites for update to authenticated using (public.can_write_suite(id));
create policy suite_delete on public.benchmark_suites for delete to authenticated using (owner_id = auth.uid());
create policy task_select on public.benchmark_tasks for select to authenticated using (public.can_read_suite(suite_id));
create policy task_write on public.benchmark_tasks for all to authenticated using (public.can_write_suite(suite_id)) with check
(public.can_write_suite(suite_id));
create policy run_select on public.benchmark_runs for select to authenticated using (public.can_read_suite((select suite_id
from public.benchmark_tasks t where t.id = task_id)));
create policy run_insert on public.benchmark_runs for insert to authenticated with check (created_by = auth.uid() and
public.can_write_suite((select suite_id from public.benchmark_tasks t where t.id = task_id)));
create policy run_update on public.benchmark_runs for update to authenticated using (created_by = auth.uid());
create policy grade_select on public.benchmark_grades for select to authenticated using (public.can_read_suite((select
t.suite_id from public.benchmark_runs r join public.benchmark_tasks t on t.id = r.task_id where r.id = run_id)));
create policy grade_insert on public.benchmark_grades for insert to authenticated with check (grader_id = auth.uid() and
public.can_write_suite((select t.suite_id from public.benchmark_runs r join public.benchmark_tasks t on t.id = r.task_id where
r.id = run_id)));
create policy grade_update on public.benchmark_grades for update to authenticated using (grader_id = auth.uid()) with check
(grader_id = auth.uid());
```

handoff/workflow-router-app/supabase/seed.sql · 22 lines

```
-- Seeds one suite with the 12 canonical workflows for the first user. Run after signing up once:
--   supabase db reset   (local)   or paste into the SQL editor with :owner set.
do $$
declare owner uuid := (select id from auth.users order by created_at limit 1); suite uuid;
begin
  if owner is null then raise notice 'Sign up a user first'; return; end if;
  insert into public.benchmark_suites (owner_id, name, description) values (owner, 'Semester canonical 12', 'Claude Artifacts
vs ChatGPT Canvas vs v0 across the 12 canonical workflows') returning id into suite;
  insert into public.benchmark_tasks (suite_id, number, slug, title, category, prompt, recommended_platform,
requires_persistence) values
  (suite, 1, 'artifact-tracker', 'Sortable artifact tracker', 'Interactive tool', 'Build a single-file React tracker of every
Semester artifact with search, filters, sortable columns and CSV export.', 'claude-artifacts', false),
  (suite, 2, 'onboarding-prototype', 'Interactive onboarding prototype', 'Product prototype', 'Prototype Semester student
onboarding with consent one decision at a time.', 'claude-artifacts', false),
  (suite, 3, 'b2b-landing', 'B2B SaaS landing page', 'Marketing page', 'Design a Semester institution landing page with a
capability status map.', 'claude-artifacts', false),
  (suite, 4, 'architecture-diagram', 'Multi-tenant system architecture diagram', 'Diagram', 'Draw Semester multi-tenant
architecture as Mermaid with the RLS boundary.', 'claude-artifacts', false),
  (suite, 5, 'tech-design-doc', 'Technical design document', 'Document', 'Write a TDD for Semester degree-audit sync.',
'chatgpt-canvas', false),
  (suite, 6, 'launch-presentation', 'Product launch presentation', 'Presentation', 'Draft a 10-slide pilot launch presentation
for registrars.', 'chatgpt-canvas', false),
  (suite, 7, 'subscription-csv', 'Subscription CSV analysis', 'Data analysis', 'Analyse a subscription CSV: MRR, churn,
cohorts.', 'chatgpt-canvas', false),
  (suite, 8, 'metric-dashboard', 'Reusable React metric dashboard', 'React component', 'Build a reusable typed
<MetricDashboard> component with all states.', 'claude-artifacts', false),
  (suite, 9, 'auth-route-handler', 'Authenticated Next.js route handler', 'Full-stack feature', 'Implement an authenticated
route handler with Supabase, getUser and Zod.', 'v0', true),
  (suite, 10, 'supabase-crud', 'Supabase artifact CRUD workflow', 'Full-stack feature', 'Create artifact CRUD with migration,
RLS and server actions.', 'v0', true),
  (suite, 11, 'cookie-auth', 'Cookie-based Supabase Auth', 'Full-stack feature', 'Implement cookie-based Supabase Auth with
@supabase/ssr and middleware refresh.', 'v0', true),
  (suite, 12, 'multi-tenant-workspace', 'Multi-tenant SaaS workspace', 'Full-stack feature', 'Build organizations, roles and
```

```
org-scoped RLS.', 'v0', true);
end $$;
```

handoff/workflow-router-app/tests/RLS-TEST-PLAN.md · 21 lines

```
# RLS / auth test plan (run against `supabase start`)

Use two users A, B and an organization O where A=owner, C=viewer, B is not a member.

| # | As | Action | Expected |
| --- | --- | --- | --- |
| 1 | anon | select any table | 0 rows / permission denied |
| 2 | A | insert private recommendation | ok; B selects → 0 rows |
| 3 | A | insert recommendation visibility=organization, org O | C selects → 1 row; B → 0 |
| 4 | C (viewer) | insert into O | denied (rec_insert requires owner/admin/member) |
| 5 | B | insert member row into O | denied (mem_write) |
| 6 | A | create private suite + seed tasks | B selects tasks/runs/grades → 0 |
| 7 | B | insert run for A's task | denied (can_write_suite) |
| 8 | C | grade a run in an org suite | denied (viewer) |
| 9 | A | update another grader's grade | denied (grader_id) |
| 10 | any | insert run status=unsupported without reason | check constraint violation |
| 11 | any | insert run persistence_tested=true without evidence | check constraint violation |
| 12 | build | grep .next/static for SERVICE_ROLE | no match |

Automate with `supabase test db` (pgTAP) using `set local role authenticated; set local request.jwt.claims = '{"sub":"
<uuid>"}'`.
```

handoff/workflow-router-app/tests/exports.test.ts · 9 lines

```
import { describe, expect, it } from 'vitest';
import { toCsv } from '@/lib/exports/csv';
import { recommendationMarkdown } from '@/lib/exports/markdown';
import { recommend } from '@/lib/workflow-router/engine';
describe('exports', () => {
  it('escapes CSV and guards formulas', () => { expect(toCsv([{ a: 'x,y', b: '=SUM(1)'
}])).toBe('a,b\r\n"x,y","\'=SUM(1)"\r\n'); });
  it('writes a Markdown recommendation with primary and escalation', () => { const a = { deliverable: 'full-stack',
interaction: 'multi-user', backend: 'database', technology: 'nextjs-supabase', persistence: 'multi-tenant', handoff: 'vercel' }
as const; const md = recommendationMarkdown(a, recommend(a), 'p'); expect(md).toContain('| Primary | v0 |');
expect(md).toContain('Production escalation | Yes'); });
});
```

handoff/workflow-router-app/tests/persistence.test.ts · 9 lines

```
import { describe, expect, it } from 'vitest';
import { clampPersistenceGrade, persistenceCeiling } from '@/lib/benchmark/probes/persistence';
describe('persistence ceiling', () => {
  it('local state never earns more than 2', () => { expect(persistenceCeiling({ tested: true, survivedReload: true, mode:
'browser-local', evidence: 'video' }, true)).toBe(2); expect(persistenceCeiling({ tested: true, survivedReload: true, mode:
'local-preview', evidence: 'x' }, true)).toBe(1); });
  it('database with evidence can earn 5', () => { expect(persistenceCeiling({ tested: true, survivedReload: true, mode:
'database', evidence: 'row abc' }, true)).toBe(5); });
  it('untested required persistence is 0', () => { expect(clampPersistenceGrade(5, { tested: false, survivedReload: null, mode:
'database' }, true)).toBe(0); });
  it('lost on reload is 0', () => { expect(persistenceCeiling({ tested: true, survivedReload: false, mode: 'database',
evidence: 'log' }, true)).toBe(0); });
});
```

handoff/workflow-router-app/tests/routing.test.ts · 12 lines

```
import { describe, expect, it } from 'vitest';
import { recommend } from '@/lib/workflow-router/engine';
import type { RouterAnswers } from '@/lib/validation/workflow-router';
const base: RouterAnswers = { deliverable: 'interactive-tool', interaction: 'simple-local', backend: 'none', technology:
'none', persistence: 'none', handoff: 'share' };
describe('router', () => {
  it('sends a self-contained tool to Claude Artifacts', () => { expect(recommend(base).primary).toBe('claude-artifacts'); });
  it('sends documents and data analysis to Canvas', () => { expect(recommend({ ...base, deliverable: 'document', interaction:
'none', handoff: 'document-export' }).primary).toBe('chatgpt-canvas'); expect(recommend({ ...base, deliverable: 'data-
analysis', interaction: 'none' }).primary).toBe('chatgpt-canvas'); });
```

```
  const forced: Partial<RouterAnswers>[] = [{ technology: 'nextjs' }, { technology: 'nextjs-postgres' }, { technology: 'nextjs-
supabase' }, { backend: 'database' }, { backend: 'server-api' }, { backend: 'auth-secrets' }, { persistence: 'database' }, {
persistence: 'auth' }, { persistence: 'multi-tenant' }, { persistence: 'realtime' }, { interaction: 'multi-user' }, { handoff:
'github' }, { handoff: 'vercel' }];
  for (const f of forced) it('forces v0 for ' + JSON.stringify(f), () => { const r = recommend({ ...base, deliverable:
'diagram', ...f }); expect(r.primary).toBe('v0'); expect(r.forcedV0).toBe(true); expect(r.escalation).toBe(true); });
  it('bounds confidence', () => { const c = recommend(base).confidence; expect(c).toBeGreaterThan(0.5);
expect(c).toBeLessThanOrEqual(0.97); });
});
```

handoff/workflow-router-app/tests/scoring.test.ts · 15 lines

```
import { describe, expect, it } from 'vitest';
import { aggregateGrades, leaderboard, weightedScore } from '@/lib/benchmark/scoring';
describe('weightedScore', () => {
  it('uses the default weights', () => { const s = weightedScore({ correctness: 5, codeQuality: 5, rendering: 5,
stateManagement: 5, maintainability: 5, handoff: 5, sandboxFit: 5 }); expect(s).toMatchObject({ outOf5: 5, outOf100: 100,
missing: [] }); });
  it('weights correctness at 25%', () => { expect(weightedScore({ correctness: 5, codeQuality: 0, rendering: 0,
stateManagement: 0, maintainability: 0, handoff: 0, sandboxFit: 0 }).outOf100).toBe(25); });
  it('renormalises around missing grades and reports them', () => { const s = weightedScore({ correctness: 4 });
expect(s.outOf5).toBe(4); expect(s.missing).toContain('codeQuality'); });
  it('is null when nothing is graded', () => { expect(weightedScore({}).outOf5).toBeNull(); });
  it('honours task overrides incl. persistence weight', () => { expect(weightedScore({ correctness: 5, persistence: 0 }, {
correctness: 0.5, persistence: 0.5, codeQuality: 0, rendering: 0, stateManagement: 0, maintainability: 0, handoff: 0,
sandboxFit: 0 }).outOf5).toBe(2.5); });
  it('rejects out-of-range grades', () => { expect(() => weightedScore({ correctness: 6 })).toThrow(RangeError); });
});
describe('aggregate + leaderboard', () => {
  it('averages graders per criterion', () => { expect(aggregateGrades([{ correctness: 4 }, { correctness: 2
}]).outOf5).toBe(3); });
  it('ranks graded platforms above ungraded', () => { const l = leaderboard([{ key: 'v0', score: 4 }, { key: 'Canvas', score:
null }, { key: 'Claude', score: 4.5 }]); expect(l.map((r) => r.key)).toEqual(['Claude', 'v0', 'Canvas']); });
});
```

handoff/workflow-router-app/tests/validation.test.ts · 10 lines

```
import { describe, expect, it } from 'vitest';
import { RunSchema } from '@/lib/validation/benchmark';
const ok = { taskId: '00000000-0000-4000-8000-000000000000', platform: 'v0', executedPrompt: 'x', status: 'completed' };
describe('RunSchema', () => {
  it('accepts a minimal completed run', () => { expect(RunSchema.safeParse(ok).success).toBe(true); });
  it('requires a reason for manual-review-required', () => { expect(RunSchema.safeParse({ ...ok, status: 'manual-review-
required' }).success).toBe(false); });
  it('requires reload result and evidence when persistence was tested', () => { expect(RunSchema.safeParse({ ...ok,
persistenceTested: true }).success).toBe(false); expect(RunSchema.safeParse({ ...ok, persistenceTested: true,
persistenceSurvivedReload: true, persistenceEvidence: 'row 42' }).success).toBe(true); });
  it('rejects unknown platforms', () => { expect(RunSchema.safeParse({ ...ok, platform: 'bolt' }).success).toBe(false); });
});
```

handoff/workflow-router-app/tsconfig.json · 2 lines

```
{ "compilerOptions": { "target": "ES2022", "lib": ["dom", "dom.iterable", "esnext"], "strict": true, "noEmit": true, "module":
"esnext", "moduleResolution": "bundler", "resolveJsonModule": true, "isolatedModules": true, "jsx": "preserve", "incremental":
true, "plugins": [{ "name": "next" }], "paths": { "@/*": ["./*"] }, "skipLibCheck": true }, "include": ["next-env.d.ts",
"**/*.ts", "**/*.tsx", ".next/types/**/*.ts"], "exclude": ["node_modules"] }
```

handoff/workflow-router-app/types/database.ts · 12 lines

```
// Regenerate with: npx supabase gen types typescript --local > types/database.ts
type Row<T> = { Row: T; Insert: Partial<T>; Update: Partial<T>; Relationships: [] };
export interface Database { public: { Tables: {
  organizations: Row<{ id: string; name: string; created_by: string; created_at: string }>;
  organization_members: Row<{ organization_id: string; user_id: string; role: 'owner' | 'admin' | 'member' | 'viewer';
created_at: string }>;
  workflow_recommendations: Row<{ id: string; owner_id: string; organization_id: string | null; visibility: 'private' |
'organization'; title: string; answers: unknown; primary_platform: string; secondary_platform: string; confidence: number;
result: unknown; created_at: string }>;
  benchmark_suites: Row<{ id: string; owner_id: string; organization_id: string | null; visibility: 'private' | 'organization';
name: string; description: string | null; created_at: string }>;
```

```
  benchmark_tasks: Row<{ id: string; suite_id: string; number: number; slug: string; title: string; category: string; prompt:
string; requirements: unknown; acceptance: unknown; recommended_platform: string; requires_persistence: boolean; weights:
unknown }>;
  benchmark_runs: Row<{ id: string; task_id: string; platform: string; created_by: string; status: string; status_reason:
string | null; executed_prompt: string; model_name: string | null; model_version: string | null; output_url: string | null;
source_manifest: unknown; runtime_notes: string | null; console_notes: string | null; duration_ms: number | null;
persistence_tested: boolean; persistence_survived_reload: boolean | null; persistence_mode: string; persistence_evidence:
string | null; persistence_failure_notes: string | null; created_at: string }>;
  benchmark_grades: Row<{ id: string; run_id: string; grader_id: string; correctness: number | null; code_quality: number |
null; rendering: number | null; state_management: number | null; maintainability: number | null; handoff: number | null;
sandbox_fit: number | null; persistence: number | null; evidence: string | null; notes: string | null; created_at: string }>;
}; Views: {}; Functions: {}; Enums: {}; CompositeTypes: {} } }
```

handoff/workflow-router-app/vitest.config.ts · 4 lines

```
import { defineConfig } from 'vitest/config';
import paths from 'vite-tsconfig-paths';
export default defineConfig({ plugins: [paths()], test: { include: ['tests/**/*.test.ts'] } });
```

Claude Code toolkit scripts and build scripts

handoff/claude-code-toolkit/app/package.scripts.json · 10 lines

```
{
  "scripts": {
    "tokens:check": "vitest run src/lib/tokenexport.test.ts",
    "design-system:audit": "node scripts/design-system-audit.mjs",
    "design-system:report": "node scripts/design-system-report.mjs --with-tests",
    "design-system:check": "npm run tokens:check && node scripts/design-system-audit.mjs && vitest run
src/lib/designsystem.test.ts src/styles/hex.test.ts src/styles/tokens.test.ts",
    "design-system:baseline": "node scripts/design-system-audit.mjs --write-baseline"
  }
}
```

handoff/claude-code-toolkit/app/scripts/design-system-audit.mjs · 170 lines

```
#!/usr/bin/env node
/**
 * Semester design-system audit — read-only, no dependencies.
 *
 * Checks, against the repository's own authorities:
 *   1. Raw colour literals in .ts/.tsx (quoted #hex, rgb()/hsl()/oklch()) against the
 *      LEDGER in src/styles/hex.test.ts — the same count the census prints.
 *   2. CSS custom properties used with var() and no fallback that are defined nowhere
 *      (stylesheets, inline style objects, or tokensFor keys in src/lib/look.ts).
 *   3. tokens.css ↔ design-tokens/semester.tokens.json: every semantic name exported, and
 *      no exported semantic name missing from tokens.css.
 *   4. docs/design-system/FIGMA-MAPPING.md: every primitive.* / semantic.* path resolves in the export;
 *      valid / missing (obsolete) / unmapped-but-not-required candidates are reported separately.
 *   5. Warnings (minor, file:line): numeric zIndex, raw durations, cubic-bezier, raw box-shadow,
 *      numeric fontSize, numeric borderRadius in .ts/.tsx — each with the token to use instead.
 *
 * Usage (from app/):  node scripts/design-system-audit.mjs [--json] [--root <repo-root>]
 * Exit 1 when any violation is found. Warnings never fail.
 */
import { readFileSync, readdirSync, statSync, existsSync } from 'node:fs';
import { join, relative, resolve, dirname } from 'node:path';
import { fileURLToPath } from 'node:url';

const here = dirname(fileURLToPath(import.meta.url));

function walk(dir, exts, out = []) {
  if (!existsSync(dir)) return out;
  for (const name of readdirSync(dir)) {
    if (name === 'node_modules' || name.startsWith('.')) continue;
    const p = join(dir, name);
    const s = statSync(p);
    if (s.isDirectory()) walk(p, exts, out);
    else if (exts.some((e) => name.endsWith(e))) out.push(p);
  }
  return out;
}
const read = (p) => (existsSync(p) ? readFileSync(p, 'utf8') : '');
// Comments are blanked, newlines kept, so reported line numbers match the source file.
const noComments = (t) => t.replace(/\/\*[\s\S]*?\*\//g, (m) => m.replace(/[^\n]/g, '')).replace(/(^|[^:'"`])\/\/.*$/gm, '$1');
const lineOf = (t, i) => t.slice(0, i).split('\n').length;
const SUGGEST = { colour: 'an --app-* / --surface-* / --text-* / --status-* token, or a hex.test.ts LEDGER entry with a
reason', z: 'var(--layer-*) (styles/stacking.test.ts)', ms: 'var(--motion-*) (zeroed by reduced motion and data-calm)', ease:
'var(--ease-standard) / var(--ease-emphasized)', shadow: 'var(--elevation-*) / var(--lift-*)', font: 'var(--type-*) (scaled by
--text-scale; styles/scale.test.ts)', radius: 'var(--shape-*) / var(--r-*) (follow the Corners setting)' };
const isTest = (p) => /\.test\.(t|j)sx?$/.test(p) || /\/__fixtures__\//.test(p);

export function readLedger(hexTestText) {
  const out = {};
```

```
  for (const m of hexTestText.matchAll(/'([^']+\.tsx?)'\s*:\s*\{\s*count:\s*(\d+)/g)) out[m[1]] = Number(m[2]);
  return out;
}

export function semanticNames(tokensCss) {
  const clean = tokensCss.replace(/\/\*[\s\S]*?\*\//g, '');
  const block = /:root\s*\{([\s\S]*?)\n\}/.exec(clean)?.[1] ?? '';
  return [...block.matchAll(/(--[a-z0-9-]+)\s*:/g)].map((m) => m[1].slice(2));
}

export function mappingPaths(mappingMd) {
  const out = [];
  for (const line of mappingMd.split('\n')) {
    if (!line.startsWith('|')) continue;
    const cell = line.split('|').map((c) => c.trim()).find((c) => /^(primitive|semantic)\.[a-z0-9-]+$/.test(c));
    if (cell) out.push(cell);
  }
  return out;
}

export function audit({ root = resolve(here, '../..') } = {}) {
  const app = join(root, 'app');
  const src = join(app, 'src');
  const violations = [];
  const warnings = [];
  const stats = {};

  // 1. Raw colours vs the hex ledger
  const ledger = readLedger(read(join(src, 'styles/hex.test.ts')));
  const HEX = /['"`]#[0-9a-fA-F]{3,8}\b/g;
  const FN = /\b(rgba?|hsla?|oklch)\(\s*\d/g;
  const code = walk(src, ['.ts', '.tsx']).filter((p) => !isTest(p));
  let hexTotal = 0;
  for (const p of code) {
    const rel = relative(src, p).split('\\').join('/');
    if (rel === 'lib/look.ts' || rel.startsWith('lib/contrast')) continue; // the colour authorities
    const text = noComments(read(p));
    const n = text.match(HEX)?.length ?? 0;
    hexTotal += n;
    const allowed = ledger[rel] ?? 0;
    if (n > allowed) for (const m of text.matchAll(HEX)) violations.push({ check: 'raw-colour', severity: 'major', file: rel,
line: lineOf(text, m.index), value: m[0].slice(1), detail: n + ' hex literal(s); ledger allows ' + allowed + ' — use ' +
SUGGEST.colour });
    const fns = text.match(FN)?.length ?? 0;
    if (fns && !(rel in ledger)) warnings.push({ check: 'raw-colour-fn', file: rel, detail: fns + ' rgb()/hsl()/oklch()
literal(s) — prefer an --app-* / --status-* token' });
    const W = [['z-index', /zIndex:\s*(\d{2,})/g, SUGGEST.z], ['duration', /(?:transition|animation)(?:Duration)?:\s*['"`]
[^'"`]*?\b(\d{2,4}ms)/g, SUGGEST.ms], ['easing', /(cubic-bezier\([^)]*\))/g, SUGGEST.ease], ['shadow', /boxShadow:\s*['"`]
((?!var\()[^'"`]*\d+px[^'"`]*)['"`]/g, SUGGEST.shadow], ['font-size', /fontSize:\s*(\d+(?:\.\d+)?)\b(?!\s*\*)/g, SUGGEST.font],
['radius', /borderRadius:\s*(\d+)\b/g, SUGGEST.radius]];
    for (const [check, re, suggest] of W) for (const m of text.matchAll(re)) warnings.push({ check, severity: 'minor', file:
rel, line: lineOf(text, m.index), value: m[1], detail: 'raw value — prefer ' + suggest });
  }
  stats.hexLiterals = hexTotal;
  stats.ledgerTotal = Object.values(ledger).reduce((a, b) => a + b, 0);

  // 2. Undefined custom properties
  const css = walk(src, ['.css']);
  const defined = new Set();
  const addDefs = (t) => { for (const m of t.matchAll(/(--[a-z0-9-]+)\s*:/gi)) defined.add(m[1]); for (const m of
t.matchAll(/['"`](--[a-z0-9-]+)['"`]\s*[:\]]/gi)) defined.add(m[1]); };
  for (const p of css) addDefs(read(p));
  for (const p of code) addDefs(read(p));
  addDefs(read(join(src, 'lib/look.ts')));
  const used = new Map();
  const scan = (p, t) => { for (const m of t.matchAll(/var\(\s*(--[a-z0-9-]+)\s*(,)?/gi)) { if (m[2]) continue; if
(!used.has(m[1])) used.set(m[1], relative(src, p).split('\\').join('/')); } };
  for (const p of css) scan(p, noComments(read(p)));
  for (const p of code) scan(p, noComments(read(p)));
  const undefinedVars = [...used].filter(([v]) => !defined.has(v));
  for (const [v, file] of undefinedVars) violations.push({ check: 'undefined-var', severity: 'major', file, value: v, detail: v
+ ' is used without a fallback and defined nowhere' });
  stats.customPropertiesDefined = defined.size;

  // 3. tokens.css ↔ export
  const tokensCss = read(join(src, 'styles/tokens.css'));
  const exportPath = join(app, 'design-tokens/semester.tokens.json');
```

```
  let exported = null;
  try { exported = JSON.parse(read(exportPath)); } catch { violations.push({ check: 'export', file: 'design-
tokens/semester.tokens.json', detail: 'missing or not valid JSON — run npm run tokens:export' }); }
  if (exported) {
    const names = semanticNames(tokensCss);
    const sem = Object.keys(exported.semantic ?? {});
    for (const n of names) if (!sem.includes(n)) violations.push({ check: 'export-sync', file: 'styles/tokens.css', detail: '--
' + n + ' is not in semester.tokens.json — run npm run tokens:export' });
    for (const n of sem) if (!names.includes(n)) violations.push({ check: 'export-sync', file: 'design-
tokens/semester.tokens.json', detail: 'semantic.' + n + ' no longer exists in tokens.css — run npm run tokens:export' });
    stats.semanticTokens = sem.length;
    stats.primitiveTokens = Object.keys(exported.primitive ?? {}).length;
  }

  // 4. Figma mapping
  const MAP = 'docs/design-system/FIGMA-MAPPING.md';
  const mapping = read(join(root, MAP));
  const map = { valid: [], missing: [], unmappedCandidates: [] };
  if (!mapping) warnings.push({ check: 'figma-mapping', severity: 'minor', file: MAP, detail: 'not found — Figma parity is
unchecked' });
  else if (exported) {
    const paths = mappingPaths(mapping);
    for (const p of paths) {
      const [layer, key] = p.split('.');
      if (exported[layer] && key in exported[layer]) map.valid.push(p);
      else { map.missing.push(p); violations.push({ check: 'figma-mapping', severity: 'major', file: MAP, value: p, detail: p +
' does not resolve in semester.tokens.json (obsolete or misspelt)' }); }
    }
    const mapped = new Set(paths);
    map.unmappedCandidates = Object.keys(exported.semantic ?? {}).map((k) => 'semantic.' + k).filter((p) => !mapped.has(p));
  }
  stats.figma = { valid: map.valid.length, missing: map.missing.length, unmappedCandidates: map.unmappedCandidates.length };

  // Ratchet: app/design-system-baseline.json records today's raw-value warnings per check|file.
  // Anything above the baseline is a violation; a file below it must lower its entry (it only shrinks).
  const basePath = join(app, 'design-system-baseline.json');
  const counts = {};
  for (const w of warnings) if (w.line) counts[w.check + '|' + w.file] = (counts[w.check + '|' + w.file] ?? 0) + 1;
  stats.rawValueWarnings = Object.values(counts).reduce((x, y) => x + y, 0);
  if (existsSync(basePath)) {
    const base = JSON.parse(read(basePath));
    for (const [k, n] of Object.entries(counts)) if (n > (base[k] ?? 0)) { const [check, file] = k.split('|');
violations.push({ check: 'baseline-' + check, severity: 'major', file, detail: n + ' raw ' + check + ' value(s); baseline
allows ' + (base[k] ?? 0) + ' — use the suggested token' }); }
    for (const [k, n] of Object.entries(base)) if ((counts[k] ?? 0) < n) { const [check, file] = k.split('|');
violations.push({ check: 'baseline-stale', severity: 'minor', file, detail: check + ': baseline says ' + n + ', file has ' +
(counts[k] ?? 0) + ' — lower the entry (npm run design-system:baseline)' }); }
    stats.baselineTotal = Object.values(base).reduce((x, y) => x + y, 0);
  }
  return { ok: violations.length === 0, violations, warnings, stats, figma: map, rawCounts: counts };
}

if (import.meta.url === `file://${process.argv[1]}` || process.argv[1]?.endsWith('design-system-audit.mjs')) {
  const args = process.argv.slice(2);
  const rootArg = args.indexOf('--root');
  const result = audit(rootArg >= 0 ? { root: resolve(args[rootArg + 1]) } : {});
  if (args.includes('--write-baseline')) { const { writeFileSync } = await import('node:fs'); const p = join(rootArg >= 0 ?
resolve(args[rootArg + 1]) : resolve(here, '../..'), 'app/design-system-baseline.json'); writeFileSync(p,
JSON.stringify(Object.fromEntries(Object.entries(result.rawCounts).sort()), null, 2) + '\n'); console.log('Baseline written: '
+ p + ' (' + result.stats.rawValueWarnings + ' entries counted)'); process.exit(0); }
  if (args.includes('--json')) process.stdout.write(JSON.stringify(result, null, 2) + '\n');
  else {
    const loc = (x) => x.file + (x.line ? ':' + x.line : '');
    for (const v of result.violations) console.log('× [' + v.check + '] ' + loc(v) + (v.value ? ' ' + v.value : '') + ' — ' +
v.detail);
    for (const w of result.warnings) console.log('! [' + w.check + '] ' + loc(w) + (w.value ? ' ' + w.value : '') + ' — ' +
w.detail);
    console.log((result.ok ? '✓ ' : '× ') + result.violations.length + ' violation(s), ' + result.warnings.length + '
warning(s) · ' + JSON.stringify(result.stats));
  }
  process.exit(result.ok ? 0 : 1);
}
```

handoff/claude-code-toolkit/app/scripts/design-system-report.mjs · 66 lines

```
#!/usr/bin/env node
/**
 * A CI-readable summary of the design-system audit, as Markdown.
 * Usage (from app/): node scripts/design-system-report.mjs [--out design-system-report.md] [--root <repo-root>]
 * Exits with the audit's status so CI can gate on it.
 */
import { writeFileSync, mkdirSync } from 'node:fs';
import { resolve, join, dirname } from 'node:path';
import { spawnSync } from 'node:child_process';
import { fileURLToPath } from 'node:url';
const appDir = resolve(dirname(fileURLToPath(import.meta.url)), '..');
/** Runs the existing contract tests and records pass/fail — never weakens or rewrites them. */
export function contractStatus(files) {
  const out = {};
  for (const f of files) { const p = spawnSync('npx', ['vitest', 'run', f], { cwd: appDir, encoding: 'utf8' }); out[f] =
p.status === 0 ? 'pass' : 'fail'; }
  return out;
}
const CONTRACTS = ['src/lib/tokenexport.test.ts', 'src/styles/tokens.test.ts', 'src/styles/hex.test.ts',
'src/styles/stacking.test.ts', 'src/styles/taps.test.ts', 'src/styles/breakpoints.test.ts', 'src/styles/motion.test.ts',
'src/styles/scale.test.ts'];
import { audit } from './design-system-audit.mjs';

export function report(result, contracts = {}) {
  const s = result.stats;
  const by = (list) => list.reduce((m, x) => ((m[x.check] = (m[x.check] || 0) + 1), m), {});
  const lines = [
    '# Semester design-system report',
    '',
    '**Status:** ' + (result.ok ? '✓ passing' : '× ' + result.violations.length + ' violation(s)') + ' · ' +
result.warnings.length + ' warning(s)',
    '',
    '| Measure | Value |', '| --- | --- |',
    '| Hex literals in .ts/.tsx | ' + (s.hexLiterals ?? '—') + ' (ledger allows ' + (s.ledgerTotal ?? '—') + ') |',
    '| Custom properties defined | ' + (s.customPropertiesDefined ?? '—') + ' |',
    '| Semantic tokens exported | ' + (s.semanticTokens ?? '—') + ' |',
    '| Primitive tokens exported | ' + (s.primitiveTokens ?? '—') + ' |',
    '| Figma mappings valid / missing / unmapped candidates | ' + (s.figma ? s.figma.valid + ' / ' + s.figma.missing + ' / ' +
s.figma.unmappedCandidates : 'no mapping') + ' |',
    '| Token export (tokenexport.test.ts) | ' + (contracts['src/lib/tokenexport.test.ts'] ?? 'not run (pass --with-tests)') + '
|',
    '| Findings blocker / major / minor | ' + ['blocker', 'major', 'minor'].map((k) => [...result.violations,
...result.warnings].filter((x) => x.severity === k).length).join(' / ') + ' |',
    '',
    ...(Object.keys(contracts).length ? ['## Existing style contracts', '', '| Test | Result |', '| --- | --- |',
...Object.entries(contracts).map(([f, st]) => '| ' + f + ' | ' + st + ' |'), ''] : []),
    '',
    '## Violations by check', '',
    ...(result.violations.length ? Object.entries(by(result.violations)).map(([k, n]) => '- ' + k + ': ' + n) : ['None.']),
    '',
    ...(result.violations.length ? ['| Check | File | Detail |', '| --- | --- | --- |', ...result.violations.map((v) => '| ' +
v.check + ' | ' + v.file + (v.line ? ':' + v.line : '') + ' | ' + (v.value ?? '') + ' ' + v.detail + ' |'), ''] : []),
    '## Warnings', '',
    ...(result.warnings.length ? ['| Check | File | Detail |', '| --- | --- | --- |', ...result.warnings.slice(0, 200).map((w)
=> '| ' + w.check + ' | ' + w.file + (w.line ? ':' + w.line : '') + ' | ' + (w.value ?? '') + ' ' + w.detail + ' |')] :
['None.']),
    '',
  ];
  return lines.join('\n');
}

if (process.argv[1]?.endsWith('design-system-report.mjs')) {
  const args = process.argv.slice(2);
  const r = args.indexOf('--root');
  const result = audit(r >= 0 ? { root: resolve(args[r + 1]) } : {});
  const contracts = args.includes('--with-tests') ? contractStatus(CONTRACTS) : {};
  const md = report(result, contracts);
  const o = args.indexOf('--out');
  const out = o >= 0 ? args[o + 1] : join(appDir, 'reports/design-system/report.md');
  mkdirSync(dirname(out), { recursive: true });
  writeFileSync(out, md);
  writeFileSync(out.replace(/\.md$/, '.json'), JSON.stringify({ ...result, contracts }, null, 2) + '\n');
  console.log('Wrote ' + out);
  if (Object.values(contracts).includes('fail')) process.exit(1);
```

```
  process.exit(result.ok ? 0 : 1);
}
```

handoff/claude-code-toolkit/app/src/lib/designsystem.test.ts · 116 lines

```
import { describe, expect, it } from 'vitest';
import { mkdtempSync, mkdirSync, writeFileSync, readFileSync } from 'node:fs';
import { tmpdir } from 'node:os';
import { join } from 'node:path';
// @ts-expect-error — plain ESM script, no types
import { audit, mappingPaths, readLedger, semanticNames } from '../../scripts/design-system-audit.mjs';
// @ts-expect-error — plain ESM script, no types
import { report } from '../../scripts/design-system-report.mjs';
import { buildTokenExport, serialise } from './tokenexport';

/**
 * The design-system audit is a guard only if it has failed at least once (CLAUDE.md).
 * Each check is proven against a fixture that violates it, and the real tree is held clean.
 */

const repo = new URL('../../..', import.meta.url).pathname;

function fixture(files: Record<string, string>) {
  const root = mkdtempSync(join(tmpdir(), 'semester-ds-'));
  for (const [p, text] of Object.entries(files)) {
    const full = join(root, p);
    mkdirSync(join(full, '..'), { recursive: true });
    writeFileSync(full, text);
  }
  return root;
}

const TOKENS = ':root {\n  --surface-base: var(--app-panel);\n  --text-primary: var(--app-fg);\n}\n';
const LOOK = "export const x = { '--app-panel': '#12141a', '--app-fg': '#eceef2' };\n";
const EXPORT = JSON.stringify({ primitive: { 'app-panel': {}, 'app-fg': {} }, semantic: { 'surface-base': {}, 'text-primary':
{} } });
const HEXTEST = "export const LEDGER = { 'components/Ok.tsx': { count: 1, why: 'x' } };\n";
const base = {
  'app/src/styles/tokens.css': TOKENS,
  'app/src/lib/look.ts': LOOK,
  'app/src/styles/hex.test.ts': HEXTEST,
  'app/design-tokens/semester.tokens.json': EXPORT,
  'app/src/components/Ok.tsx': "export const C = () => <div style={{ color: '#fff', background: 'var(--surface-base)' }}
/>;\n",
  'docs/design-system/FIGMA-MAPPING.md': '| semantic.surface-base | surface/base |\n| primitive.app-fg | base/fg |\n',
};

describe('the committed export is generated, not edited', () => {
  it('equals buildTokenExport of the current stylesheets', () => {
    const tokensCss = readFileSync(join(repo, 'app/src/styles/tokens.css'), 'utf8');
    const appCss = readFileSync(join(repo, 'app/src/styles/app.css'), 'utf8');
    const committed = readFileSync(join(repo, 'app/design-tokens/semester.tokens.json'), 'utf8');
    expect(committed).toBe(serialise(buildTokenExport(tokensCss, appCss)));
  });
});

describe('design-system audit', () => {
  it('passes a valid fixture', () => {
    expect(audit({ root: fixture(base) }).violations).toEqual([]);
  });

  it('reports file, line and value for a violation', () => {
    const r = audit({ root: fixture({ ...base, 'app/src/components/Bad.tsx': "// one\nexport const B = { color: '#ff0000' };\n"
}) });
    expect(r.violations[0]).toMatchObject({ file: 'components/Bad.tsx', line: 2, value: '#ff0000' });
  });

  it('separates valid and missing Figma mappings', () => {
    const r = audit({ root: fixture({ ...base, 'docs/design-system/FIGMA-MAPPING.md': '| semantic.surface-base |\n|
semantic.nope |\n' }) });
    expect(r.figma.valid).toEqual(['semantic.surface-base']);
    expect(r.figma.missing).toEqual(['semantic.nope']);
    expect(r.figma.unmappedCandidates).toEqual(['semantic.text-primary']);
  });
```

```
  it('fails a hex literal beyond the hex.test.ts ledger', () => {
    const r = audit({ root: fixture({ ...base, 'app/src/components/Bad.tsx': "export const B = { color: '#ff0000' };\n" }) });
    expect(r.ok).toBe(false);
    expect(r.violations.map((v: { check: string }) => v.check)).toContain('raw-colour');
  });

  it('fails a custom property used without a fallback and defined nowhere', () => {
    const r = audit({ root: fixture({ ...base, 'app/src/styles/x.css': '.a { color: var(--nope); }\n' }) });
    expect(r.violations.some((v: { detail: string }) => v.detail.startsWith('--nope'))).toBe(true);
  });

  it('allows an undefined property when a fallback is given', () => {
    const r = audit({ root: fixture({ ...base, 'app/src/styles/x.css': '.a { color: var(--nope, red); }\n' }) });
    expect(r.violations.filter((v: { check: string }) => v.check === 'undefined-var')).toEqual([]);
  });

  it('fails when tokens.css and the export disagree', () => {
    const r = audit({ root: fixture({ ...base, 'app/src/styles/tokens.css': TOKENS.replace('}', '  --surface-new: var(--app-
panel);\n}') }) });
    expect(r.violations.map((v: { check: string }) => v.check)).toContain('export-sync');
  });

  it('fails a Figma mapping row that does not resolve', () => {
    const r = audit({ root: fixture({ ...base, 'docs/design-system/FIGMA-MAPPING.md': '| semantic.surface-gone | surface/gone
|\n' }) });
    expect(r.violations.map((v: { check: string }) => v.check)).toContain('figma-mapping');
  });

  it('fails a raw value above the baseline and passes at it', () => {
    const files = { ...base, 'app/src/components/Z.tsx': "export const Z = () => <div style={{ zIndex: 80 }} />;\n" };
    expect(audit({ root: fixture({ ...files, 'app/design-system-baseline.json': '{}' }) }).violations.map((v: { check: string
}) => v.check)).toContain('baseline-z-index');
    expect(audit({ root: fixture({ ...files, 'app/design-system-baseline.json': JSON.stringify({ 'z-index|components/Z.tsx': 1
}) }) }).ok).toBe(true);
  });

  it('flags a stale baseline so it can only shrink', () => {
    const r = audit({ root: fixture({ ...base, 'app/design-system-baseline.json': JSON.stringify({ 'z-
index|components/Gone.tsx': 3 }) }) });
    expect(r.violations.map((v: { check: string }) => v.check)).toContain('baseline-stale');
  });

  it('reads the real ledger and the real mapping', () => {
    expect(Object.keys(readLedger(readFileSync(join(repo, 'app/src/styles/hex.test.ts'), 'utf8'))).length).toBeGreaterThan(0);
    expect(semanticNames(readFileSync(join(repo, 'app/src/styles/tokens.css'), 'utf8'))).toContain('surface-base');
    expect(mappingPaths(readFileSync(join(repo, 'docs/design-system/FIGMA-MAPPING.md'), 'utf8')).length).toBeGreaterThan(10);
  });

  it('holds the real tree clean, and the report says so', () => {
    const r = audit({ root: repo });
    expect(r.violations, report(r)).toEqual([]);
    expect(report(r)).toContain('✓ passing');
  });
});
```

handoff/claude-code-toolkit/gitignore.addition · 3 lines

```
# Generated design-system reports (uploaded as CI artifacts, not committed)
app/reports/
```

handoff/install.sh · 21 lines

```
#!/usr/bin/env bash
# Install the Semester handoff into the repo. Run from the repo root: bash design/handoff/install.sh
# Safe by default: never overwrites an existing file unless FORCE=1. Strips the .txt suffix design files carry.
set -euo pipefail
SRC="${SRC:-design/handoff}"; DEST="${DEST:-.}"; APP="${APP:-app}"
[ -d "$SRC" ] || { echo "Missing $SRC. Copy the design project export to ./design first."; exit 1; }
copy() { local from="$1" to="$2"; mkdir -p "$(dirname "$to")"; if [ -e "$to" ] && [ "${FORCE:-0}" != 1 ]; then echo "SKIP
(exists) $to"; else cp "$from" "$to"; echo "OK   $to"; fi; }
strip() { local p="$1"; echo "${p%.txt}"; }
# 1 semester-core → $APP/src/lib/core
for f in "$SRC"/semester-core/*.ts.txt; do copy "$f" "$DEST/$APP/src/lib/core/$(basename "$(strip "$f")")"; done
```

```
# 2 semester-platform: migrations, lib, api routes, tests, middleware, seed
for f in "$SRC"/semester-platform/supabase/migrations/*.sql.txt; do copy "$f" "$DEST/supabase/migrations/$(basename "$(strip
"$f")")"; done
for f in $(cd "$SRC/semester-platform" && find lib app tests -name '*.txt'); do copy "$SRC/semester-platform/$f"
"$DEST/$APP/$(strip "$f")"; done
copy "$SRC/semester-platform/middleware.security.ts.txt" "$DEST/$APP/middleware.security.ts"
copy "$SRC/semester-platform/supabase/seed.sql.txt" "$DEST/supabase/seed.sql"
copy "$SRC/semester-platform/tests/helpers.ts.txt" "$DEST/$APP/tests/helpers.ts"
copy "$SRC/semester-platform/.env.test.txt" "$DEST/$APP/.env.test"
copy "$SRC/semester-platform/.env.example.txt" "$DEST/$APP/.env.example"
copy "$SRC/semester-platform/.github/ci-security-job.yml.txt" "$DEST/.github/ci-security-job.yml"
echo; echo "Installed. Next: bash design/handoff/preflight.sh && bash design/handoff/verify.sh"
```

handoff/preflight.sh · 13 lines

```
#!/usr/bin/env bash
# Preflight: confirms tools and project shape before any stream runs. Exits non-zero with the exact fix.
set -uo pipefail; fail=0
need() { command -v "$1" >/dev/null 2>&1 || { echo "MISSING $1 — $2"; fail=1; }; }
need node "install Node 20+"; need npm "comes with Node"; need npx "comes with Node"; need git "install git"; need docker
"Supabase local needs Docker running"
node -e 'process.exit(+process.versions.node.split(".")[0] < 20)' || { echo "Node 20+ required"; fail=1; }
npx --yes supabase --version >/dev/null 2>&1 || { echo "MISSING supabase CLI — npx supabase works once network is available";
fail=1; }
[ -d supabase ] || { echo "No supabase/ — run: npx supabase init"; fail=1; }
[ -f "${APP:-app}/package.json" ] || { echo "No ${APP:-app}/package.json — set APP=<dir> to the web app"; fail=1; }
grep -q '"next"' "${APP:-app}/package.json" 2>/dev/null || echo "NOTE  ${APP:-app} is not a Next.js app. API routes in
app/api/* need a Next.js app (see BUILD.md §2 'App target')."
git diff --quiet || echo "NOTE  working tree has uncommitted changes; streams expect a clean branch"
[ $fail = 0 ] && echo "PREFLIGHT OK" || { echo "PREFLIGHT FAILED"; exit 1; }
```

handoff/verify.sh · 14 lines

```
#!/usr/bin/env bash
# Every gate, in order. Stops at the first failure and prints which gate failed.
set -euo pipefail; APP="${APP:-app}"
step() { echo; echo "== $1"; }
step "1 database reset (migrations 010–080 + seed)"; npx supabase db reset
step "2 typecheck"; (cd "$APP" && npx tsc --noEmit)
step "3 lint"; (cd "$APP" && npm run lint)
step "4 unit tests (semester-core, validate, AI red-team)"; (cd "$APP" && npx vitest run src/lib/core tests/validate.test.ts
tests/ai-redteam.test.ts)
step "5 RLS + security + command tests"; (cd "$APP" && npx dotenv -e .env.test -- npx vitest run tests/rls.test.ts
tests/security.test.ts tests/commands.test.ts)
step "6 design-system check"; (cd "$APP" && npm run design-system:check)
step "7 build"; (cd "$APP" && npm run build)
step "8 secret scan"; npx --yes gitleaks detect --no-banner || { echo "gitleaks missing or found secrets"; exit 1; }
echo; echo "ALL GATES PASSED — record this output in docs/execute/<stream>-report.md"
```

handoff/manifest.json · 93 lines

```
{
  "version": "2026-10-05",
  "entry": "handoff/BUILD.md",
  "scripts": [
    "handoff/preflight.sh",
    "handoff/install.sh",
    "handoff/verify.sh"
  ],
  "migrationOrder": [
    "010_platform_core",
    "020_community",
    "030_marketplace",
    "040_ops",
    "050_security_hardening",
    "060_command_functions",
    "070_ai_gateway",
    "080_sources_storage_delegations"
  ],
  "files": [
    {
```

```
      "from": "semester-core/*.ts.txt",
      "to": "<APP>/src/lib/core/",
      "check": "unit tests"
    },
    {
      "from": "semester-platform/supabase/migrations/010–080",
      "to": "supabase/migrations/",
      "check": "db reset + RLS/security/command tests"
    },
    {
      "from": "semester-platform/supabase/seed.sql.txt",
      "to": "supabase/seed.sql",
      "check": "db reset"
    },
    {
      "from": "semester-platform/lib/**",
      "to": "<APP>/lib/",
      "check": "typecheck"
    },
    {
      "from": "semester-platform/app/api/**",
      "to": "<APP>/app/api/",
      "check": "typecheck + build"
    },
    {
      "from": "semester-platform/tests/**",
      "to": "<APP>/tests/",
      "check": "vitest"
    },
    {
      "from": "semester-platform/middleware.security.ts.txt",
      "to": "<APP>/middleware.security.ts (merge into middleware.ts)",
      "check": "build; headers present"
    },
    {
      "from": "semester-platform/.env.example.txt, .env.test.txt",
      "to": "<APP>/",
      "check": "—"
    },
    {
      "from": "semester-platform/.github/ci-security-job.yml.txt",
      "to": ".github/ (merge into ci.yml)",
      "check": "CI"
    },
    {
      "from": "claude-code-toolkit/**",
      "to": "repo root per its EXECUTE.md",
      "check": "design-system:check"
    }
  ],
  "pilotCriticalPath": [
    "00",
    "01",
    "02",
    "03",
    "06",
    "05",
    "09",
    "10"
  ],
  "humanOnly": [
    "insurance",
    "DPA",
    "HECVAT",
    "attorney review",
    "IdP metadata",
    "contacts",
    "baseline approval",
    "PR approvals",
    "cutover approval",
    "AI provider terms"
  ]
}
```

APPENDIX

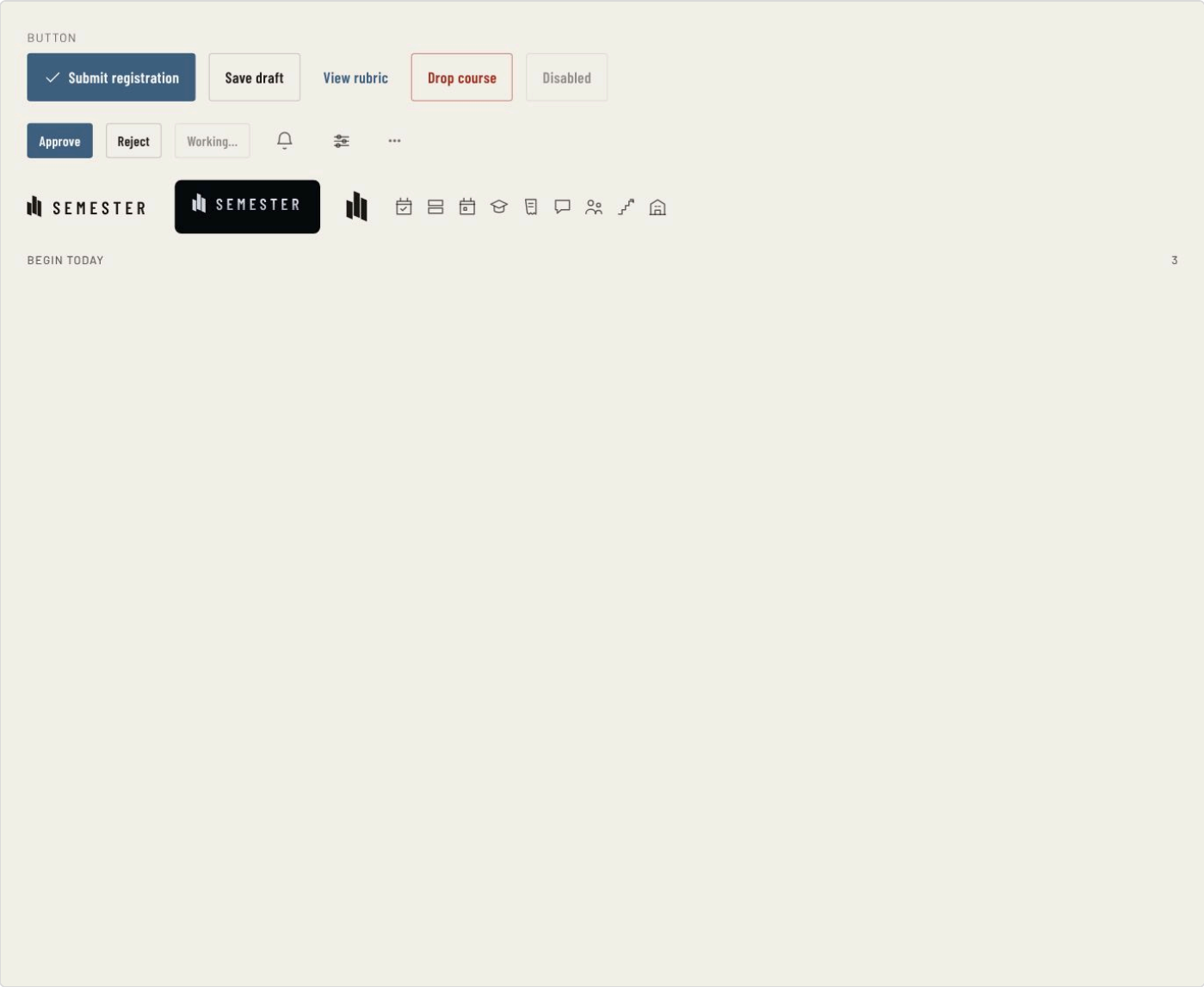
Archive

Everything moved aside during the migration, still rendered as it was.

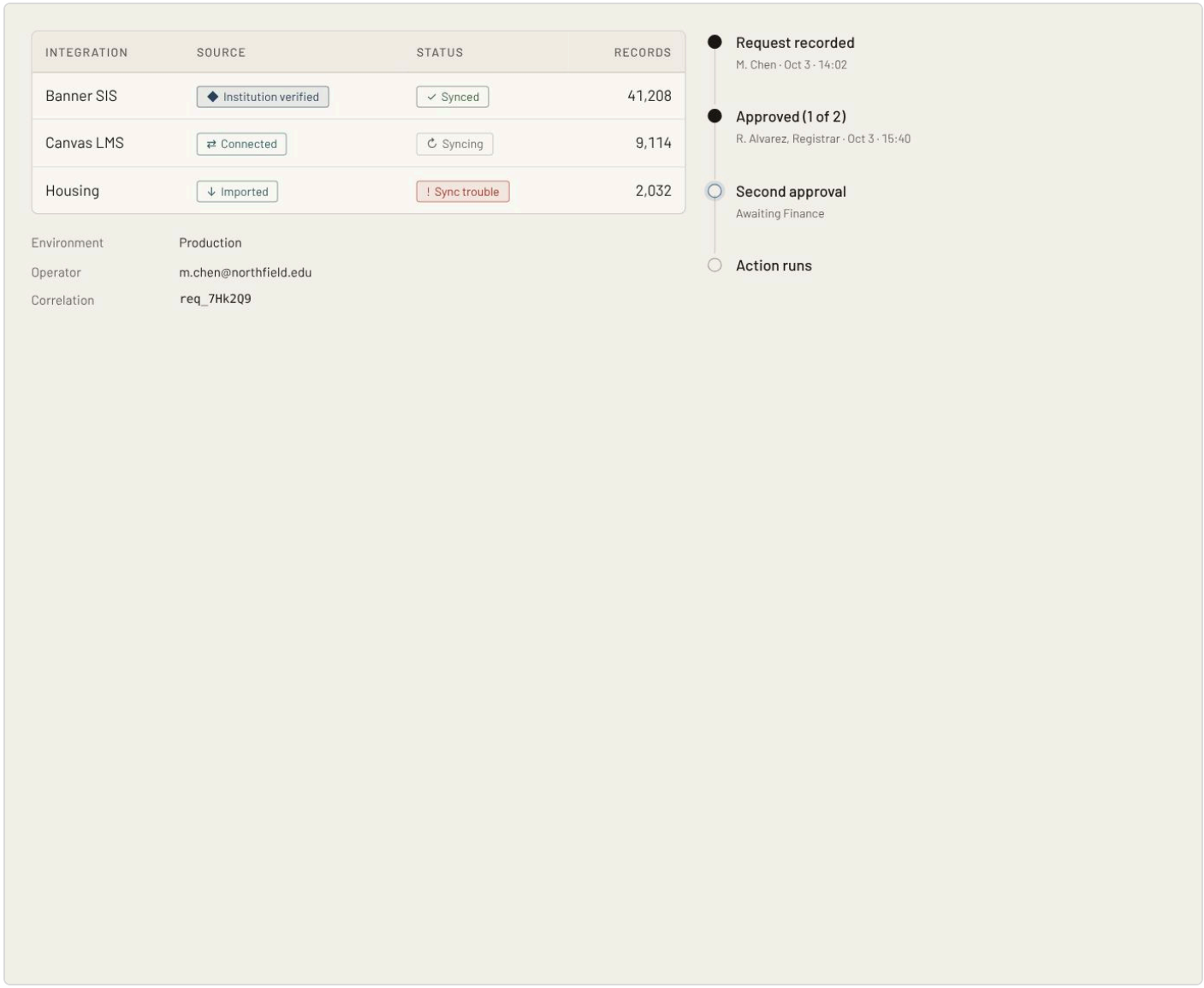
Archive: the pre-migration cards and specimens

The original group cards and specimen pages, as they rendered before the system moved to the new format. They are kept in archived/.

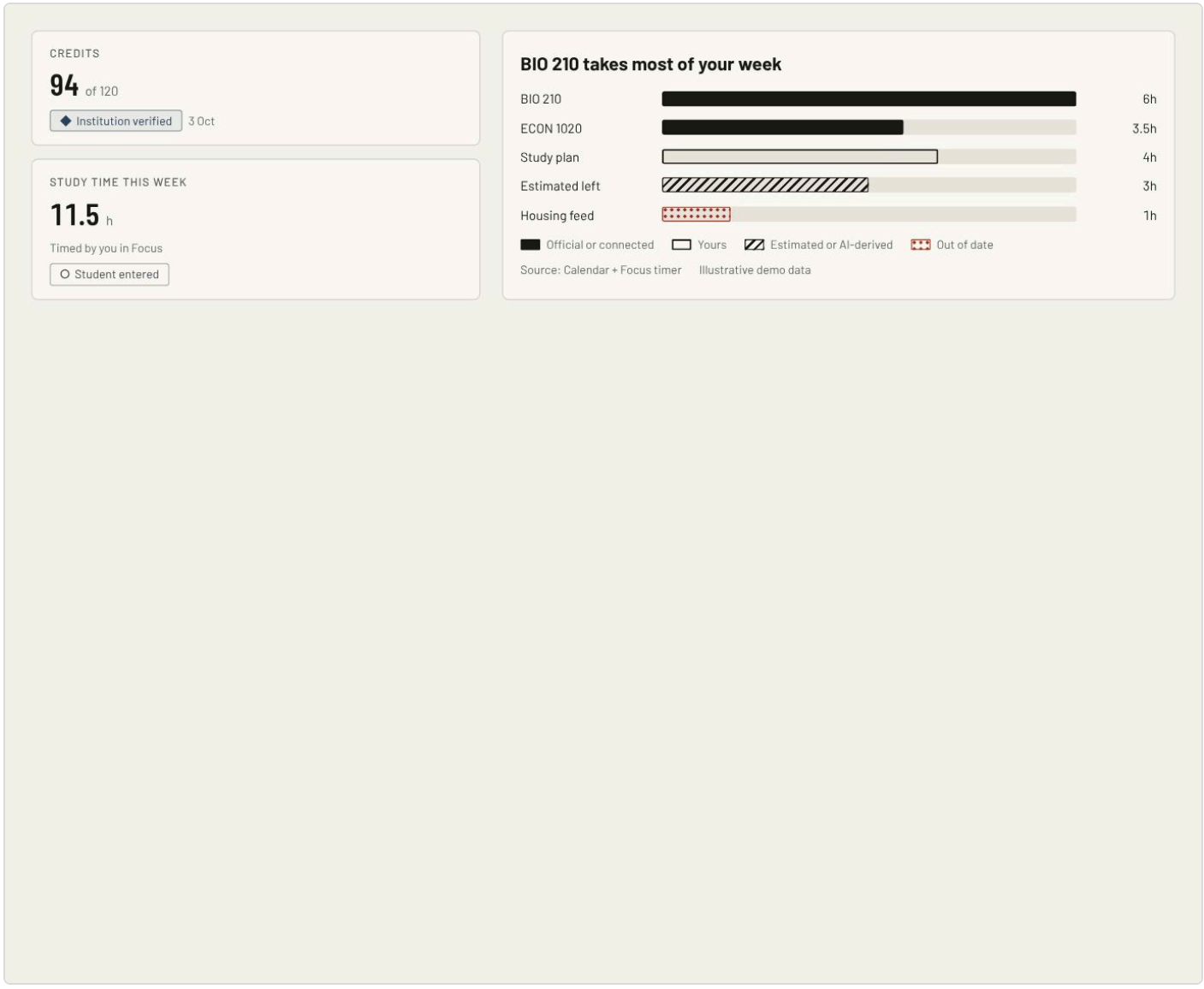
Group card: core



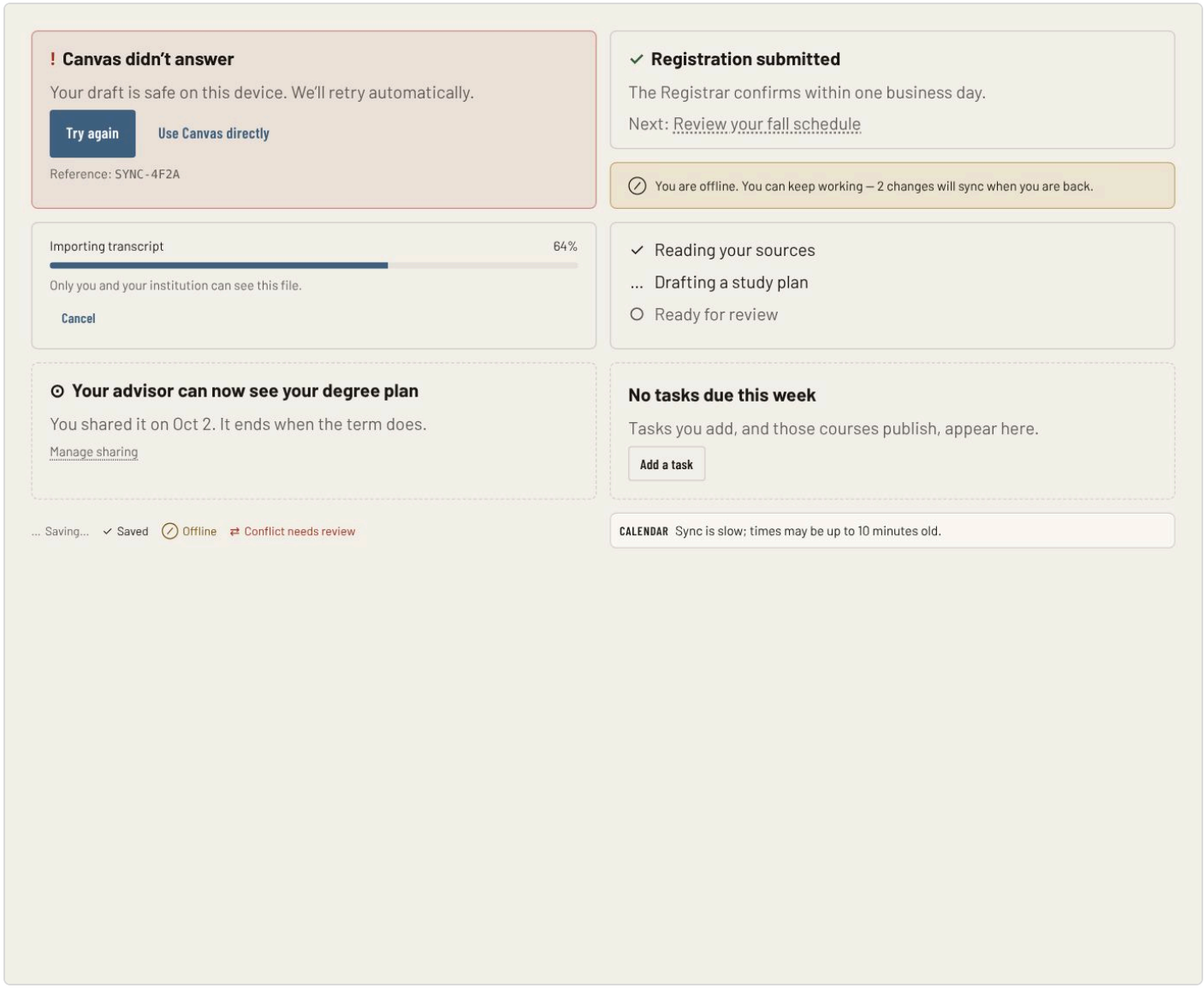
Group card: data



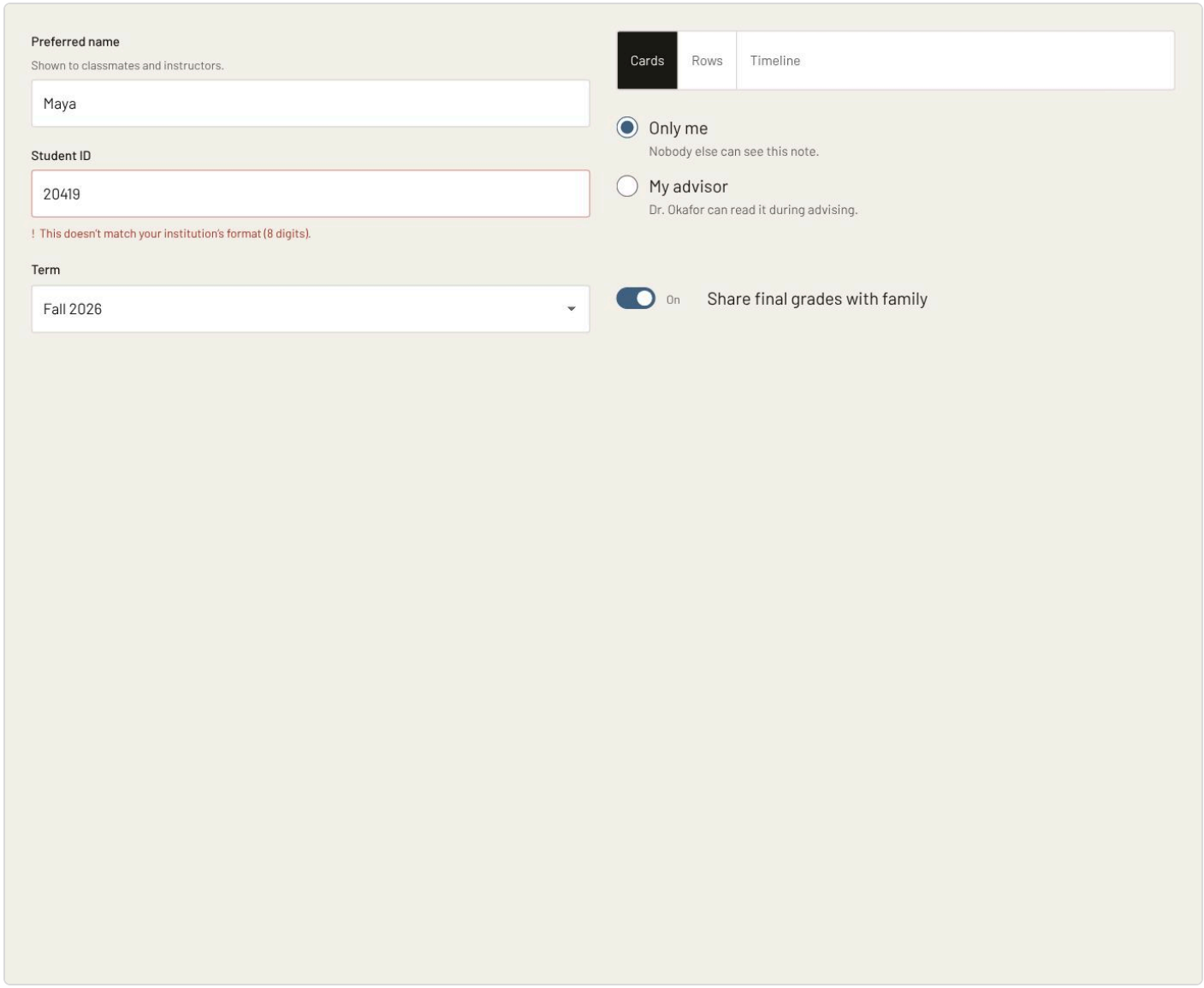
Group card: data-viz



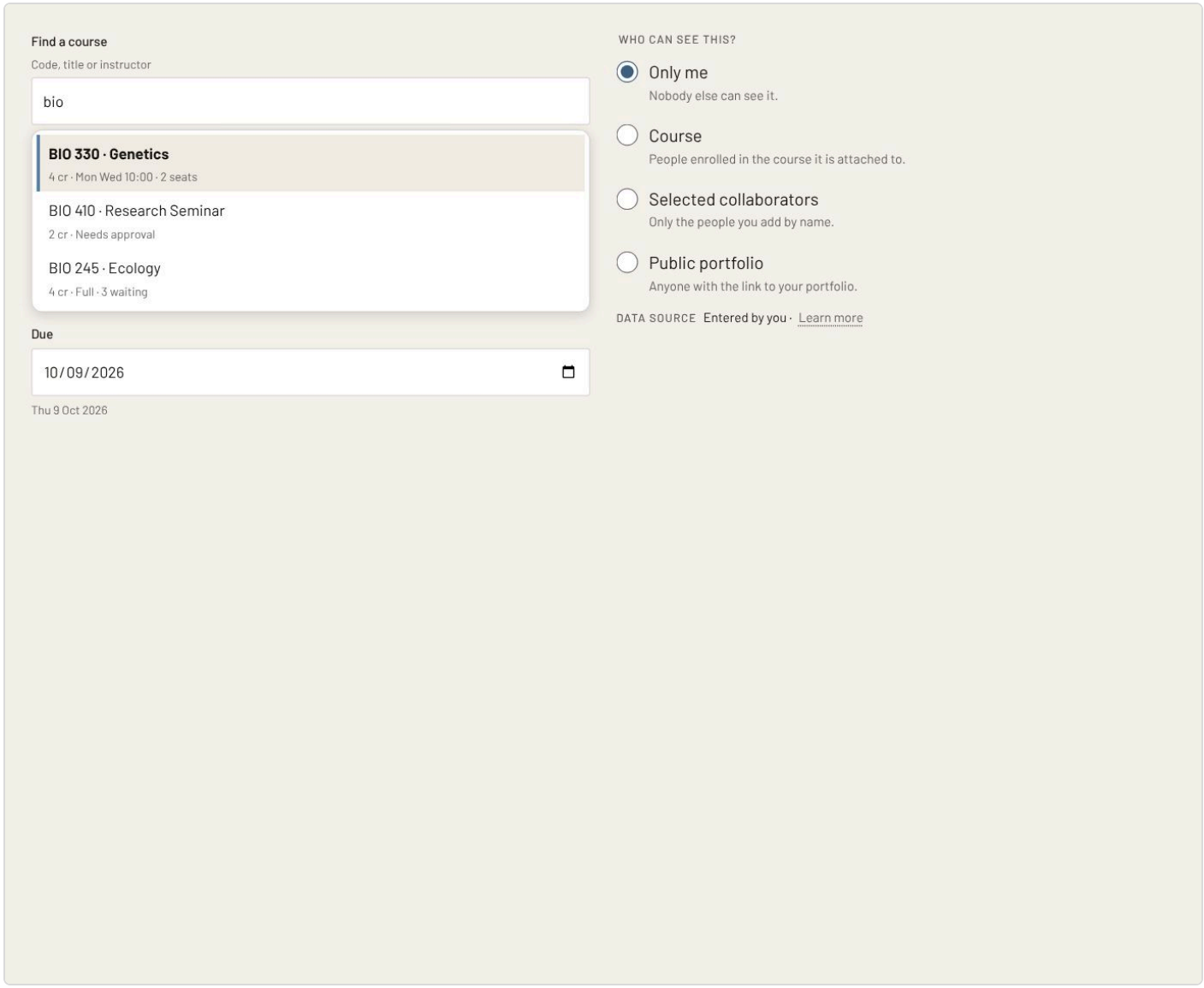
Group card: feedback



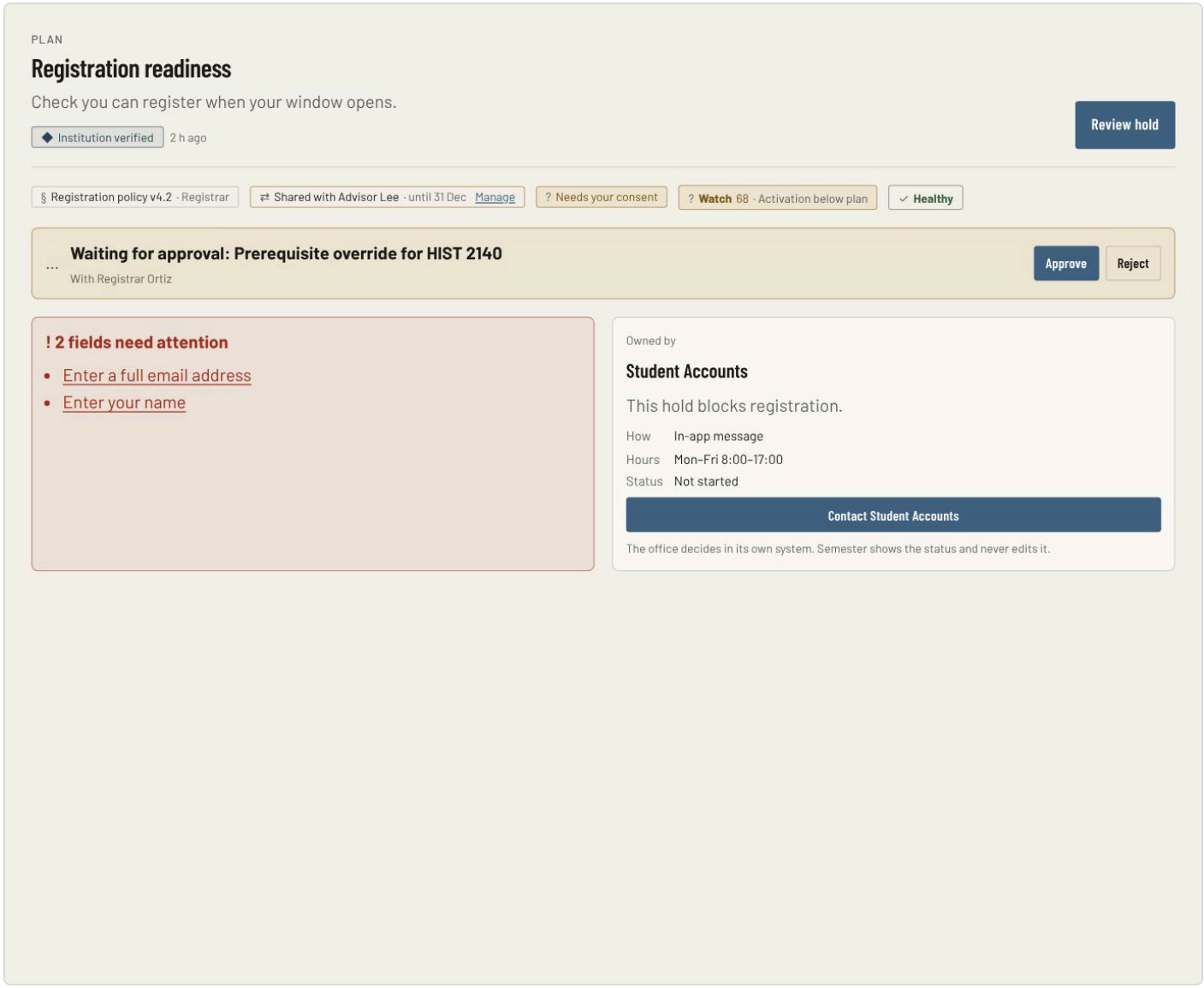
Group card: forms



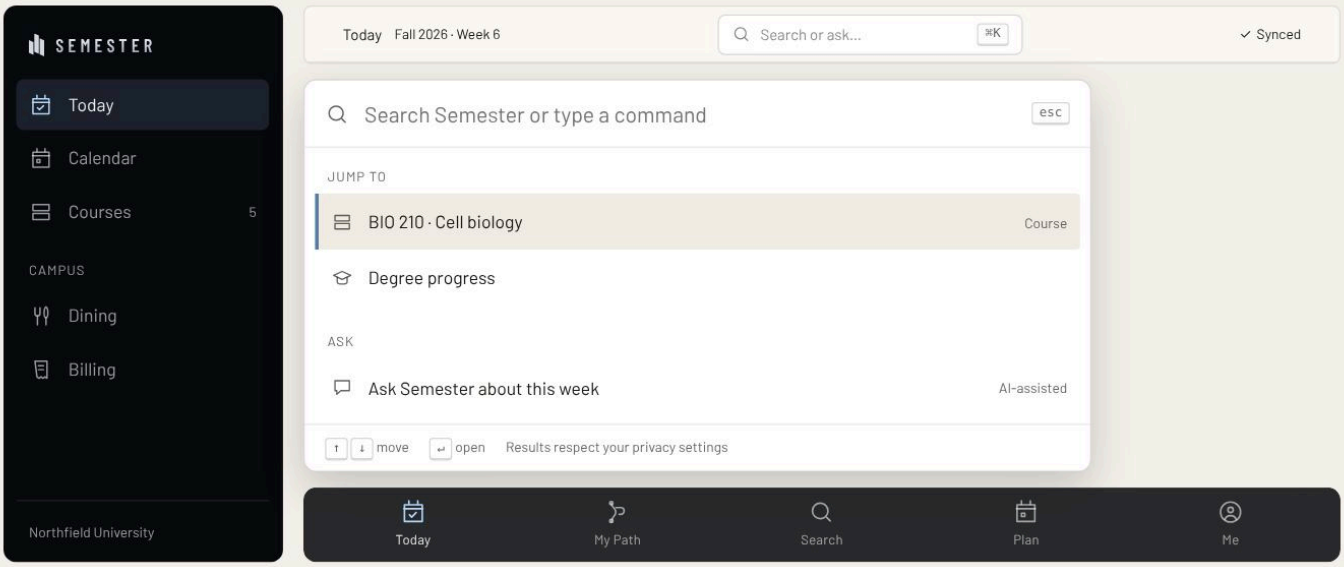
Group card: forms-advanced



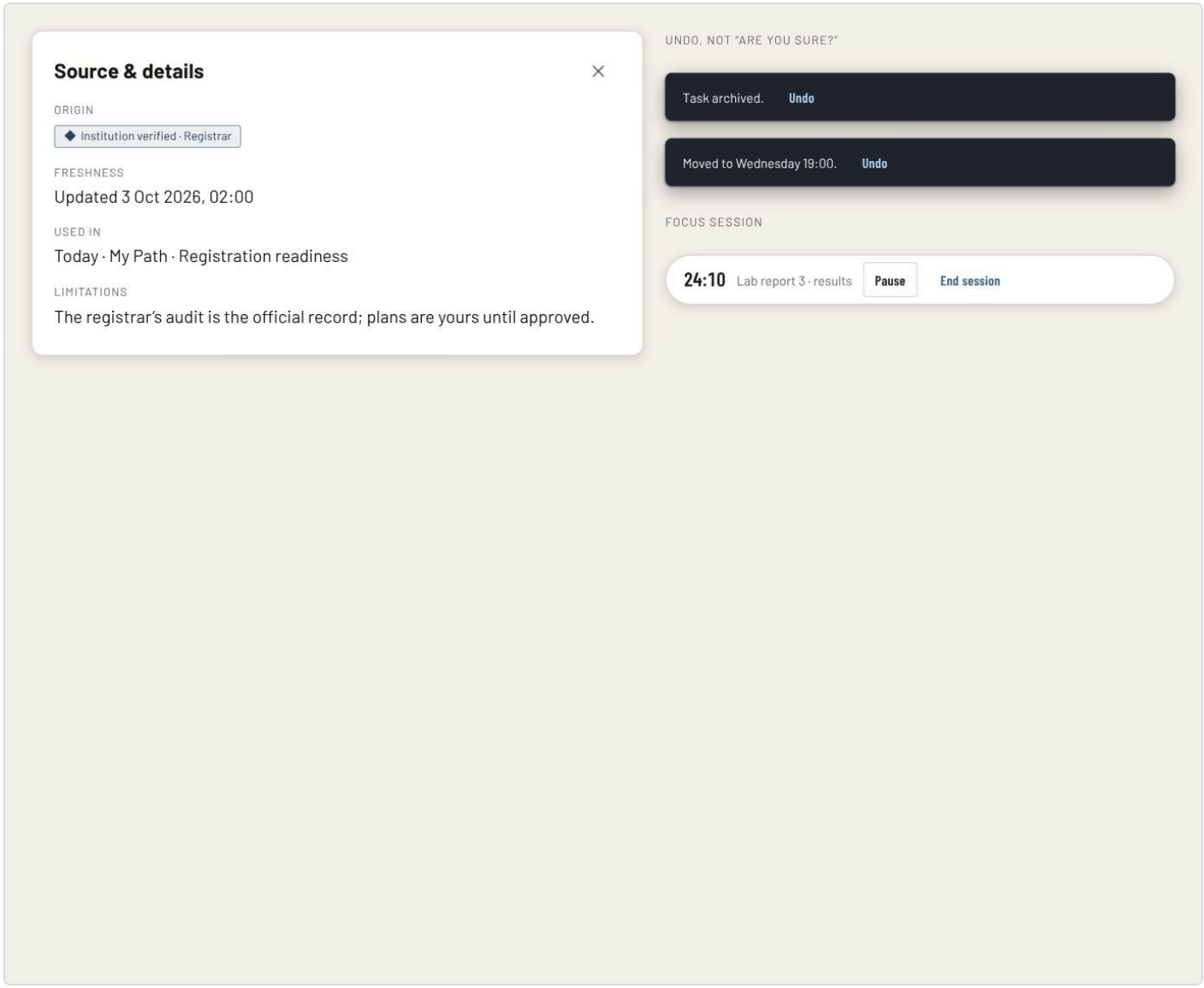
Group card: governance



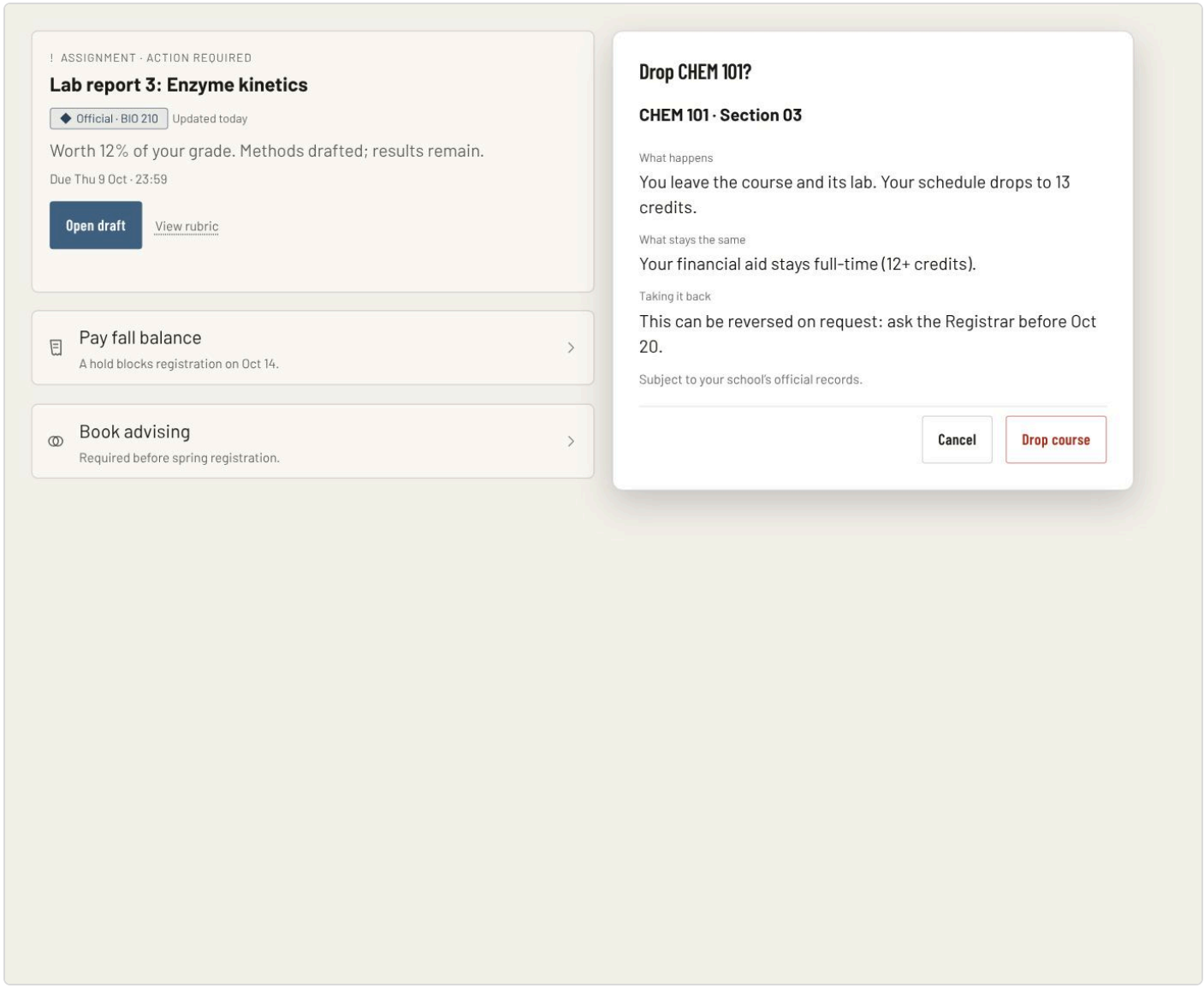
Group card: navigation



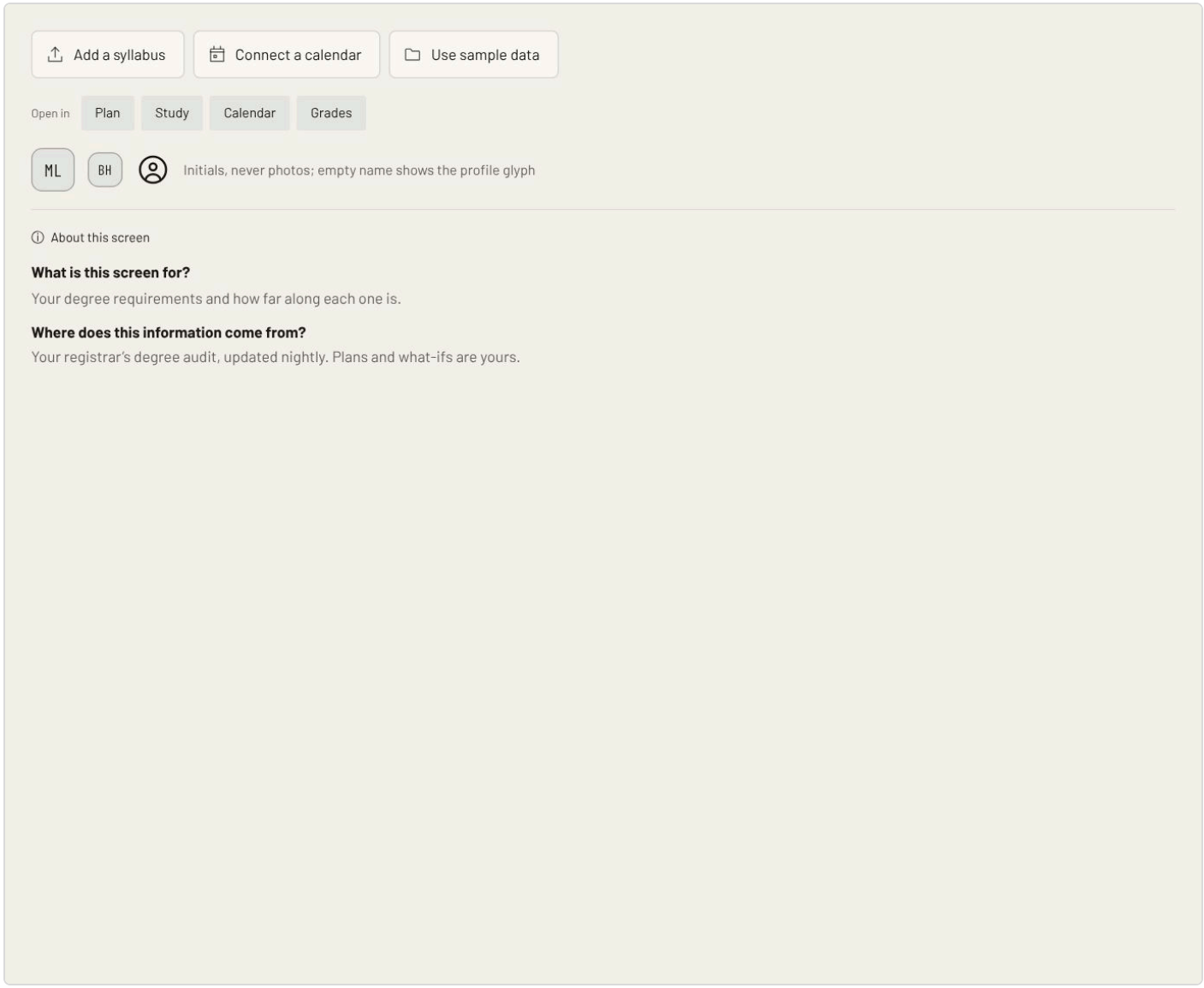
Group card: overlays



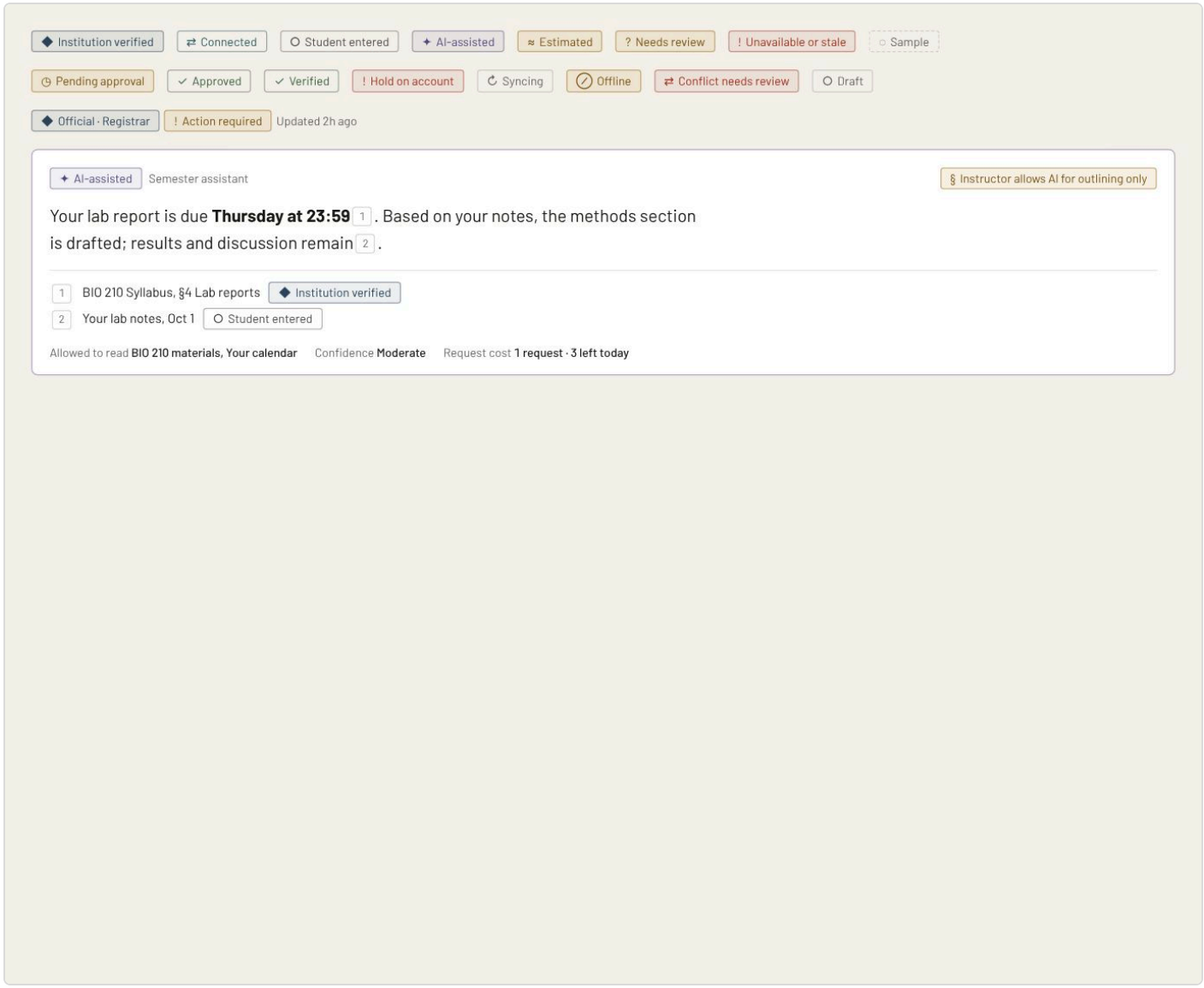
Group card: surfaces



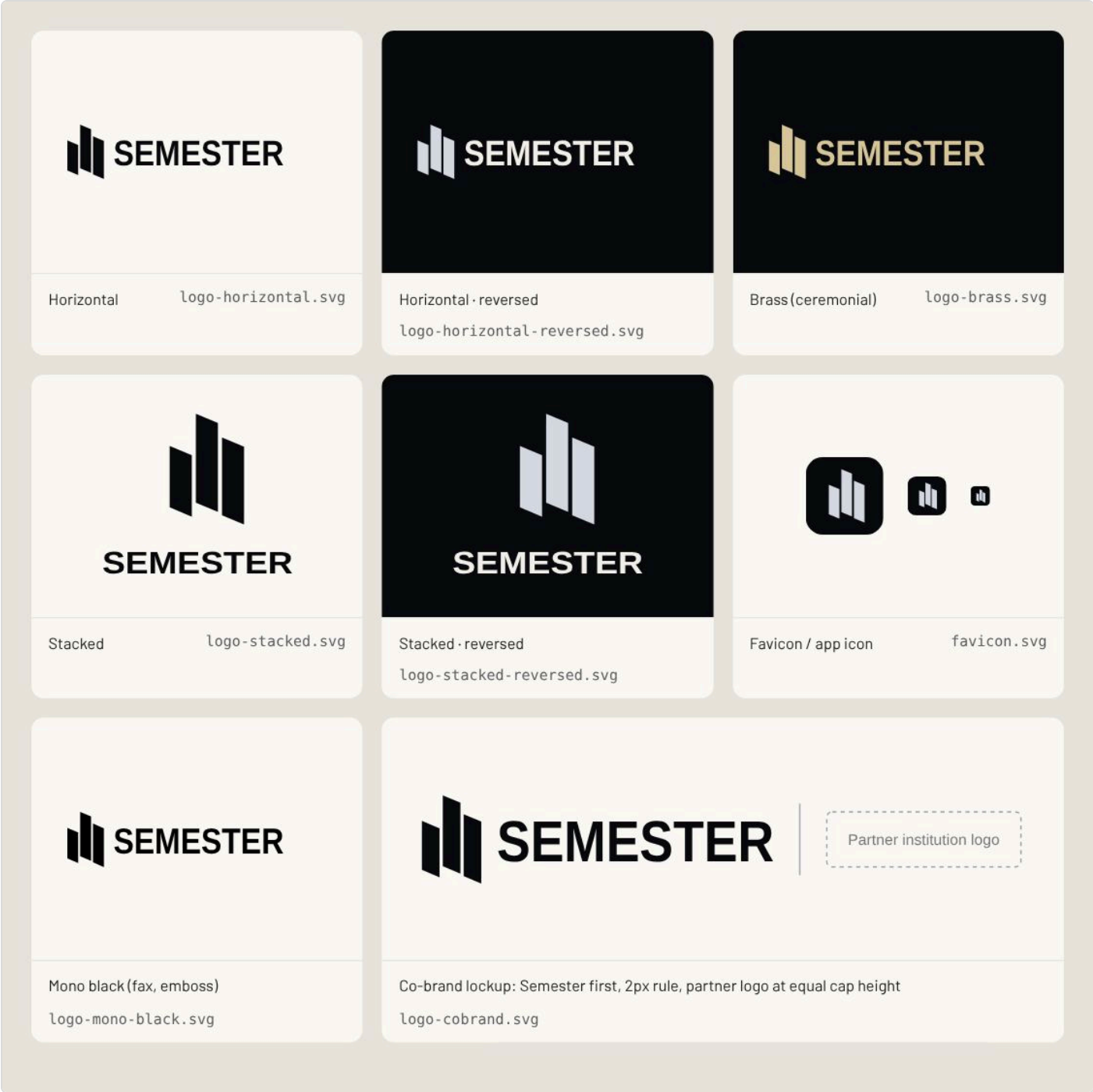
Group card: surfaces-more



Group card: trust



Specimen: brand-logos



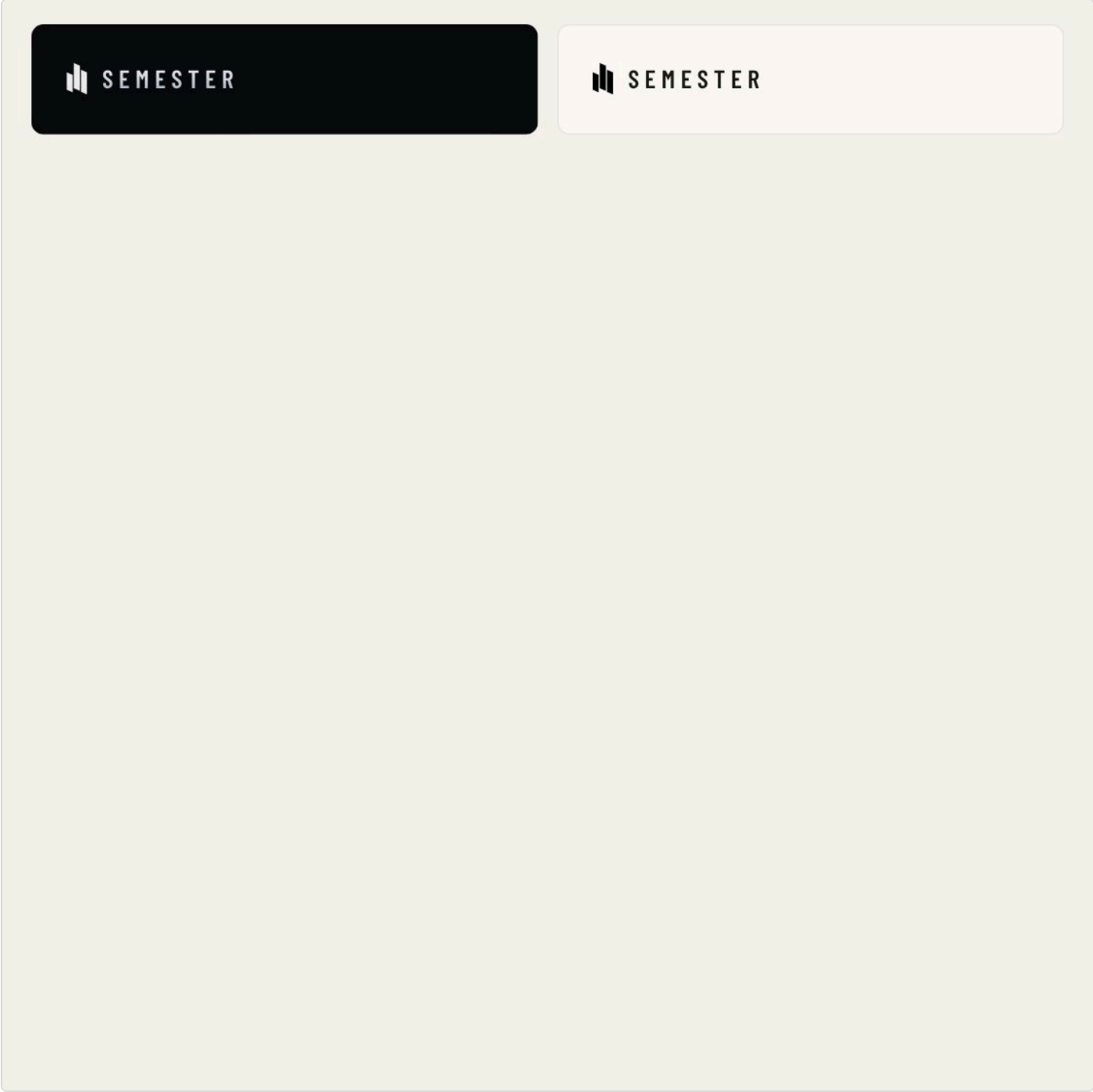
Specimen: brand-mark





Lit silver on Ink for app icons and splash; flat currentColor everywhere in-app so it follows the ground. Never recolour into the accent, add glow, or place on photography.

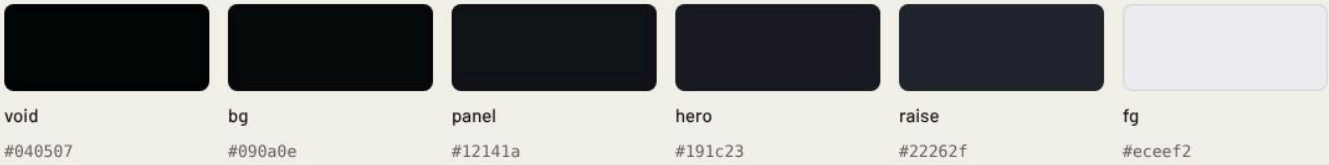
Specimen: brand-wordmark



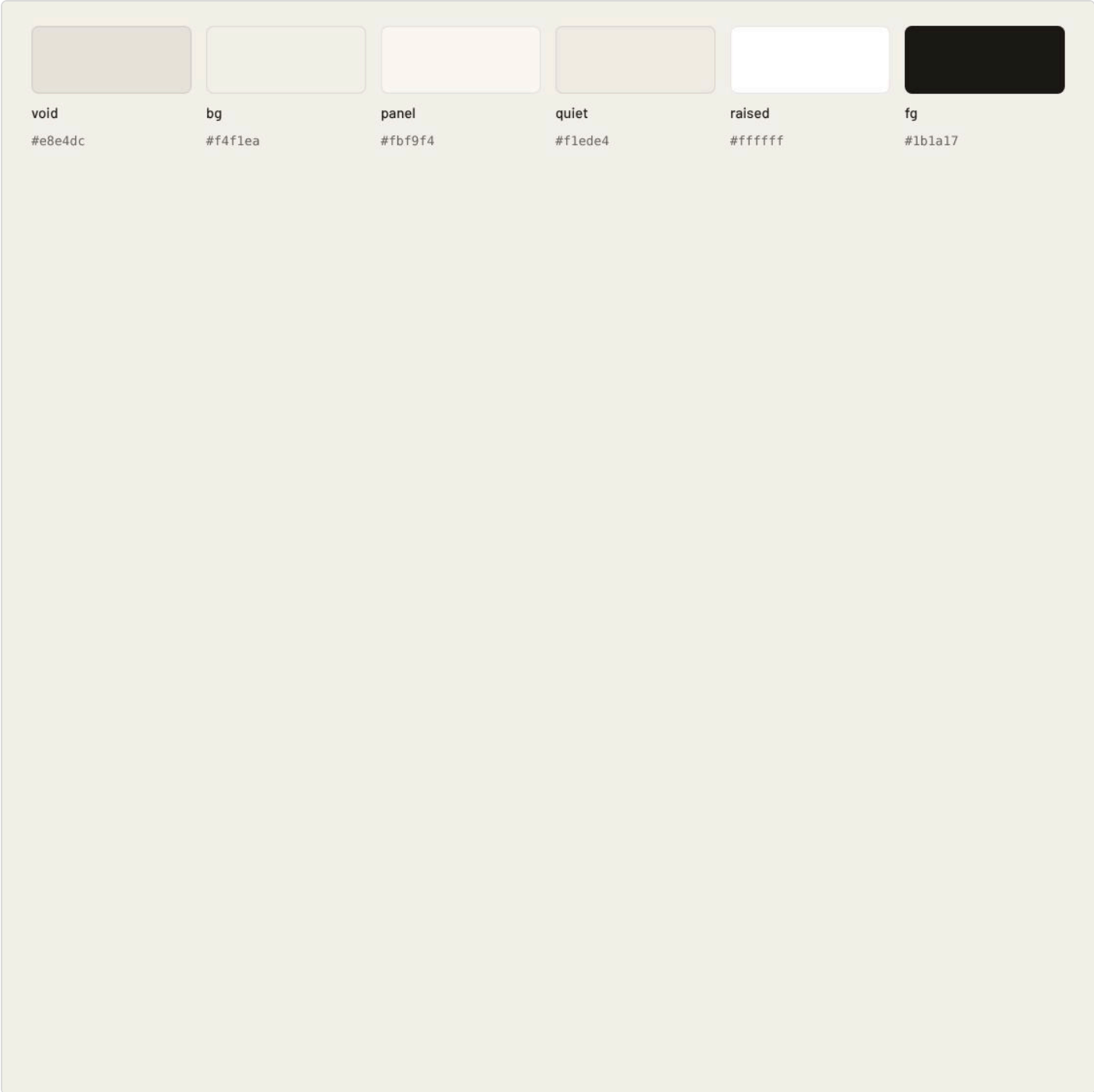
Specimen: colors-blue



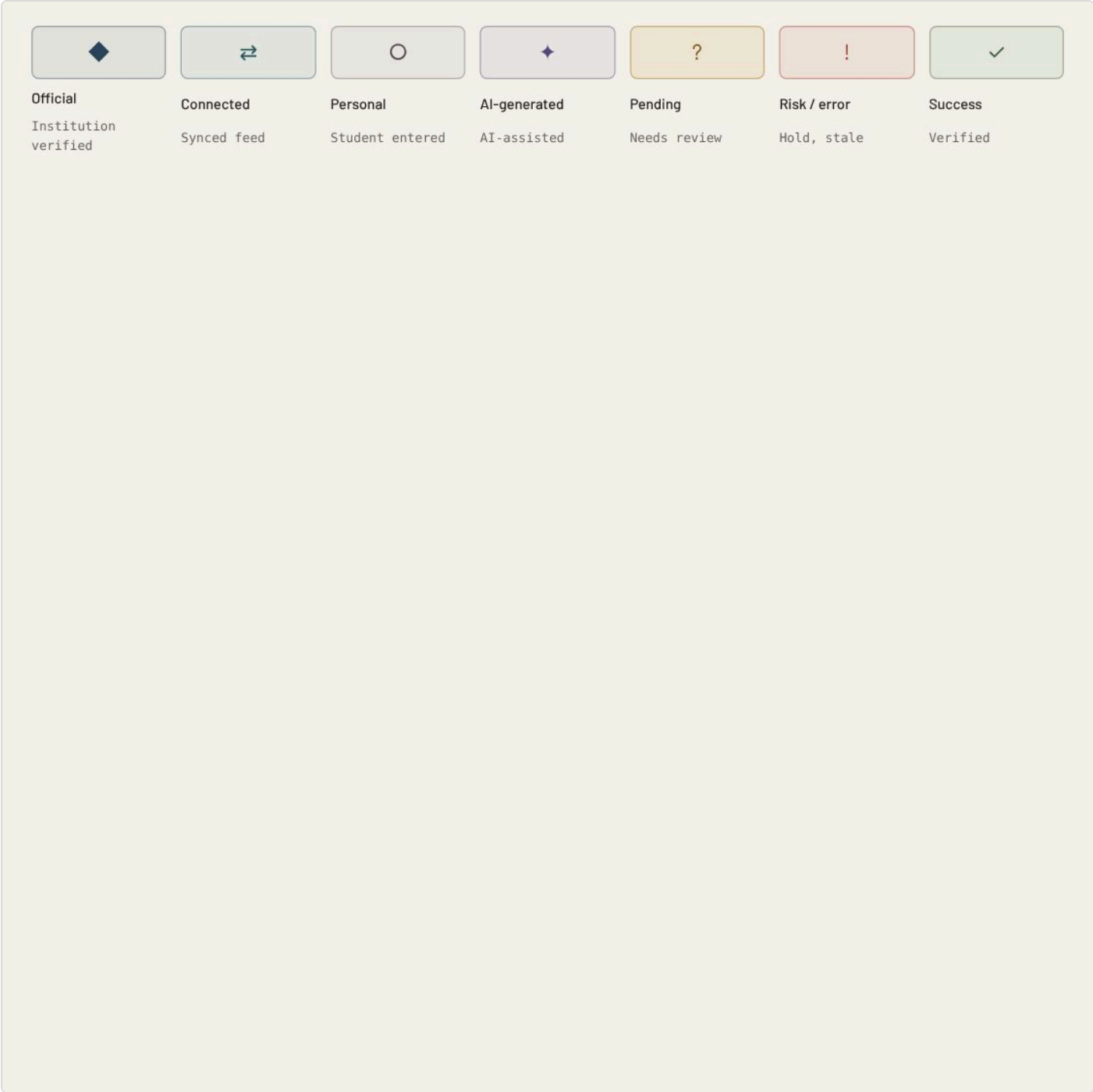
Specimen: colors-ink



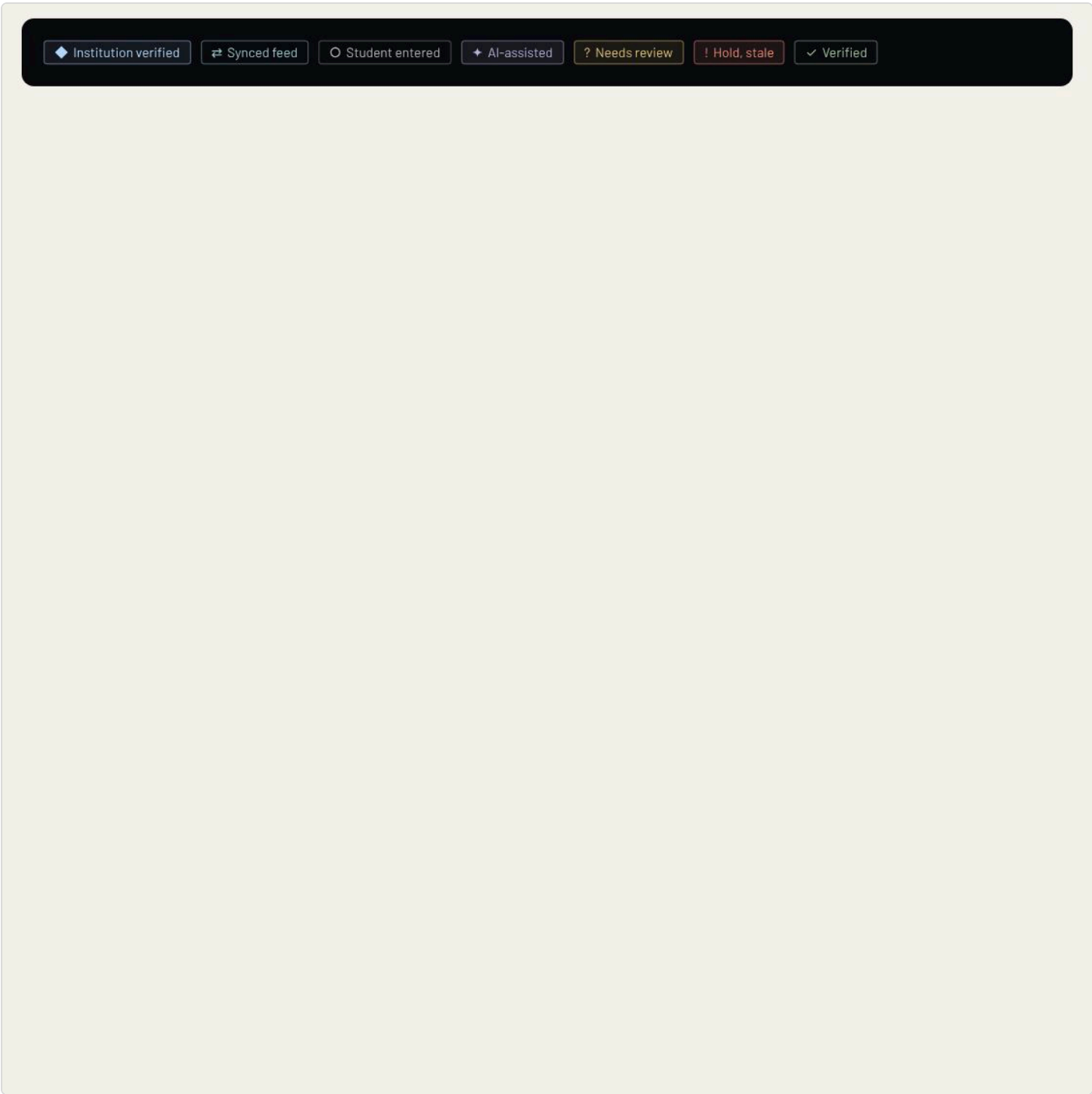
Specimen: colors-parchment



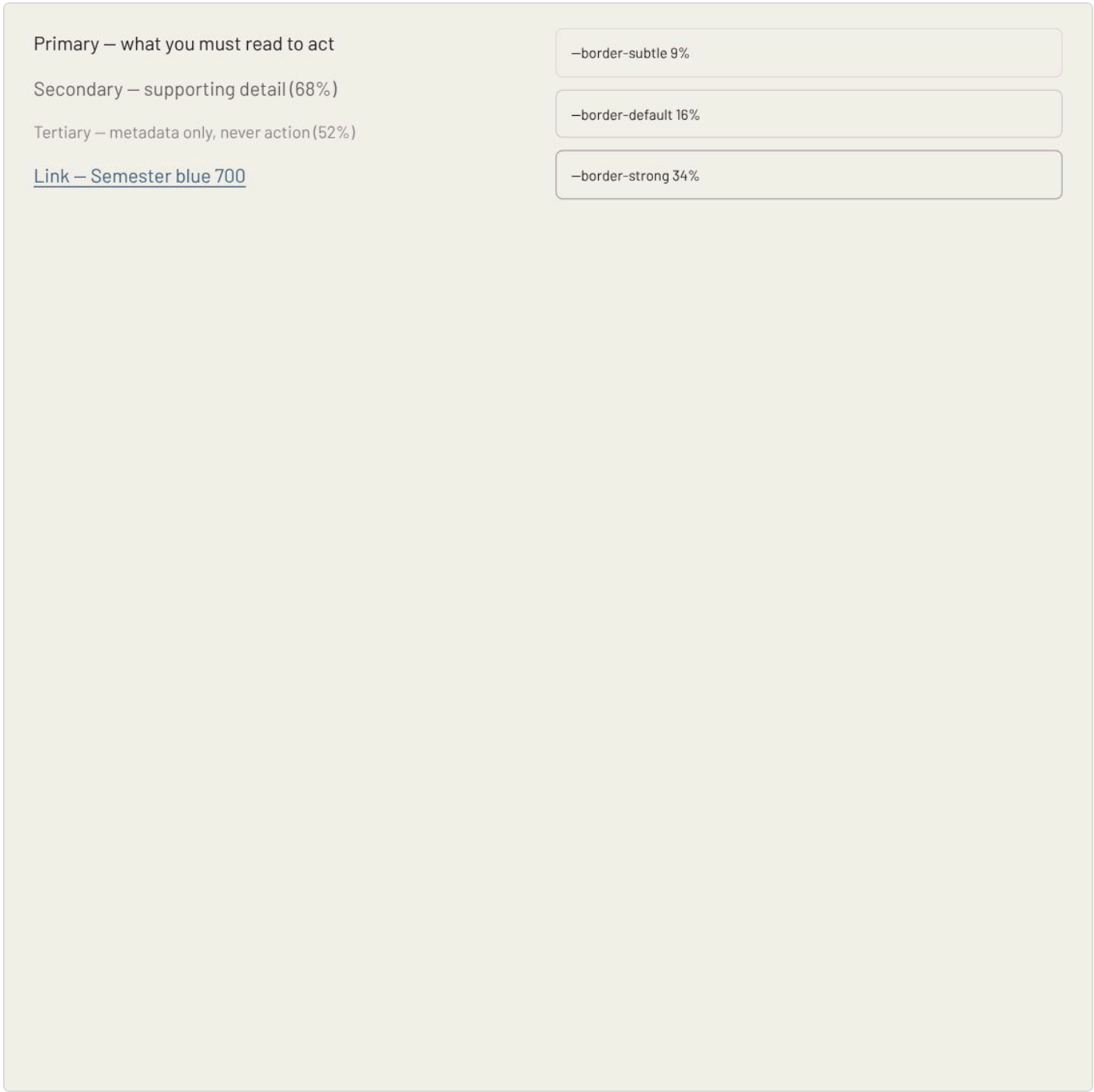
Specimen: colors-source



Specimen: colors-source-dark



Specimen: colors-text



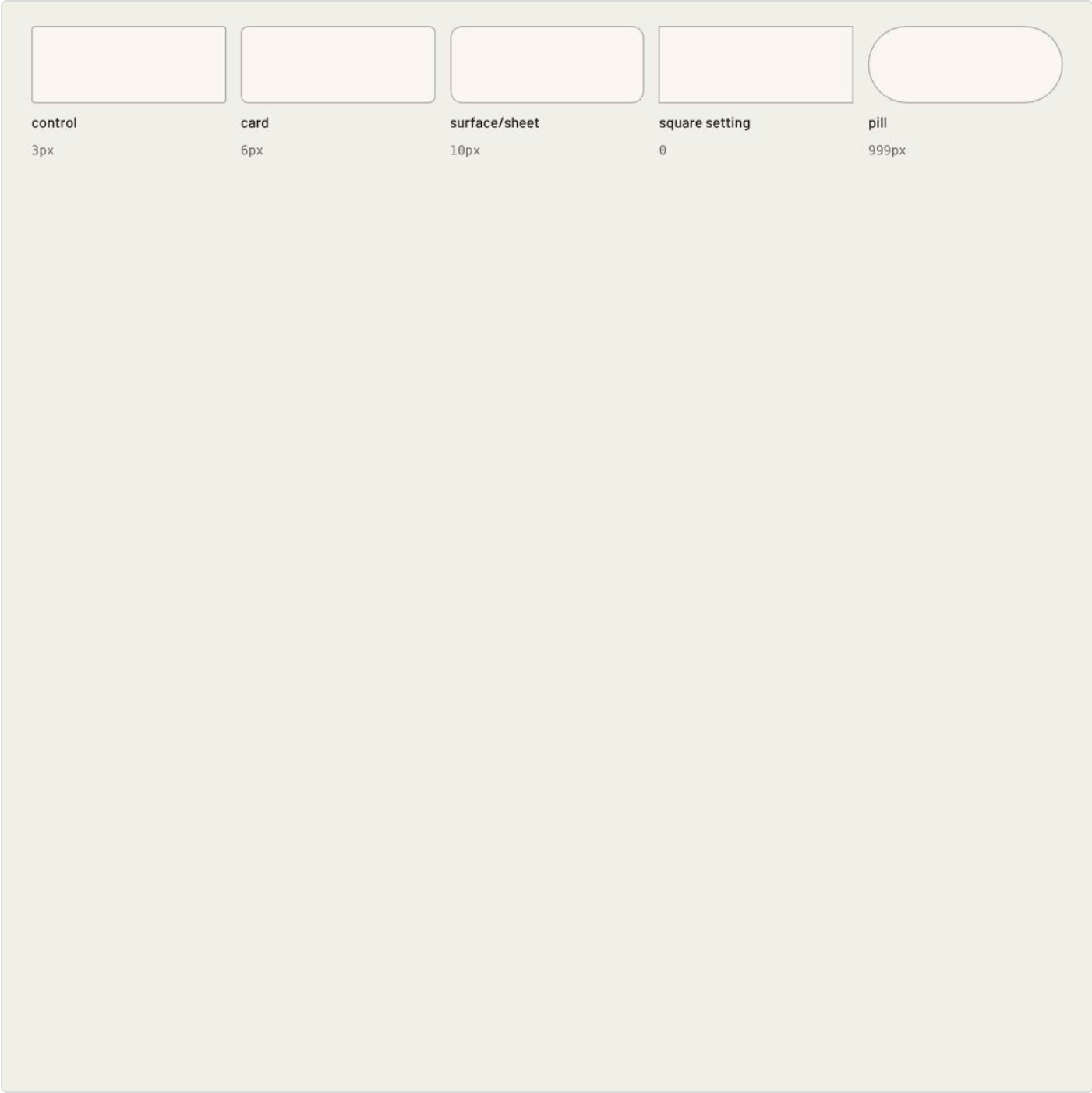
Specimen: elevation



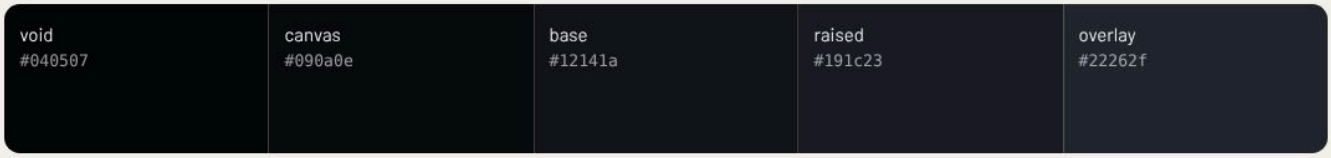
Specimen: iconography



Specimen: radii



Specimen: tonal-elevation



APPENDIX

Source research

The research reports and planning documents the founder gathered while building Semester, reproduced in full.

Semester — domain context map

Twelve modules in one deployable plus three. Border shows where each module's data lives today, measured 2026-10-04.

Solid blue — server-backed under RLS

Dashed orange — device state, rebuild as native

Student web
apps/student-web

Staff and faculty consoles
registrar · instructor · ops

Guardian web
consented sharing only

Mobile
Capacitor · SQLCipher store

Public and partner sites
company-site · trust room

API edge / gateway — Bearer auth · tenant from membership, never the client · PDP decide() · idempotency key · correlation id · audit + outbox in one transaction

`services/core · modular monolith · 12 modules`

identity
tenancy · roles · consent

productivity
Today · tasks · calendar

academic
registration · records

learning
gradebook · study tools

campus
maps · dining · housing

community
posts · moderation

family
guardian links · shares

finance
ledger on server, bill UI static

career
portfolio · applications

marketplace
gated off · not built

support-trust
help · holds · DSR

admin
console · rollout · migration

`platform modules · packages/platform (built, not yet adopted)`

Policy · PDP

Audit · hash chain

Outbox · events

Flags · kill switches

Files · notify · search

Consent · relationships

ai-gateway
every model call · tenant AI policy (not enforced yet)

sync-gateway
commands · idempotency · conflicts

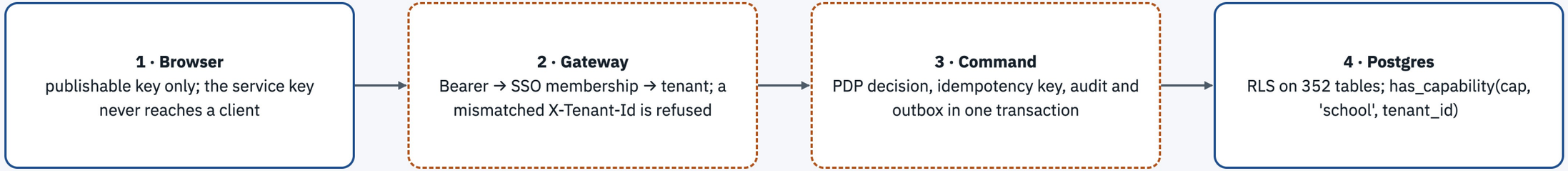
integration-hub
mappings · reconciliation · dead letters

External
SIS · LMS · IdP · LTI

Postgres 17 — 352 tables, RLS on every one · tenant_id from server · 112 policy suites · definer owners still bypass RLS (RISK-004)

Request path and tenant isolation

Where a tenant boundary is enforced on each hop, and what was proven on 2026-10-04.



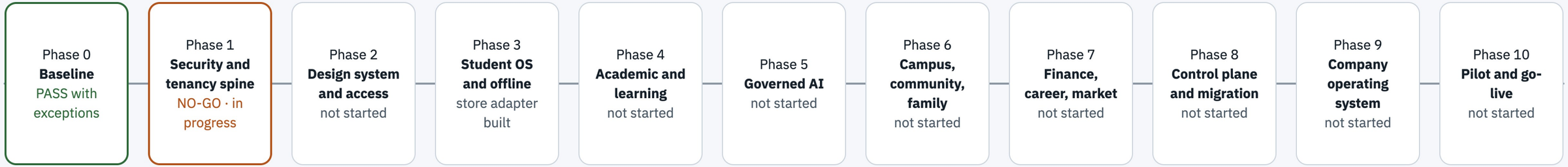
Cross-tenant probe — passing
A tenant-A admin holding every role was evaluated against every policy on 160 tenant-keyed tables. 0 admissions to tenant B. Control: admitted to A on 93 tables. A planted leaking policy is caught by name.

RISK-004 · Definers bypass RLS
postgres owns every table and the 250 client-callable definers, and has BYPASSRLS. Forcing RLS would change nothing; isolation rests on each function's own gate. Fix: non-bypass owner roles, then FORCE.

RISK-008 · AI ignores tenant policy
The claude edge function checks only the global kill switch; effective_ai_policy has no caller. A school that turns AI off is not obeyed at the call. Pulled into Phase 1 (E1.9).

Phase roadmap and gates

Eleven gated phases. A phase starts only when the one before it passes or carries dated, owned exceptions.



Phase 1, next gated work in order (about 22.5 engineer-weeks)

E1.10 record the restore drill → E1.5 revoke TRUNCATE (trial: 109 of 112 suites, only the ratchet moves) → E1.9 tenant AI policy at the call → E1.8 SAST and dependency review → E1.2 definer forged-argument sweep → E1.7 apply staging → E1.1 one tenancy source → E1.3 gateway on the platform spine → E1.4 mount the productivity API → E1.6 non-bypass definer owners, then FORCE RLS

Build a read-model architecture dashboard for your backend — command query separation, projection pipelines, and caching patterns

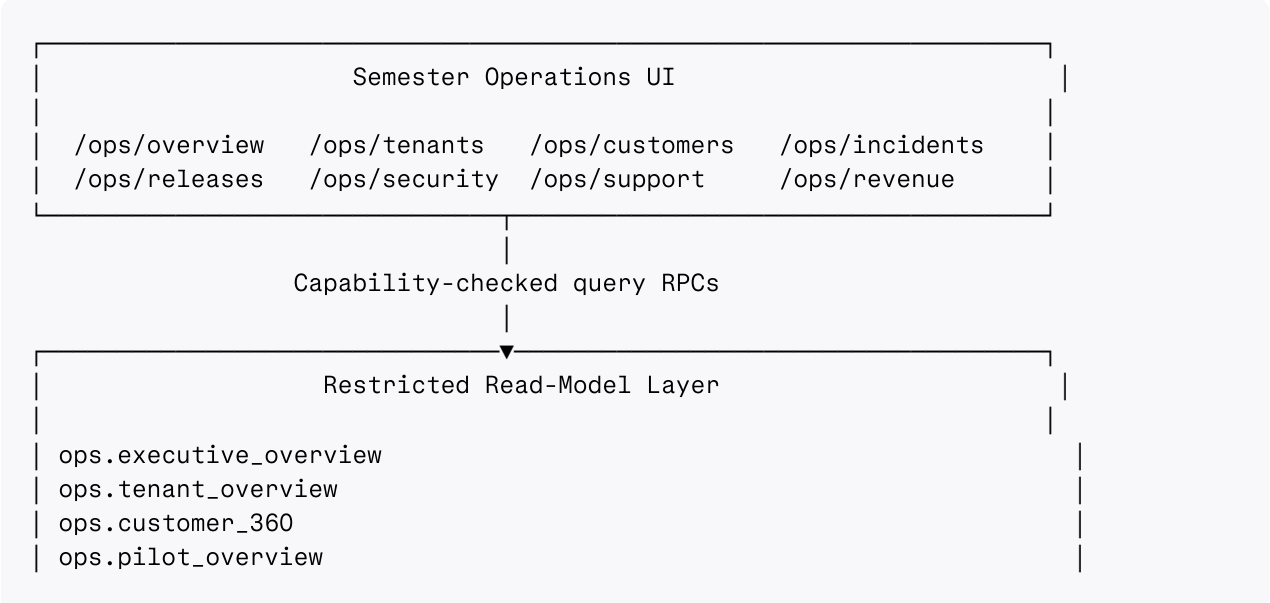
Computer
Design a coherent operating UI component library for your backend data models — state synchronization, optimistic updates, and error handling
Computer
Show me how to structure the incident and SLO command center
How should we implement the access review and security boundary
Detail the tenant health and rollout control plane modules

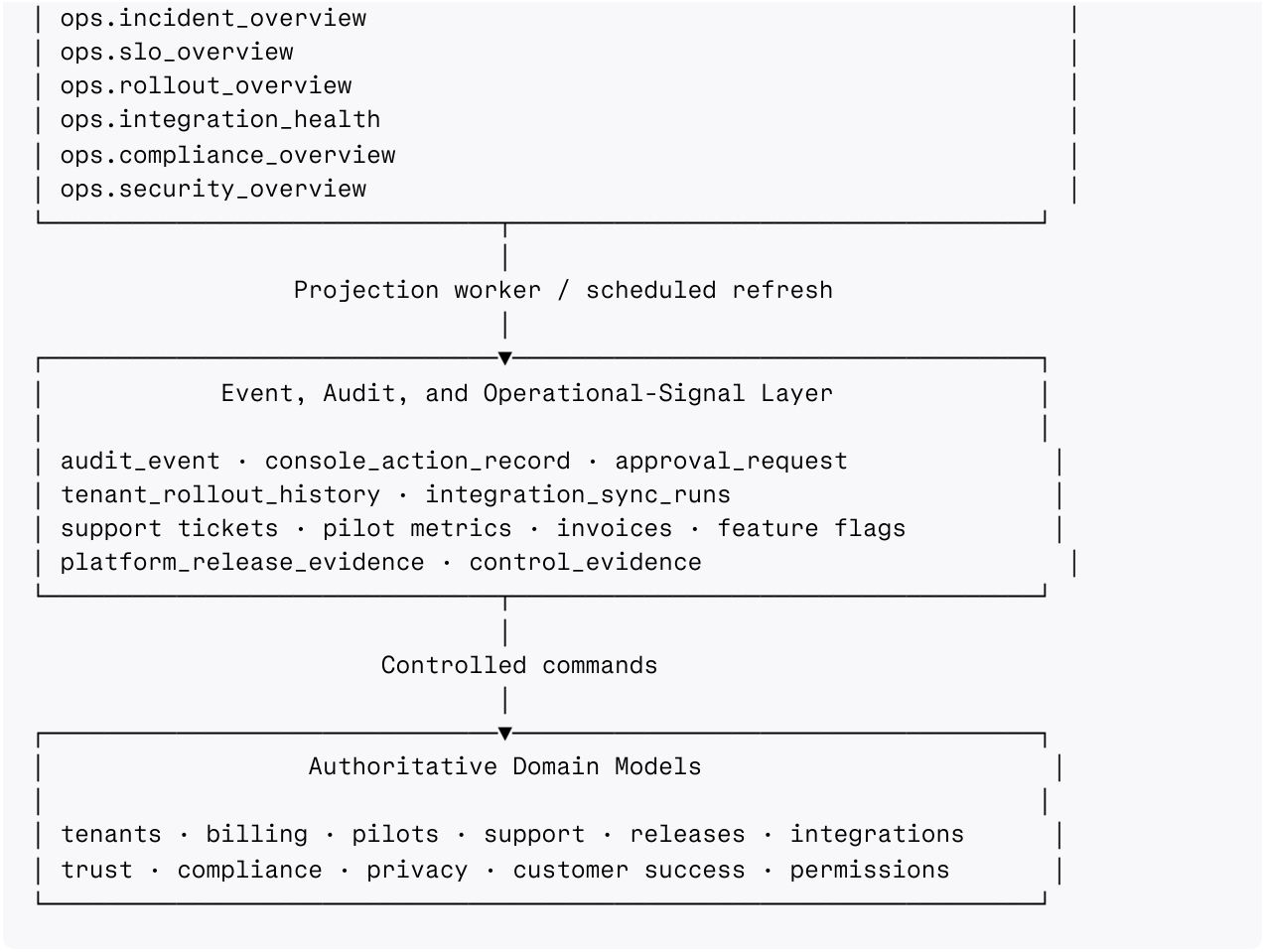
Semester should implement a **bounded CQRS-style operations layer**: keep existing domain tables and controlled RPCs as the command/source-of-truth side, then build restricted, denormalized, freshness-labeled projections specifically for console screens. Do not turn the entire product into event sourcing or duplicate every table—the goal is fast, safe operational views and predictable actions over the backend you already have.

CQRS is valuable when the query shape differs substantially from the transactional write model: instead of expensive symmetric reads/writes, maintain read-only views designed for specific query needs and update them from domain events or reliable projections. [1] [2]

Read-model architecture

Target topology





Rule: writes stay in domains

Need	Correct implementation	Avoid
Change subscription	Controlled commercial/billing RPC or server action	Editing `subscriptions` in a console table
Pause integration	Existing capability-checked integration RPC	Updating connection status directly from client
Enable feature	Approval-gated tenant feature/rollout action	Generic toggle that bypasses rollout evidence
Issue refund	Finance workflow with invoice/payment reference and approval	Ad hoc “refund” button
Handle privacy request	Controlled privacy/retention workflow	Direct deletion of customer records
Investigate tenant issue	Read-model view + time-boxed break-glass if needed	Global direct access to student content
Approve risky action	`approval_request` → `approval_decision` → execution	Browser-side confirmation only

Read-model contracts

Create a dedicated `ops` schema or an equivalent set of private/public capability-checked RPCs. Do not expose raw internal tables to anonymous or generic authenticated clients.

Base envelope

Every console query should return a consistent envelope:

```
type OpsEnvelope<T> = {
  data: T;
  meta: {
    generatedAt: string;
    sourceUpdatedAt: string | null;
    freshness: "live" | "recent" | "delayed" | "stale" | "failed";
    coverage: "complete" | "partial" | "sample" | "unavailable";
    authority: "system-of-record" | "derived" | "operator-entered";
    cache: {
      hit: boolean;
      key: string;
      expiresAt: string | null;
    };
    correlationId: string;
  };
  permissions: {
    canView: boolean;
    canExport: boolean;
    allowedActions: string[];
  };
  warnings: Array<{
    code: string;
    severity: "info" | "warning" | "critical";
    message: string;
    actionUrl?: string;
  }>;
};
```

Required projections

Projection	Main question	Suggested refresh
`ops.executive_overview`	Is the company/product healthy this week?	5–15 minutes
`ops.tenant_overview`	Is this tenant ready, healthy, safe, and commercially current?	1–5 minutes
`ops.customer_360`	What is the account's commercial, implementation, adoption, and renewal state?	5–15 minutes
`ops.pilot_overview`	Is a pilot on track to convert?	5–15 minutes
`ops.support_overview`	What customer-impacting support work needs attention?	Near-real-time
`ops.release_overview`	Is release exposure safe and within SLO/error budget?	1–5 minutes

Projection	Main question	Suggested refresh
`ops.incident_overview`	What incidents are open, escalating, or awaiting updates?	Near-real-time
`ops.slo_overview`	Are services consuming error budget too fast?	1–5 minutes
`ops.integration_overview`	Which integrations are stale, failing, drifting, or blocked?	1–5 minutes
`ops.compliance_overview`	Which controls/evidence/artifacts are missing or expiring?	Daily + on change
`ops.security_overview`	Are high-risk security controls, access reviews, secrets, and alerts current?	Daily + event-driven
`ops.privacy_overview`	Which privacy/retention/export/erasure requests need action?	Hourly
`ops.approval_inbox`	Which controlled actions need review or execution?	Near-real-time

Projection pipeline patterns

Pattern A: live query RPC

Use when data is small, bounded, capability-checked, and must be current.

```
Console screen
  → ops_tenant_overview(tenant_id)
  → checks capability + tenant scope
  → joins only needed domain tables
  → returns redacted/aggregated response
  → emits sensitive-read audit event if required
```

Use for:

- One tenant detail view
- One customer detail view
- One approval request
- One support ticket
- A break-glass session
- One rollout state

Pattern B: materialized projection

Use when a screen needs cross-domain aggregates or expensive joins.

```
Domain event or scheduled job
  → projection worker
  → compute aggregate
  → write ops.tenant_health_snapshot
```

```
  → update source watermark
  → console reads snapshot
```

Use for:

- Customer health
- Executive dashboard
- Pilot scorecards
- Revenue forecast
- Compliance completion
- Tenant rollout readiness
- SLO/error-budget trends

Pattern C: event-driven projection

Use when there is a meaningful state transition:

```
tenant_rollout changed
  → write immutable rollout history
  → enqueue projection task
  → recompute tenant readiness
  → recompute executive summary
  → create inbox item if gate failed
```

For correctness, use an outbox pattern rather than assuming a client call will update every downstream projection.

```
Transactional command
  ├── Updates domain record
  ├── Writes audit event
  └── Writes outbox event in same transaction

Worker
  ├── Claims outbox event
  ├── Updates projection(s)
  ├── Marks event processed
  └── Retries / dead-letters failures
```

Projection tables

```
create schema if not exists ops;

create table ops.projection_watermarks (
  projection_name text primary key,
  source_event_cursor bigint,
  source_updated_at timestamptz,
  projected_at timestamptz not null default now(),
```

```
  status text not null check (
    status in ('healthy', 'delayed', 'failed', 'rebuilding')
  ),
  last_error text
);

create table ops.tenant_health_snapshots (
  tenant_id uuid primary key,
  health_score integer check (health_score between 0 and 100),
  health_band text check (
    health_band in ('healthy', 'watch', 'at_risk', 'critical', 'unknown')
  ),
  activation_score integer,
  adoption_score integer,
  support_score integer,
  commercial_score integer,
  implementation_score integer,
  integration_score integer,
  security_score integer,
  renewal_score integer,
  reasons jsonb not null default '[]'::jsonb,
  recommended_actions jsonb not null default '[]'::jsonb,
  source_updated_at timestamptz,
  computed_at timestamptz not null default now()
);

create table ops.operations_inbox_items (
  id uuid primary key default gen_random_uuid(),
  item_type text not null,
  severity text not null check (severity in ('p0', 'p1', 'p2', 'p3')),
  status text not null check (
    status in ('open', 'acknowledged', 'blocked', 'resolved', 'dismissed')
  ),
  tenant_id uuid,
  customer_id uuid,
  source_type text not null,
  source_id uuid,
  title text not null,
  summary text,
  owner_id uuid,
  due_at timestamptz,
  evidence jsonb not null default '{}'::jsonb,
  recommended_action jsonb not null default '{}'::jsonb,
  correlation_id text,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);
```

Do not apply this blindly to production. First reconcile it with existing `tenant_rollout`, `account_health_snapshots`, approval, support, integration, and governance models, because the current backend may already contain equivalent concepts.[3] [4]

Caching policy

Use caching to reduce expensive reads, not to hide stale operational risk.

Data class	Cache strategy	TTL	Invalidation
Approval inbox	Minimal or none	15–60 sec	Approval request/decision event
Open incident/SLO state	Minimal	15–60 sec	Incident, alert, or SLO event
Tenant overview	Short-lived	1–5 min	Tenant/config/feature/rollout change
Executive metrics	Projection cache	5–15 min	Source event + scheduled refresh
Finance forecast	Versioned snapshot	15–60 min	Forecast input/import change
Compliance/trust	Projection cache	1–24 hr	Evidence/artifact/status change
Static capability metadata	Longer cache	1–24 hr	Role/capability migration/change
Historical reports	Immutable cache	Long	Never; version by report/run ID

Cache keys

```
ops:v1:executive:{timeRange}:{operatorScope}
ops:v1:tenant:{tenantId}:{operatorScope}
ops:v1:customer:{customerId}:{operatorScope}
ops:v1:inbox:{role}:{ownerId}:{filtersHash}
ops:v1:slo:{service}:{window}
ops:v1:compliance:{tenantId}:{framework}:{period}
```

Never put raw student PII, session tokens, secrets, unredacted support content, or credential material into shared cache keys or browser storage.

UI component library

Build a domain-aware internal component library—not merely generic cards and tables.

```
app/src/ops/
├── components/
│   ├── OpsShell.tsx
│   ├── OpsContextBar.tsx
│   ├── OpsCommandPalette.tsx
│   ├── OpsMetricCard.tsx
│   ├── OpsFreshnessBadge.tsx
│   ├── OpsHealthBadge.tsx
│   ├── OpsEvidenceDrawer.tsx
│   ├── OpsApprovalBanner.tsx
│   ├── OpsRiskBanner.tsx
│   ├── OpsTimeline.tsx
│   ├── OpsAuditTrail.tsx
│   ├── OpsEmptyState.tsx
│   ├── OpsForbiddenState.tsx
│   ├── OpsFailureState.tsx
│   ├── OpsMutationPreview.tsx
```

```
│   ├── OpsActionDialog.tsx
│   ├── OpsAssignmentControl.tsx
│   ├── OpsDataTable.tsx
│   └── OpsExportButton.tsx
├── data/
│   ├── queryKeys.ts
│   ├── opsClient.ts
│   ├── opsTypes.ts
│   ├── opsMappers.ts
│   └── freshness.ts
├── state/
│   ├── opsContextStore.ts
│   ├── optimisticMutationQueue.ts
│   └── userPreferences.ts
└── screens/
    ├── OverviewScreen.tsx
    ├── TenantScreen.tsx
    ├── IncidentScreen.tsx
    ├── SloScreen.tsx
    ├── AccessReviewScreen.tsx
    └── RolloutScreen.tsx
```

Essential components

Component	Required behavior
`OpsContextBar`	Global/tenant/customer/time-range scope; shows data freshness
`OpsMetricCard`	Value, trend, threshold, freshness, definition, evidence link
`OpsEvidenceDrawer`	Source records, query definition, timestamp, owner, linked decision
`OpsApprovalBanner`	Shows whether action is allowed, pending, approved, expired, or denied
`OpsMutationPreview`	Exact impact, rollback, dependencies, approvals, audit events
`OpsTimeline`	Cross-domain sequence of rollout, approval, support, incident, integration events
`OpsDataTable`	Server-side pagination/filter/sort; permission-aware columns; export scope warning
`OpsFailureState`	Retry, incident link, last known data, stale-data warning
`OpsForbiddenState`	Explains lack of capability without revealing protected data
`OpsActionDialog`	Confirmation, reason, ticket/evidence attachment, approval path

State synchronization

Query ownership

Use one query key per data contract:

```
export const opsQueryKeys = {
  executive: (range: string) => ["ops", "executive", range] as const,
  inbox: (filters: string) => ["ops", "inbox", filters] as const,
```

```
  tenant: (tenantId: string) => ["ops", "tenant", tenantId] as const,
  customer: (customerId: string) => ["ops", "customer", customerId] as const,
  pilot: (pilotId: string) => ["ops", "pilot", pilotId] as const,
  incident: (incidentId: string) => ["ops", "incident", incidentId] as const,
  slo: (service: string, window: string) => ["ops", "slo", service, window] as const
  rollout: (tenantId: string) => ["ops", "rollout", tenantId] as const,
};
```

Optimistic updates

Only use optimistic updates for low-risk, reversible actions:

Action	Optimistic?	Reason
Acknowledge inbox item	Yes	Reversible, no domain-side effect
Assign internal work item	Yes	Reversible, conflict-manageable
Save personal view preference	Yes	User-scoped and low risk
Add internal note	Yes, with pending marker	Can retry safely
Update draft filter state	Yes	Client-only
Approve high-risk action	No	Must await authoritative backend result
Enable tenant feature	No	Approval/rollout gate required
Change plan/entitlement	No	Commercial impact
Issue refund/credit	No	Financial impact
Start break-glass access	No	Must await server grant
Delete/export data	No	Privacy/legal impact
Pause integration	Usually no	Must show current state and outcome
Kill switch	No	Critical system impact

Optimistic mutation pattern

```
const mutation = useMutation({
  mutationFn: acknowledgeInboxItem,
  onMutate: async ({ id }) => {
    await queryClient.cancelQueries({ queryKey: ["ops", "inbox"] });

    const previous = queryClient.getQueryData<InboxItem[]>(["ops", "inbox"]);

    queryClient.setQueryData<InboxItem[]>(["ops", "inbox"], (items = []) =>
      items.map((item) =>
        item.id === id
          ? { ...item, status: "acknowledged", pendingSync: true }
          : item
      )
    );
```

```
    return { previous };
  },
  onError: (_error, _variables, context) => {
    queryClient.setQueryData(["ops", "inbox"], context?.previous);
  },
  onSuccess: (serverItem) => {
    queryClient.setQueryData<InboxItem[]>(["ops", "inbox"], (items = []) =>
      items.map((item) => (item.id === serverItem.id ? serverItem : item))
    );
  },
  onSettled: () => {
    queryClient.invalidateQueries({ queryKey: ["ops", "inbox"] });
  },
});
```

Do not mirror the same mutation into unrelated dashboard cards manually. Invalidate or patch only the authoritative read-model contracts.

Error handling

Error classes

```
type OpsErrorCode =
  | "UNAUTHENTICATED"
  | "FORBIDDEN"
  | "TENANT_SCOPE_DENIED"
  | "APPROVAL_REQUIRED"
  | "APPROVAL_EXPIRED"
  | "CONFLICT"
  | "STALE_VERSION"
  | "VALIDATION_FAILED"
  | "RATE_LIMITED"
  | "DEPENDENCY_UNAVAILABLE"
  | "PROJECTION_STALE"
  | "INTEGRATION_PAUSED"
  | "INCIDENT_ACTIVE"
  | "UNKNOWN";
```

UI behavior

Error	UI behavior
Unauthenticated	Route to sign-in; preserve safe return path
Forbidden	Show capability explanation; no sensitive preview
Approval required	Offer “request approval” with reason/evidence form
Stale version/conflict	Show server state, user’s attempted change, merge/retry choice
Projection stale	Show last known value, stale timestamp, refresh state
Dependency unavailable	Show affected source, fallback data, incident link

Error	UI behavior
Integration paused	Explain why, scope, owner, and approved next action
Active incident	Warn before non-essential change; show release policy
Validation failed	Field-level errors and retained form values
Unknown	Correlation ID, retry guidance, support/incident escalation path

Incident command center

Incident modules

```
/ops/incidents
├── Active incidents
├── Incident detail
├── Service health
├── Alert timeline
├── Customer impact
├── Communications
├── Mitigation actions
├── Evidence/log links
├── Postmortems
└── Follow-up actions
```

Incident record

```
type Incident = {
  id: string;
  status: "declared" | "investigating" | "mitigating" | "monitoring" | "resolved";
  severity: "SEV0" | "SEV1" | "SEV2" | "SEV3";
  title: string;
  startedAt: string;
  detectedAt: string;
  commanderId: string;
  technicalLeadId?: string;
  communicationsLeadId?: string;
  impactedServices: string[];
  affectedTenants: number;
  affectedUsers: number | null;
  customerImpactSummary: string;
  currentHypothesis?: string;
  mitigationPlan?: string;
  nextUpdateAt: string;
  linkedChangeIds: string[];
  linkedSloIds: string[];
  linkedRunbooks: string[];
  linkedAuditEvents: string[];
  correlationId: string;
};
```

Incident workflow

```
Signal/alert
  ↓
Triage
  ↓
Declare severity
  ↓
Assign commander, technical lead, communications lead
  ↓
Freeze or restrict risky releases as required
  ↓
Assess tenant/customer impact
  ↓
Mitigate
  ↓
Update internal stakeholders and affected customers
  ↓
Monitor recovery
  ↓
Resolve
  ↓
Postmortem
  ↓
Corrective actions tracked in operations inbox
```

Severity rules

Severity	Example	Command-center behavior
SEV0	Confirmed major security incident, broad data exposure, complete critical outage	Immediate command mode, executive/legal/security escalation, release freeze
SEV1	Major tenant or workflow outage, serious integrity/reliability issue	Incident commander, customer updates, high-priority mitigation
SEV2	Material degraded feature with workaround	Owner, timeline, monitored impact
SEV3	Minor defect or localized issue	Normal triage and backlog path

Incident dashboard cards

- Active incidents by severity
- Customers/tenants affected
- Time to acknowledge
- Time to mitigate
- Time to resolve
- Next update deadlines
- Error-budget burn rate
- Linked releases

- Open corrective actions
- Repeated incident patterns
- Customer communication status

SLO command center

Define services, then objectives

Do not start with arbitrary uptime percentages. First define critical user journeys:

Service/journey	Example SLI	Example initial SLO
Authentication	Successful authentication requests / valid auth attempts	99.9% monthly
Core student workspace read	Successful read responses within latency threshold	99.9% monthly
Critical write command	Successful accepted commands / valid command attempts	99.9% monthly
Tenant rollout control	Successful rollout/config reads and approved writes	99.9% monthly
Integration sync	Successful scheduled syncs completed within freshness target	99.0% weekly
Support notifications	Accepted notification jobs delivered within target window	99.5% monthly
Billing webhook processing	Verified event processing within target	99.9% monthly
Admin command center	Successful authorized console queries	99.5% monthly

An SLO is a target value/range for a measured service indicator; error budgets are the permissible miss rate and help decide whether release velocity should slow when reliability degrades.[5] [6]

SLO schema

```
create table ops.service_catalog (
  id uuid primary key default gen_random_uuid(),
  service_key text unique not null,
  name text not null,
  owner_role text not null,
  criticality text not null check (criticality in ('tier_0', 'tier_1', 'tier_2', 'ti
  description text not null,
  runbook_url text,
  created_at timestamptz not null default now()
);

create table ops.slo_definitions (
  id uuid primary key default gen_random_uuid(),
  service_id uuid not null references ops.service_catalog(id),
  name text not null,
  sli_query_version text not null,
  objective numeric(5,4) not null check (objective > 0 and objective < 1),
  window_days integer not null,
  alert_policy jsonb not null,
  release_policy jsonb not null,
```

```
  effective_at timestamptz not null,
  retired_at timestamptz
);

create table ops.slo_snapshots (
  slo_id uuid not null references ops.slo_definitions(id),
  captured_at timestamptz not null,
  eligible_events bigint not null,
  bad_events bigint not null,
  compliance_ratio numeric(8,6) not null,
  error_budget_remaining numeric(8,6) not null,
  burn_rate_1h numeric(10,4),
  burn_rate_6h numeric(10,4),
  burn_rate_24h numeric(10,4),
  source_status text not null,
  primary key (slo_id, captured_at)
);
```

Error-budget policy

Remaining budget	Policy
75–100%	Normal release velocity
50–75%	Release owner reviews risk; prioritize reliability work
25–50%	Restrict high-risk changes; require additional approval
0–25%	Freeze non-essential changes for affected service
Exhausted	Incident/reliability review; corrective plan before resuming normal releases

Google's SRE error-budget approach is designed to protect users from repeated reliability failures while giving teams a concrete mechanism for balancing reliability and feature delivery. [7]

Access review and security boundary

Access review workflow

Run these on defined schedules:

Review	Cadence	Scope
Privileged console access	Monthly	`app_admins`, operator roles, high-risk capabilities
Tenant admin access	Quarterly or customer-defined	Institution membership and school-scoped grants
Break-glass access	Per event + monthly review	Every grant, use, expiry, reviewer sign-off
Service accounts	Monthly	Purpose, owner, scope, rotation, last use
Integration scopes	Quarterly and on change	External read/write scopes and approvals
Finance permissions	Monthly	Quote, plan, refund, credit, dunning, invoice actions

Review	Cadence	Scope
Support access	Weekly	Time-bound support grants and aggregate access
Vendor access	Quarterly	Contractor/vendor account access and offboarding

Access review record

```
type AccessReview = {
  id: string;
  reviewType:
    | "privileged_console"
    | "tenant_admin"
    | "service_account"
    | "integration_scope"
    | "finance"
    | "support"
    | "break_glass";
  scopeType: "global" | "tenant" | "customer" | "service";
  scopeId?: string;
  periodStart: string;
  periodEnd: string;
  reviewerId: string;
  status: "draft" | "in_review" | "completed" | "overdue";
  findings: Array<{
    principalId: string;
    access: string;
    decision: "retain" | "modify" | "revoke";
    reason: string;
    actionEvidence?: string;
  }>;
  dueAt: string;
  completedAt?: string;
};
```

Security boundary rules

```
Browser:
- Uses only publishable keys.
- Never receives service-role credentials.
- Reads only capability-checked query endpoints.
- Sends commands only to typed, validated, auditable server/RPC endpoints.

Server/RPC:
- Validates identity with authoritative session/user context.
- Checks capability and tenant/customer scope.
- Validates schema and business invariants.
- Enforces approval and duty requirements.
- Writes audit event and correlation ID.
- Queues side effects through outbox/worker where needed.

Database:
- Uses RLS as a data-layer backstop.
```

```
- Keeps private/internal data in non-exposed schemas.
- Exposes narrow RPC/read-model contracts, not broad raw tables.
- Uses immutable audit/evidence records for high-risk actions.

Workers:
- Own provider credentials and integration secrets.
- Operate from queue/outbox jobs.
- Apply idempotency keys.
- Record sanitized errors and retries.
- Never expose raw upstream payloads to general operators.
```

NIST CSF 2.0 explicitly frames cybersecurity as an organization-wide risk-management concern, including governance, roles, policies, and oversight—not only technical controls. [8] [9]

Tenant health and rollout control plane

Tenant health dimensions

```
Tenant Health Score
├── Commercial
│   ├── Contract current
│   ├── Billing current
│   ├── Renewal date risk
│   └── Plan/entitlement alignment
├── Implementation
│   ├── Milestone completion
│   ├── Admin training
│   ├── Integration readiness
│   └── Launch blockers
├── Adoption
│   ├── Invited users
│   ├── Activation
│   ├── First meaningful action
│   ├── Workflow completion
│   └── Weekly active use
├── Support
│   ├── P0/P1 ticket count
│   ├── SLA attainment
│   ├── Sentiment
│   └── Repeat issues
├── Reliability
│   ├── Tenant-specific errors
│   ├── Integration freshness
│   ├── Feature/release incidents
│   └── SLO impact
├── Security/compliance
│   ├── SSO state
│   ├── Access-review status
│   ├── Trust evidence status
│   └── Open security/privacy issues
└── Relationship
    ├── Champion engagement
    ├── Executive sponsor attendance
```

```
    ├── QBR status
    └── Expansion/referral signal
```

Health score approach

Do not use a universal opaque score. Store components and explain them:

```
type TenantHealth = {
  overallBand: "healthy" | "watch" | "at_risk" | "critical" | "unknown";
  overallScore?: number;
  components: {
    commercial: number | null;
    implementation: number | null;
    adoption: number | null;
    support: number | null;
    reliability: number | null;
    security: number | null;
    relationship: number | null;
  };
  topDrivers: Array<{
    direction: "positive" | "negative";
    severity: "info" | "warning" | "critical";
    message: string;
    source: string;
    observedAt: string;
  }>;
  recommendedActions: Array<{
    action: string;
    ownerRole: string;
    dueAt?: string;
    approvalRequired: boolean;
  }>;
  computedAt: string;
  sourceFreshness: string;
};
```

Rollout control plane

Your rollout UI should enforce a state machine:

```
Draft
→ Pilot designed
→ Contracting
→ Security/procurement review
→ Implementation
→ Launch readiness
→ Controlled launch
→ Live pilot
→ Conversion review
→ Annual live
→ Expansion
→ Renewal risk
→ Offboarding
```

```
→ Archived
→ Purge eligible
```

For each transition:

```
Current state
Required evidence
Blocked by
Required capability
Required approvers
Rollback path
Customer communication
Audit event
Next review date
```

Rollout module sections

Tab	What it shows
Overview	Lifecycle state, owner, health, next gate, blockers
Gate checklist	Required evidence and approval progress
Implementation	Projects, milestones, data/integration setup, training
Feature exposure	Tenant feature policy, cohorts, kill switches, release history
Integrations	Connection state, scopes, sync freshness, errors, drift
Adoption	Activation, first action, workflow completion, retention
Support	Open tickets, trends, SLA, systemic problems
Commercial	Plan, contract, quote, invoice, renewal, commitments
Security/trust	SSO, compliance evidence, trust-room status, review dates
Privacy	Data requests, retention/hold constraints, export/deletion status
Audit	Timeline of significant operator, tenant, and automated events

Build command

Use this as the next implementation prompt:

```
Build the Semester Operations Intelligence Layer on top of the existing
Semester backend and Operations Command Center.

Do not rebuild domains or create a generic admin database browser.

Implement a bounded CQRS-style architecture:

1. Keep existing domain tables, controlled RPCs, approval workflows, audit
   records, rollout state, GTM, billing, support, trust, compliance, privacy,
   integration, and tenant models as authoritative command-side sources.
```

2. Create restricted operational read models for console screens:
   - executive overview
   - operations inbox
   - tenant overview
   - customer 360
   - pilot overview
   - support overview
   - release overview
   - incident overview
   - SLO overview
   - integration overview
   - trust/compliance overview
   - privacy overview
   - security overview
   - audit search

3. Every query contract must:
   - check capabilities server-side
   - enforce tenant/customer scope
   - return only minimum required fields
   - redact sensitive data by default
   - include freshness, source, coverage, authority, and correlation metadata
   - support pagination and filters
   - emit audit evidence for sensitive reads where appropriate

4. Build:
   - Unified Operations Inbox
   - Cross-domain work orchestration linked to existing source records
   - Evidence drawers
   - Action previews
   - Decision Center
   - Release and error-budget control
   - Incident Command Center
   - Tenant Health and Rollout Control Plane
   - Access Review workspace
   - Risk Register
   - Weekly/monthly operating-review report generator

5. Use optimistic updates only for low-risk reversible actions:
   - acknowledge inbox item
   - assign work item
   - save personal view
   - add internal note
   All high-risk actions must wait for authoritative server responses.

6. For incident/SLO work:
   - define services, SLIs, SLOs, error budgets, alert policies, and release policies
   - pause or restrict non-essential releases when error budgets are low or exhausted
   - link incidents to releases, tenants, affected customers, audit events, runbooks, and corrective actions

7. For access review:
   - build monthly privileged-console reviews

```
   - time-boxed break-glass review
   - quarterly tenant-admin and integration-scope review
   - service-account review
   - finance permission review
   - require retain/modify/revoke decisions with evidence

8. For tenant health:
   - make scores explainable
   - show commercial, implementation, adoption, support, reliability,
     security, and relationship components
   - show freshness and source metadata
   - generate assigned recommended actions

9. For rollout:
   - enforce explicit lifecycle transitions
   - require gate evidence
   - show blockers and approval requirements
   - preserve rollback paths and audit events
   - never permit client-only state changes to change tenant rollout status

10. Add:
   - authorization tests
   - tenant-isolation tests
   - projection freshness tests
   - optimistic-update rollback tests
   - incident transition tests
   - error-budget policy tests
   - access-review workflow tests
   - rollout gate tests
   - export/redaction tests

Before implementation, classify all current Supabase advisor findings:
- intentionally private/internal
- intentionally exposed and RLS-protected
- needs privilege narrowing
- needs dedicated RPC/read model
- needs test
- needs remediation

Do not blindly revoke permissions or add policies. Document every exposure,
its intended client, its RLS expectation, its function-grant expectation,
and its remediation owner.
```

Priority order

1. **Exposure classification and access boundary**
2. **Read-model contracts**
3. **Operations inbox**
4. **Tenant lifecycle and health**
5. **Incident/SLO command center**
6. **Access review**

7. **Release and rollout controls**
8. **Customer/pilot/revenue intelligence**
9. **Trust, compliance, privacy, and audit workspaces**
10. **Automated operating-review reports**

This creates an internal system that is fast for operators, explainable for leadership, safe for institutions, and resistant to the common failure mode of "admin dashboards" that quietly bypass the controls the product claims to enforce.

⁂

1. https://microservices.io/patterns/data/cqrs.html
2. https://microservices.io/patterns/
3. https://info.lse.ac.uk/staff/divisions/Eden-Centre/Assets-EC/Documents/Claude-Guides-Dec-2025/Guide-Creating-Claude-Artifacts-V1.0-011225.pdf
4. https://academy.claude.com/courses/claude-101/creating-with-artifacts
5. https://sre.google/sre-book/service-level-objectives/
6. https://sre.google/workbook/implementing-slos/
7. https://sre.google/workbook/error-budget-policy/
8. https://www.nist.gov/cyberframework
9. https://csrc.nist.gov/news/2024/the-nist-csf-20-is-here
10. https://microservices.io/patterns/data/event-sourcing.html
11. https://microservices.io/tags/pattern
12. https://attractgroup.com/blog/mastering-error-budgets-a-complete-guide-to-slos-slis-and-reliability-for-service-levels/
13. https://www.reddit.com/r/sre/comments/1j0yekt/how_do_you_define_error_budgets/
14. https://www.reddit.com/r/microservices/comments/1d1tt1p/what_is_cqrs_design_pattern_in_microservices/
15. https://stackoverflow.com/questions/58436626/read-model-in-microservice-architecture
16. https://discuss.kurrent.io/t/es-cqrs-microservice-setup-read-model-generation/3294
17. https://sre.google/workbook/alerting-on-slos/
18. https://sre.google/workbook/index/
19. https://cloud.google.com/blog/products/management-tools/sre-error-budgets-and-maintenance-windows

Build an interactive CQRS read-model dashboard for projections, lag, and cache invalidation — with live simulator controls

Computer
Design a CQRS component library for React or Vue — sortable data grids, optimistic mutation hooks, and projection state sync
Computer
Show me the projection pipeline schema implementation
How do we handle cache invalidation for projections
Write the capability-checked query RPC definitions

Implement the projection layer with **transactional outbox events, idempotent projection workers, versioned read models, targeted cache invalidation, and capability-checked query RPCs.** The browser should never query broad internal tables directly; it receives only scoped read-model contracts.

CQRS works well here because Semester's operational screens need denormalized, fast, cross-domain reads that are different from the transactional domain models used for tenant, billing, rollout, support, and approval writes. A reliable approach is to atomically persist domain changes plus outbox events, then let idempotent consumers update materialized projections.[1] [2]

Projection pipeline schema

1. Schemas and boundaries

Use three database layers:

```
public
  Authoritative domain tables and controlled RPCs.
  RLS-enabled tables and approved public API functions only.

private
  Worker-only outbox, event receipt, secrets-adjacent metadata,
  queue state, internal projection helpers.

ops
  Read-only operational projections, query functions, summary views,
  freshness state, inbox items, and dashboard data contracts.
```

The browser should call only:

```
public.ops_executive_overview(...)
public.ops_tenant_overview(...)
public.ops_operations_inbox(...)
public.ops_incident_overview(...)
public.ops_slo_overview(...)
public.ops_access_review_overview(...)
public.ops_rollout_overview(...)
```

Do not expose `ops` tables directly through generic PostgREST/GraphQL. Prefer capability-checked RPCs that return only the exact fields required by the screen.

2. Core event/outbox tables

Create a migration, for example:

```
supabase/migrations/20261005_ops_projection_pipeline.sql
```

```
create schema if not exists ops;
create schema if not exists private;

revoke all on schema ops from anon, authenticated;
revoke all on schema private from anon, authenticated;

create type private.domain_event_status as enum (
  'pending',
  'processing',
  'processed',
  'failed',
  'dead_letter'
);

create table if not exists private.domain_outbox_events (
  id uuid primary key default gen_random_uuid(),

  event_type text not null,
  event_version integer not null default 1,

  aggregate_type text not null,
  aggregate_id uuid not null,

  tenant_id uuid,
  customer_id uuid,

  correlation_id text not null,
  causation_id uuid,

  actor_type text not null default 'system',
  actor_id uuid,

  payload jsonb not null default '{}'::jsonb,
```

```
  payload_hash text not null,

  occurred_at timestamptz not null default now(),

  status private.domain_event_status not null default 'pending',
  attempts integer not null default 0,
  available_at timestamptz not null default now(),
  locked_at timestamptz,
  locked_by text,
  processed_at timestamptz,
  last_error text,

  created_at timestamptz not null default now(),

  unique (event_type, aggregate_type, aggregate_id, payload_hash)
);

create index if not exists domain_outbox_events_pending_idx
  on private.domain_outbox_events (status, available_at, occurred_at)
  where status in ('pending', 'failed');

create index if not exists domain_outbox_events_tenant_idx
  on private.domain_outbox_events (tenant_id, occurred_at desc)
  where tenant_id is not null;

create index if not exists domain_outbox_events_correlation_idx
  on private.domain_outbox_events (correlation_id, occurred_at);

create table if not exists private.projection_event_receipts (
  projection_name text not null,
  event_id uuid not null references private.domain_outbox_events(id) on delete casca
  processed_at timestamptz not null default now(),
  projection_version integer not null default 1,
  result_hash text,
  primary key (projection_name, event_id)
);

create table if not exists ops.projection_watermarks (
  projection_name text primary key,
  projection_version integer not null default 1,

  last_event_id uuid,
  last_event_at timestamptz,

  source_updated_at timestamptz,
  projected_at timestamptz not null default now(),

  status text not null default 'healthy'
    check (status in ('healthy', 'delayed', 'failed', 'rebuilding')),

  lag_seconds integer,
  last_error text,
  rebuild_requested_at timestamptz,
  rebuilt_at timestamptz
);
```

```
create table if not exists ops.projection_invalidations (
  id uuid primary key default gen_random_uuid(),

  namespace text not null,
  cache_key text not null,

  tenant_id uuid,
  customer_id uuid,

  reason text not null,
  event_id uuid references private.domain_outbox_events(id) on delete set null,

  invalidated_at timestamptz not null default now(),
  expires_at timestamptz,

  unique (namespace, cache_key)
);

create index if not exists projection_invalidations_active_idx
  on ops.projection_invalidations (namespace, cache_key, invalidated_at desc);

create table if not exists ops.projection_rebuild_runs (
  id uuid primary key default gen_random_uuid(),

  projection_name text not null,
  projection_version integer not null,

  requested_by uuid,
  requested_reason text not null,

  mode text not null
    check (mode in ('incremental', 'full', 'blue_green')),

  status text not null default 'queued'
    check (status in ('queued', 'running', 'completed', 'failed', 'cancelled')),

  started_at timestamptz,
  completed_at timestamptz,

  events_replayed bigint not null default 0,
  rows_written bigint not null default 0,

  error_summary text,
  correlation_id text not null,

  created_at timestamptz not null default now()
);
```

3. Domain-event writer

Do not emit events from the browser. Domain commands write an event **in the same database transaction** as the domain change.

```
create or replace function private.record_domain_event(
  p_event_type text,
  p_event_version integer,
  p_aggregate_type text,
  p_aggregate_id uuid,
  p_tenant_id uuid,
  p_customer_id uuid,
  p_correlation_id text,
  p_causation_id uuid,
  p_actor_type text,
  p_actor_id uuid,
  p_payload jsonb
)
returns uuid
language plpgsql
security definer
set search_path = ''
as $$
declare
  v_event_id uuid;
  v_payload_hash text;
begin
  v_payload_hash := encode(
    extensions.digest(
      coalesce(p_payload::text, '{}'),
      'sha256'
    ),
    'hex'
  );

  insert into private.domain_outbox_events (
    event_type,
    event_version,
    aggregate_type,
    aggregate_id,
    tenant_id,
    customer_id,
    correlation_id,
    causation_id,
    actor_type,
    actor_id,
    payload,
    payload_hash
  )
  values (
    p_event_type,
    p_event_version,
    p_aggregate_type,
    p_aggregate_id,
    p_tenant_id,
    p_customer_id,
    p_correlation_id,
    p_causation_id,
    p_actor_type,
    p_actor_id,
    p_payload,
```

```
    v_payload_hash
  )
  on conflict (event_type, aggregate_type, aggregate_id, payload_hash)
  do update set occurred_at = excluded.occurred_at
  returning id into v_event_id;

  return v_event_id;
end;
$$;

revoke all on function private.record_domain_event(
  text, integer, text, uuid, uuid, uuid, text, uuid, text, uuid, jsonb
) from public;
```

`SECURITY DEFINER` should be used sparingly. Supabase recommends preferring `SECURITY INVOKER` where possible; when `SECURITY DEFINER` is needed, set an empty `search_path` and schema-qualify referenced relations to reduce the blast radius.[3] [4]

4. Example command with event emission

This pattern applies to a controlled tenant rollout transition:

```
create or replace function public.transition_tenant_rollout(
  p_tenant_id uuid,
  p_next_state text,
  p_evidence jsonb,
  p_correlation_id text
)
returns jsonb
language plpgsql
security definer
set search_path = ''
as $$
declare
  v_actor_id uuid := auth.uid();
  v_previous_state text;
  v_event_id uuid;
begin
  if v_actor_id is null then
    raise exception 'unauthenticated';
  end if;

  if not private.has_capability(
    v_actor_id,
    'tenant:rollout:transition',
    p_tenant_id
  ) then
    raise exception 'forbidden';
  end if;

  select rollout_state
    into v_previous_state
  from public.tenant_rollout
  where tenant_id = p_tenant_id
```

```
  for update;

  if v_previous_state is null then
    raise exception 'tenant rollout record not found';
  end if;

  perform private.assert_rollout_transition_allowed(
    p_tenant_id,
    v_previous_state,
    p_next_state,
    p_evidence
  );

  update public.tenant_rollout
  set
    rollout_state = p_next_state,
    updated_at = now()
  where tenant_id = p_tenant_id;

  insert into public.tenant_rollout_history (
    tenant_id,
    previous_state,
    next_state,
    changed_by,
    evidence,
    correlation_id
  )
  values (
    p_tenant_id,
    v_previous_state,
    p_next_state,
    v_actor_id,
    p_evidence,
    p_correlation_id
  );

  v_event_id := private.record_domain_event(
    'tenant.rollout.transitioned',
    1,
    'tenant_rollout',
    p_tenant_id,
    p_tenant_id,
    null,
    p_correlation_id,
    null,
    'user',
    v_actor_id,
    jsonb_build_object(
      'previous_state', v_previous_state,
      'next_state', p_next_state,
      'evidence', p_evidence
    )
  );

  return jsonb_build_object(
    'tenant_id', p_tenant_id,
```

```
    'previous_state', v_previous_state,
    'next_state', p_next_state,
    'event_id', v_event_id,
    'correlation_id', p_correlation_id
  );
end;
$$;

revoke all on function public.transition_tenant_rollout(
  uuid, text, jsonb, text
) from public;

grant execute on function public.transition_tenant_rollout(
  uuid, text, jsonb, text
) to authenticated;
```

The internal capability function must check both the operator's role/capability and scope. A user who can administer Tenant A must not gain access to Tenant B because they supplied a different `tenant_id`.

Projection implementation

Tenant health projection

```
create table if not exists ops.tenant_health_snapshots (
  tenant_id uuid primary key,

  overall_score integer
    check (overall_score between 0 and 100),

  health_band text not null
    check (
      health_band in (
        'healthy',
        'watch',
        'at_risk',
        'critical',
        'unknown'
      )
    ),

  commercial_score integer,
  implementation_score integer,
  adoption_score integer,
  support_score integer,
  reliability_score integer,
  integration_score integer,
  security_score integer,
  relationship_score integer,

  top_drivers jsonb not null default '[]'::jsonb,
  recommended_actions jsonb not null default '[]'::jsonb,
```

```
  source_updated_at timestamptz,
  computed_at timestamptz not null default now(),

  projection_version integer not null default 1,
  freshness text not null default 'recent'
    check (freshness in ('live', 'recent', 'delayed', 'stale', 'failed'))
);

create index if not exists tenant_health_band_idx
  on ops.tenant_health_snapshots (health_band, computed_at desc);
```

Projection worker pseudocode

The worker can run as a controlled Supabase Edge Function, a Vercel cron/server function, or an internal queue consumer. It must use a server-only credential and never run in the browser.

```
type DomainOutboxEvent = {
  id: string;
  event_type: string;
  event_version: number;
  aggregate_type: string;
  aggregate_id: string;
  tenant_id: string | null;
  customer_id: string | null;
  correlation_id: string;
  payload: Record<string, unknown>;
  occurred_at: string;
};

async function processProjectionBatch(workerId: string) {
  const events = await claimPendingEvents({
    workerId,
    maxEvents: 50,
    lockForSeconds: 60,
  });

  for (const event of events) {
    try {
      await processEvent(event);

      await markEventProcessed(event.id);
    } catch (error) {
      await recordProjectionFailure({
        eventId: event.id,
        error: sanitizeError(error),
        retryAt: calculateRetryTime(event),
      });
    }
  }
}

async function processEvent(event: DomainOutboxEvent) {
  switch (event.event_type) {
    case "tenant.rollout.transitioned":
```

```
    case "tenant.feature.changed":
    case "tenant.plan.changed":
    case "integration.sync.completed":
    case "support.ticket.updated":
    case "pilot.metric.recorded":
    case "renewal.opportunity.updated":
      if (event.tenant_id) {
        await rebuildTenantHealthProjection(event.tenant_id, event);
        await invalidateTenantCache(event.tenant_id, event);
      }
      break;

    case "approval.request.created":
    case "approval.decision.recorded":
      await rebuildApprovalInboxProjection(event);
      await invalidateApprovalCache(event);
      break;

    case "incident.declared":
    case "incident.updated":
    case "slo.snapshot.recorded":
      await rebuildIncidentAndSloProjection(event);
      await invalidateIncidentCache(event);
      break;

    default:
      await recordIgnoredEvent(event);
  }
}
```

Idempotency guard

```
create or replace function private.claim_projection_receipt(
  p_projection_name text,
  p_event_id uuid,
  p_projection_version integer
)
returns boolean
language plpgsql
security definer
set search_path = ''
as $$
begin
  insert into private.projection_event_receipts (
    projection_name,
    event_id,
    projection_version
  )
  values (
    p_projection_name,
    p_event_id,
    p_projection_version
  )
  on conflict do nothing;
```

```
  return found;
end;
$$;
```

Before modifying a projection:

```
if receipt already exists:
  skip event

otherwise:
  apply event
  update projection
  update watermark
  create cache invalidation
```

This handles duplicate delivery safely. The transactional outbox and idempotent consumer pattern are specifically recommended for CQRS read-model updates because the consumer must tolerate duplicate delivery while maintaining consistent projections. [1]

Cache invalidation strategy

Use a combination of **event-driven invalidation**, **versioned projection metadata**, and **short TTLs**. Do not use one global `clearCache()` call.

Invalidation flow

```
Authoritative write succeeds
  ↓
Outbox event created atomically
  ↓
Projection worker applies event
  ↓
Projection version/freshness updated
  ↓
Relevant cache namespaces invalidated
  ↓
Browser receives targeted invalidation or refetch signal
  ↓
UI shows updated server-authoritative read model
```

Cache invalidation matrix

Domain event	Projections to refresh	Browser query keys
Tenant rollout changed	Tenant overview, executive overview, rollout overview, inbox	`["ops","tenant",tenantId]`, `["ops","rollout",tenantId]`
Support ticket P0/P1 updated	Support overview, tenant health, executive overview, inbox	`["ops","support"]`, `["ops","tenant",tenantId]`

Domain event	Projections to refresh	Browser query keys
Pilot metric recorded	Pilot overview, tenant health, customer 360, executive overview	`["ops","pilot",pilotId]`, `["ops","tenant",tenantId]`
Renewal status changed	Customer 360, revenue overview, tenant health, inbox	`["ops","customer",customerId]`
Feature flag changed	Release overview, tenant overview, rollout overview	`["ops","releases"]`, `["ops","tenant",tenantId]`
Integration sync failed	Integration overview, tenant health, inbox, incident overview if critical	`["ops","integration",tenantId]`
Compliance evidence expired	Compliance overview, trust overview, tenant overview, inbox	`["ops","compliance",tenantId]`
Approval decided	Approval inbox, target entity overview, audit timeline	`["ops","approvals"]`
Incident updated	Incident overview, SLO overview, release overview, executive overview	`["ops","incidents"]`, `["ops","slos"]`

Database invalidation helper

```
create or replace function private.invalidate_ops_cache(
  p_namespace text,
  p_cache_key text,
  p_tenant_id uuid,
  p_customer_id uuid,
  p_reason text,
  p_event_id uuid
)
returns void
language plpgsql
security definer
set search_path = ''
as $$
begin
  insert into ops.projection_invalidations (
    namespace,
    cache_key,
    tenant_id,
    customer_id,
    reason,
    event_id,
    invalidated_at
  )
  values (
    p_namespace,
    p_cache_key,
    p_tenant_id,
    p_customer_id,
    p_reason,
    p_event_id,
    now()
  )
```

```
  on conflict (namespace, cache_key)
  do update set
    reason = excluded.reason,
    event_id = excluded.event_id,
    invalidated_at = excluded.invalidated_at;
end;
$$;
```

Client cache policy

With TanStack Query:

```
export const opsQueryKeys = {
  executive: (range: string) => ["ops", "executive", range] as const,
  inbox: (filters: Record<string, unknown>) =>
    ["ops", "inbox", filters] as const,
  tenant: (tenantId: string) =>
    ["ops", "tenant", tenantId] as const,
  tenantHealth: (tenantId: string) =>
    ["ops", "tenant-health", tenantId] as const,
  rollout: (tenantId: string) =>
    ["ops", "rollout", tenantId] as const,
  support: (tenantId?: string) =>
    ["ops", "support", tenantId ?? "all"] as const,
  incident: (incidentId: string) =>
    ["ops", "incident", incidentId] as const,
  slos: (service?: string) =>
    ["ops", "slos", service ?? "all"] as const,
};
```

```
export const opsQueryDefaults = {
  staleTime: 60_000,
  gcTime: 10 * 60_000,
  refetchOnWindowFocus: true,
  retry: 2,
};
```

TanStack Query's cache is keyed by serializable query keys and handles caching, background refetch, request deduplication, retries, and targeted invalidation. Its documented optimistic-update pattern uses `onMutate` to snapshot/update cache, `onError` to roll back, and `onSettled` to reconcile with server truth.[5] [6]

Targeted invalidation handler

```
type OpsInvalidation = {
  namespace: string;
  tenantId?: string;
  customerId?: string;
  incidentId?: string;
  reason: string;
  version?: number;
```

```
};

export function applyOpsInvalidation(
  queryClient: QueryClient,
  message: OpsInvalidation
) {
  switch (message.namespace) {
    case "tenant":
      if (message.tenantId) {
        queryClient.invalidateQueries({
          queryKey: opsQueryKeys.tenant(message.tenantId),
        });

        queryClient.invalidateQueries({
          queryKey: opsQueryKeys.tenantHealth(message.tenantId),
        });

        queryClient.invalidateQueries({
          queryKey: opsQueryKeys.rollout(message.tenantId),
        });
      }

      queryClient.invalidateQueries({
        queryKey: ["ops", "executive"],
      });
      break;

    case "approval":
      queryClient.invalidateQueries({
        queryKey: ["ops", "inbox"],
      });
      queryClient.invalidateQueries({
        queryKey: ["ops", "approvals"],
      });
      break;

    case "incident":
      queryClient.invalidateQueries({
        queryKey: ["ops", "incidents"],
      });
      queryClient.invalidateQueries({
        queryKey: ["ops", "slos"],
      });
      queryClient.invalidateQueries({
        queryKey: ["ops", "executive"],
      });
      break;

    case "support":
      queryClient.invalidateQueries({
        queryKey: ["ops", "support"],
      });
      if (message.tenantId) {
        queryClient.invalidateQueries({
          queryKey: opsQueryKeys.tenantHealth(message.tenantId),
        });
```

```
      }
      break;
  }
}
```

Realtime warning

Use realtime only for:

- Approval-inbox changes
- Incident state changes
- Support P0/P1 changes
- Projection invalidation signals
- Tenant rollout state changes

Do **not** stream every underlying domain-table row into the browser. Stream a minimal invalidation message, then refetch the scoped read model.

```
Realtime signal:
{
  namespace: "tenant",
  tenantId: "tenant-id",
  version: 42,
  reason: "integration.sync.completed"
}

Browser:
- confirms current user can view current scope
- invalidates only relevant query keys
- refetches capability-checked server contract
```

Capability-checked query RPCs

Capability function

Use one canonical internal authorization helper. Adapt names/types to your existing `app_capabilities`, `role_grants`, `role_capabilities`, and tenant membership model.

```
create or replace function private.require_capability(
  p_capability text,
  p_tenant_id uuid default null,
  p_customer_id uuid default null
)
returns void
language plpgsql
security definer
set search_path = ''
as $$
declare
```

```
  v_actor_id uuid := auth.uid();
  v_allowed boolean;
begin
  if v_actor_id is null then
    raise exception using
      errcode = '42501',
      message = 'Unauthenticated';
  end if;

  select exists (
    select 1
    from public.role_grants grant
    join public.role_capabilities capability
      on capability.role_id = grant.role_id
    join public.app_capabilities definition
      on definition.id = capability.capability_id
    where grant.user_id = v_actor_id
      and definition.key = p_capability
      and private.scope_allows(
        grant.scope_type,
        grant.scope_id,
        p_tenant_id,
        p_customer_id
      )
      and grant.revoked_at is null
      and (
        grant.expires_at is null
        or grant.expires_at > now()
      )
  )
  into v_allowed;

  if not coalesce(v_allowed, false) then
    raise exception using
      errcode = '42501',
      message = 'Forbidden',
      detail = format(
        'Missing capability %s for requested scope',
        p_capability
      );
  end if;
end;
$$;

revoke all on function private.require_capability(text, uuid, uuid) from public;
```

Executive overview RPC

```
create or replace function public.ops_executive_overview(
  p_range_start timestamptz,
  p_range_end timestamptz
)
returns jsonb
language plpgsql
security definer
```

```
set search_path = ''
as $$
declare
  v_result jsonb;
begin
  perform private.require_capability('ops:executive:read');

  select jsonb_build_object(
    'data', jsonb_build_object(
      'tenants', (
        select jsonb_build_object(
          'total', count(*),
          'healthy', count(*) filter (
            where health_band = 'healthy'
          ),
          'at_risk', count(*) filter (
            where health_band in ('at_risk', 'critical')
          )
        )
        from ops.tenant_health_snapshots
      ),

      'approvals', (
        select jsonb_build_object(
          'pending', count(*) filter (
            where status in ('pending', 'awaiting_approval')
          ),
          'overdue', count(*) filter (
            where due_at < now()
              and status in ('pending', 'awaiting_approval')
          )
        )
        from public.approval_request
      ),

      'support', (
        select jsonb_build_object(
          'open_p0', count(*) filter (
            where severity = 'p0'
              and status not in ('resolved', 'closed')
          ),
          'open_p1', count(*) filter (
            where severity = 'p1'
              and status not in ('resolved', 'closed')
          )
        )
        from public.support_tickets
      ),

      'projection_status', (
        select coalesce(
          jsonb_agg(
            jsonb_build_object(
              'projection', projection_name,
              'status', status,
              'lagSeconds', lag_seconds,
```

```
              'projectedAt', projected_at
            )
            order by projection_name
          ),
          '[]'::jsonb
        )
        from ops.projection_watermarks
      )
    ),

    'meta', jsonb_build_object(
      'generatedAt', now(),
      'freshness', 'recent',
      'authority', 'derived',
      'rangeStart', p_range_start,
      'rangeEnd', p_range_end
    )
  )
  into v_result;

  return v_result;
end;
$$;

revoke all on function public.ops_executive_overview(
  timestamptz,
  timestamptz
) from public;

grant execute on function public.ops_executive_overview(
  timestamptz,
  timestamptz
) to authenticated;
```

Tenant overview RPC

```
create or replace function public.ops_tenant_overview(
  p_tenant_id uuid
)
returns jsonb
language plpgsql
security definer
set search_path = ''
as $$
declare
  v_result jsonb;
begin
  perform private.require_capability(
    'ops:tenant:read',
    p_tenant_id,
    null
  );

  select jsonb_build_object(
    'data', jsonb_build_object(
```

```
      'tenant', (
        select jsonb_build_object(
          'id', school.id,
          'name', school.name,
          'rolloutState', rollout.rollout_state,
          'plan', plan.plan_key,
          'ssoRequired', coalesce(sso.required, false),
          'health', jsonb_build_object(
            'band', health.health_band,
            'score', health.overall_score,
            'drivers', health.top_drivers,
            'recommendedActions', health.recommended_actions,
            'computedAt', health.computed_at
          )
        )
        from public.schools school
        left join public.tenant_rollout rollout
          on rollout.tenant_id = school.id
        left join public.tenant_plan plan
          on plan.tenant_id = school.id
        left join public.tenant_sso_policy sso
          on sso.tenant_id = school.id
        left join ops.tenant_health_snapshots health
          on health.tenant_id = school.id
        where school.id = p_tenant_id
      ),

      'blockers', (
        select coalesce(
          jsonb_agg(
            jsonb_build_object(
              'type', item.item_type,
              'severity', item.severity,
              'title', item.title,
              'summary', item.summary,
              'dueAt', item.due_at,
              'status', item.status,
              'recommendedAction', item.recommended_action
            )
            order by
              case item.severity
                when 'p0' then 1
                when 'p1' then 2
                when 'p2' then 3
                else 4
              end,
              item.due_at nulls last
          ),
          '[]'::jsonb
        )
        from ops.operations_inbox_items item
        where item.tenant_id = p_tenant_id
          and item.status in ('open', 'acknowledged', 'blocked')
      )
    ),
```

```
    'meta', jsonb_build_object(
      'generatedAt', now(),
      'tenantId', p_tenant_id,
      'authority', 'derived',
      'freshness', coalesce(
        (
          select freshness
          from ops.tenant_health_snapshots
          where tenant_id = p_tenant_id
        ),
        'unknown'
      )
    )
  )
  into v_result;

  if v_result->'data'->'tenant' is null then
    raise exception using
      errcode = 'P0002',
      message = 'Tenant not found';
  end if;

  return v_result;
end;
$$;

revoke all on function public.ops_tenant_overview(uuid) from public;
grant execute on function public.ops_tenant_overview(uuid) to authenticated;
```

Operations inbox RPC

```
create or replace function public.ops_operations_inbox(
  p_tenant_id uuid default null,
  p_owner_id uuid default null,
  p_severity text default null,
  p_limit integer default 50,
  p_offset integer default 0
)
returns jsonb
language plpgsql
security definer
set search_path = ''
as $$
declare
  v_limit integer := greatest(1, least(coalesce(p_limit, 50), 200));
  v_offset integer := greatest(0, coalesce(p_offset, 0));
begin
  perform private.require_capability(
    'ops:inbox:read',
    p_tenant_id,
    null
  );

  return jsonb_build_object(
    'data', coalesce(
```

```
      (
        select jsonb_agg(
          jsonb_build_object(
            'id', item.id,
            'type', item.item_type,
            'severity', item.severity,
            'status', item.status,
            'title', item.title,
            'summary', item.summary,
            'tenantId', item.tenant_id,
            'customerId', item.customer_id,
            'ownerId', item.owner_id,
            'dueAt', item.due_at,
            'sourceType', item.source_type,
            'sourceId', item.source_id,
            'recommendedAction', item.recommended_action,
            'correlationId', item.correlation_id,
            'createdAt', item.created_at
          )
          order by
            case item.severity
              when 'p0' then 1
              when 'p1' then 2
              when 'p2' then 3
              else 4
            end,
            item.due_at nulls last,
            item.created_at asc
        )
        from (
          select *
          from ops.operations_inbox_items
          where status in ('open', 'acknowledged', 'blocked')
            and (p_tenant_id is null or tenant_id = p_tenant_id)
            and (p_owner_id is null or owner_id = p_owner_id)
            and (p_severity is null or severity = p_severity)
          order by
            case severity
              when 'p0' then 1
              when 'p1' then 2
              when 'p2' then 3
              else 4
            end,
            due_at nulls last,
            created_at asc
          limit v_limit
          offset v_offset
        ) item
      ),
      '[]'::jsonb
    ),
    'meta', jsonb_build_object(
      'generatedAt', now(),
      'limit', v_limit,
      'offset', v_offset,
      'freshness', 'recent',
```

```
      'authority', 'derived'
    )
  );
end;
$$;

revoke all on function public.ops_operations_inbox(
  uuid,
  uuid,
  text,
  integer,
  integer
) from public;

grant execute on function public.ops_operations_inbox(
  uuid,
  uuid,
  text,
  integer,
  integer
) to authenticated;
```

React CQRS component library

Query client configuration

```
import {
  QueryClient,
  QueryClientProvider,
} from "@tanstack/react-query";

const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 60_000,
      gcTime: 10 * 60_000,
      retry: 2,
      refetchOnWindowFocus: true,
      refetchOnReconnect: true,
    },
    mutations: {
      retry: 0,
    },
  },
});

export function OpsAppProviders({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <QueryClientProvider client={queryClient}>
      {children}
```

```
    </QueryClientProvider>
  );
}
```

Capability-aware data hook

```
import { useQuery } from "@tanstack/react-query";
import { supabase } from "@/lib/supabase/client";

export function useTenantOverview(tenantId: string | null) {
  return useQuery({
    queryKey: ["ops", "tenant", tenantId],
    enabled: Boolean(tenantId),

    queryFn: async () => {
      const { data, error } = await supabase.rpc(
        "ops_tenant_overview",
        {
          p_tenant_id: tenantId,
        }
      );

      if (error) {
        throw normalizeOpsError(error);
      }

      return data as OpsEnvelope<TenantOverview>;
    },

    staleTime: 60_000,
  });
}
```

Projection-state component

```
type ProjectionStateProps = {
  freshness:
    | "live"
    | "recent"
    | "delayed"
    | "stale"
    | "failed"
    | "unknown";
  generatedAt?: string;
  sourceUpdatedAt?: string | null;
  coverage?: "complete" | "partial" | "sample" | "unavailable";
  onRefresh?: () => void;
};

export function ProjectionStateBadge({
  freshness,
  generatedAt,
  sourceUpdatedAt,
```

```
  coverage = "complete",
  onRefresh,
}: ProjectionStateProps) {
  const styles = {
    live: "border-emerald-200 bg-emerald-50 text-emerald-700",
    recent: "border-sky-200 bg-sky-50 text-sky-700",
    delayed: "border-amber-200 bg-amber-50 text-amber-800",
    stale: "border-orange-200 bg-orange-50 text-orange-800",
    failed: "border-rose-200 bg-rose-50 text-rose-700",
    unknown: "border-slate-200 bg-slate-50 text-slate-600",
  };

  return (
    <div className="flex flex-wrap items-center gap-2">
      <span
        className={`inline-flex rounded-full border px-2 py-1 text-xs font-semibold $
      >
        {freshness}
      </span>

      <span className="text-xs text-slate-500">
        Coverage: {coverage}
      </span>

      {generatedAt && (
        <span className="text-xs text-slate-500">
          Calculated: {new Date(generatedAt).toLocaleString()}
        </span>
      )}

      {sourceUpdatedAt && (
        <span className="text-xs text-slate-500">
          Source updated: {new Date(sourceUpdatedAt).toLocaleString()}
        </span>
      )}

      {onRefresh && (
        <button
          onClick={onRefresh}
          className="rounded-md border border-slate-200 px-2 py-1 text-xs font-semibo
        >
          Refresh
        </button>
      )}
    </div>
  );
}
```

Sortable server-backed grid

```
type SortDirection = "asc" | "desc";

type SortState<T extends string> = {
  column: T;
  direction: SortDirection;
```

```
};

type OpsGridProps<T, TColumn extends string> = {
  rows: T[];
  columns: Array<{
    id: TColumn;
    label: string;
    render: (row: T) => React.ReactNode;
    sortable?: boolean;
  }>;
  sort: SortState<TColumn>;
  onSortChange: (next: SortState<TColumn>) => void;
  getRowKey: (row: T) => string;
  isLoading?: boolean;
  emptyMessage?: string;
};

export function OpsDataGrid<T, TColumn extends string>({
  rows,
  columns,
  sort,
  onSortChange,
  getRowKey,
  isLoading,
  emptyMessage = "No records match the current scope.",
}: OpsGridProps<T, TColumn>) {
  const toggleSort = (column: TColumn) => {
    onSortChange({
      column,
      direction:
        sort.column === column && sort.direction === "asc"
          ? "desc"
          : "asc",
    });
  };

  if (isLoading) {
    return (
      <div className="rounded-xl border border-slate-200 p-8 text-sm text-slate-500">
        Loading operational records…
      </div>
    );
  }

  if (!rows.length) {
    return (
      <div className="rounded-xl border border-dashed border-slate-300 p-8 text-sm te
        {emptyMessage}
      </div>
    );
  }

  return (
    <div className="overflow-x-auto rounded-xl border border-slate-200">
      <table className="w-full min-w-[900px] border-collapse">
        <thead className="bg-slate-50">
```

```
          <tr>
            {columns.map((column) => (
              <th
                key={column.id}
                className="border-b border-slate-200 px-4 py-3 text-left text-xs font
              >
                {column.sortable ? (
                  <button
                    onClick={() => toggleSort(column.id)}
                    className="inline-flex items-center gap-1 hover:text-indigo-700"
                  >
                    {column.label}
                    {sort.column === column.id
                      ? sort.direction === "asc"
                        ? "↑"
                        : "↓"
                      : "↕"}
                  </button>
                ) : (
                  column.label
                )}
              </th>
            ))}
          </tr>
        </thead>

        <tbody>
          {rows.map((row) => (
            <tr
              key={getRowKey(row)}
              className="border-b border-slate-100 last:border-b-0 hover:bg-indigo-50
            >
              {columns.map((column) => (
                <td key={column.id} className="px-4 py-3 text-sm text-slate-700">
                  {column.render(row)}
                </td>
              ))}
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}
```

Optimistic low-risk mutation hook

```
import { useMutation, useQueryClient } from "@tanstack/react-query";

type InboxItem = {
  id: string;
  status: "open" | "acknowledged" | "blocked" | "resolved";
  pendingSync?: boolean;
};
```

```
export function useAcknowledgeInboxItem() {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: async (itemId: string) => {
      const { data, error } = await supabase.rpc(
        "ops_acknowledge_inbox_item",
        { p_item_id: itemId }
      );

      if (error) {
        throw normalizeOpsError(error);
      }

      return data as InboxItem;
    },

    onMutate: async (itemId) => {
      await queryClient.cancelQueries({
        queryKey: ["ops", "inbox"],
      });

      const previous = queryClient.getQueryData<InboxItem[]>([
        "ops",
        "inbox",
      ]);

      queryClient.setQueryData<InboxItem[]>(
        ["ops", "inbox"],
        (current = []) =>
          current.map((item) =>
            item.id === itemId
              ? {
                  ...item,
                  status: "acknowledged",
                  pendingSync: true,
                }
              : item
          )
      );

      return { previous };
    },

    onError: (_error, _itemId, context) => {
      queryClient.setQueryData(
        ["ops", "inbox"],
        context?.previous
      );
    },

    onSuccess: (serverItem) => {
      queryClient.setQueryData<InboxItem[]>(
        ["ops", "inbox"],
        (current = []) =>
          current.map((item) =>
```

```
            item.id === serverItem.id ? serverItem : item
          )
      );
    },

    onSettled: () => {
      queryClient.invalidateQueries({
        queryKey: ["ops", "inbox"],
      });
    },
  });
}
```

TanStack Query recommends using `onMutate` to update cached state optimistically, returning context for rollback on `onError`, and reconciling against the server after the request settles.[7] [5]

Live simulator dashboard

Build a local-only simulator first. It should visually demonstrate projection lag, event rate, worker throughput, failures, retries, cache invalidation, and stale-data recovery.

```
type SimulatorState = {
  eventRatePerMinute: number;
  workerCapacityPerMinute: number;
  transientFailureRate: number;
  projectionLagSeconds: number;
  cachedTenantOverview: boolean;
  cacheTtlSeconds: number;
  forceProjectionFailure: boolean;
  activeEvents: number;
  processedEvents: number;
  deadLetterEvents: number;
  invalidations: number;
  lastProjectionAt: string | null;
};

function calculateSimulator(state: SimulatorState) {
  const backlogDelta =
    state.eventRatePerMinute - state.workerCapacityPerMinute;

  const projectedBacklog = Math.max(
    0,
    state.activeEvents + backlogDelta
  );

  const estimatedLagSeconds =
    state.workerCapacityPerMinute <= 0
      ? Number.POSITIVE_INFINITY
      : Math.round(
          (projectedBacklog / state.workerCapacityPerMinute) * 60
        );

  const freshness =
    state.forceProjectionFailure
```

```
      ? "failed"
      : estimatedLagSeconds <= 15
      ? "live"
      : estimatedLagSeconds <= 60
      ? "recent"
      : estimatedLagSeconds <= 300
      ? "delayed"
      : "stale";

  return {
    projectedBacklog,
    estimatedLagSeconds,
    freshness,
    projectedThroughput:
      state.workerCapacityPerMinute *
      (1 - state.transientFailureRate / 100),
  };
}
```

Simulator controls:

- Event rate per minute
- Projection worker capacity
- Failure percentage
- Retry delay
- Projection rebuild mode:
  - incremental
  - full rebuild
  - blue/green rebuild
- Cache TTL
- Invalidation on/off
- Force source write
- Force worker failure
- Force stale projection
- Replay dead-letter event
- Reset scenario

Simulator cards:

Card	What it shows
Event backlog	Pending/processing/failed/dead-letter events
Projection lag	Estimated seconds behind authoritative state
Cache status	Hit/miss, key, TTL, invalidation timestamp
Freshness	Live/recent/delayed/stale/failed

Card	What it shows
Worker throughput	Processed events/minute after failures
Data mismatch	Source version vs projection version
Recommended action	Scale worker, replay, invalidate, rebuild, investigate
Safety state	Whether mutations are allowed or restricted

Tests to require

```
Projection pipeline:
- Domain command writes authoritative row and outbox event atomically.
- Duplicate event delivery does not duplicate projection effects.
- Failed event retries with bounded backoff.
- Repeated failure moves event to dead letter.
- Rebuilding projection from events produces same result as incremental projection.
- Projection watermark correctly reports lag/failure.

Cache:
- Tenant rollout event invalidates tenant, rollout, and executive keys only.
- Support event invalidates support and tenant health keys.
- Unrelated tenant event does not invalidate another tenant's detail cache.
- Stale response visibly shows stale metadata.
- Sensitive results never enter a global shared client cache.

Authorization:
- Operator lacking `ops:tenant:read` cannot call tenant overview.
- Operator with Tenant A scope cannot request Tenant B data.
- Direct tables are not used as a fallback when RPC is forbidden.
- `SECURITY DEFINER` functions have safe search_path and scope checks.
- High-risk action cannot execute before approval.

UI:
- Optimistic acknowledgement rolls back after server rejection.
- High-risk action waits for server confirmation.
- Projection failure state shows last known data plus failure metadata.
- Forbidden state reveals no protected fields.
- Evidence drawer shows source, freshness, and metric definition.
```

Recommended implementation order

1. Add outbox, receipts, watermarks, invalidations, and rebuild-run records.
2. Implement tenant-health and operations-inbox projections.
3. Add capability-checked RPCs for executive, tenant, inbox, and rollout views.
4. Build the React query/data layer and shared projection-state components.
5. Add targeted invalidation events and minimal realtime invalidation messages.
6. Build the live projection/cache simulator using local data.
7. Add incident/SLO projections and command-center screens.

8. Add access-review and security-boundary workspace.
9. Harden grants, GraphQL exposure, and `SECURITY DEFINER` function inventory before broad operator rollout.

This gives Semester a resilient read side: authoritative writes remain protected and auditable, operators receive fast coherent views, and stale or delayed data is made visible rather than silently trusted.

⁂

1. https://learn.microsoft.com/en-us/azure/architecture/patterns/cqrs
2. https://event-driven.io/en/projections_and_read_models_in_event_driven_architecture/
3. https://supabase.com/docs/guides/database/postgres/row-level-security
4. https://supabase.com/docs/guides/database/functions
5. https://tanstack.com/query/latest/docs/framework/react/guides/optimistic-updates
6. https://tanstack.com/query/latest
7. https://docs.amplify.aws/react/frontend/data/optimistic-ui/
8. https://appscale.blog/en/blog/microservices-pattern-cache-aside-cqrs-2026
9. https://makerkit.dev/blog/tutorials/supabase-rls-best-practices
10. https://oneuptime.com/blog/post/2026-01-15-react-optimistic-updates-react-query/view
11. https://dev.to/thebenforce/lock-down-your-data-implement-row-level-security-policies-in-supabase-sql-4p82
12. https://www.reddit.com/r/reactjs/comments/1mgc0ch/i_built_a_package_to_simplify_optimistic_updates/
13. https://blog.entrostat.com/supabase-rls-functions/
14. https://tanstack.com/query/latest/docs/framework/optimistic-updates
15. https://supabase.com/features/row-level-security

Interactive CQRS simulator: model event lag, projection rebuilds, and cache invalidation bottlenecks under high write throughput

Computer
CQRS Component Library: build sortable React/Vue data grids, optimistic UI mutations, and offline sync fallbacks
Computer
Show me the SQL for the transactional outbox and worker loop
How should the React UI handle projection lag and optimistic updates
Write the SQL schema for the versioned read-model views

Below is the implementation pack for a **Semester Operations CQRS simulator and read-model layer**: transactional outbox SQL, worker claim loop, projection/version schema, cache invalidation, React lag-aware UI behavior, optimistic mutation rules, and offline fallback boundaries.

The critical architectural rule is: authoritative domain writes remain in your existing tenant, billing, rollout, support, GTM, trust, and compliance models; the operations UI reads purpose-built projections with explicit freshness metadata. CQRS separates read and write models precisely for cases where the query shape differs from the transactional model, and event-driven projections can update the read side from write-side events. [1] [2]

Transactional outbox SQL

Use an outbox so a successful write and its projection event are committed in the **same transaction**. A worker processes the outbox later and updates read models idempotently.

Save as a new migration after reviewing your existing `private.domain_outbox_events` and related migration history. Your production database already appears to have an outbox/event foundation, so reconcile names before adding duplicate tables. [3]

```
create schema if not exists ops;
create schema if not exists private;

revoke all on schema ops from anon, authenticated;
revoke all on schema private from anon, authenticated;

do $$
begin
  create type private.outbox_status as enum (
    'pending',
```

```
    'processing',
    'processed',
    'failed',
    'dead_letter'
  );
exception
  when duplicate_object then null;
end $$;

create table if not exists private.domain_outbox_events (
  id uuid primary key default gen_random_uuid(),

  event_type text not null,
  event_version integer not null default 1,

  aggregate_type text not null,
  aggregate_id uuid not null,

  tenant_id uuid,
  customer_id uuid,

  correlation_id text not null,
  causation_id uuid,

  actor_type text not null default 'system'
    check (actor_type in ('user', 'service', 'system', 'worker')),

  actor_id uuid,

  payload jsonb not null default '{}'::jsonb,

  -- Persisted event hash prevents duplicate logical events.
  payload_hash text not null,

  occurred_at timestamptz not null default now(),

  status private.outbox_status not null default 'pending',

  attempts integer not null default 0
    check (attempts >= 0),

  available_at timestamptz not null default now(),

  locked_at timestamptz,
  locked_by text,

  processed_at timestamptz,
  last_error text,

  created_at timestamptz not null default now(),

  unique (
    event_type,
    aggregate_type,
    aggregate_id,
    payload_hash
```

```
  )
);

create index if not exists domain_outbox_pending_idx
  on private.domain_outbox_events (
    status,
    available_at,
    occurred_at
  )
  where status in ('pending', 'failed');

create index if not exists domain_outbox_tenant_idx
  on private.domain_outbox_events (
    tenant_id,
    occurred_at desc
  )
  where tenant_id is not null;

create index if not exists domain_outbox_correlation_idx
  on private.domain_outbox_events (
    correlation_id,
    occurred_at
  );

create index if not exists domain_outbox_dead_letter_idx
  on private.domain_outbox_events (
    status,
    occurred_at desc
  )
  where status = 'dead_letter';
```

Event writer function

This function should be called from a server-side transactional command—not directly from a client.

```
create or replace function private.record_domain_event(
  p_event_type text,
  p_event_version integer,
  p_aggregate_type text,
  p_aggregate_id uuid,
  p_tenant_id uuid,
  p_customer_id uuid,
  p_correlation_id text,
  p_causation_id uuid,
  p_actor_type text,
  p_actor_id uuid,
  p_payload jsonb
)
returns uuid
language plpgsql
security definer
set search_path = ''
as $$
```

```
declare
  v_event_id uuid;
  v_payload_hash text;
begin
  if p_event_type is null or length(trim(p_event_type)) = 0 then
    raise exception 'event_type is required';
  end if;

  if p_aggregate_type is null or length(trim(p_aggregate_type)) = 0 then
    raise exception 'aggregate_type is required';
  end if;

  if p_aggregate_id is null then
    raise exception 'aggregate_id is required';
  end if;

  if p_correlation_id is null or length(trim(p_correlation_id)) = 0 then
    raise exception 'correlation_id is required';
  end if;

  v_payload_hash := encode(
    extensions.digest(
      coalesce(p_payload::text, '{}'),
      'sha256'
    ),
    'hex'
  );

  insert into private.domain_outbox_events (
    event_type,
    event_version,
    aggregate_type,
    aggregate_id,
    tenant_id,
    customer_id,
    correlation_id,
    causation_id,
    actor_type,
    actor_id,
    payload,
    payload_hash
  )
  values (
    p_event_type,
    coalesce(p_event_version, 1),
    p_aggregate_type,
    p_aggregate_id,
    p_tenant_id,
    p_customer_id,
    p_correlation_id,
    p_causation_id,
    coalesce(p_actor_type, 'system'),
    p_actor_id,
    coalesce(p_payload, '{}'::jsonb),
    v_payload_hash
  )
```

```
  on conflict (
    event_type,
    aggregate_type,
    aggregate_id,
    payload_hash
  )
  do update set
    occurred_at = excluded.occurred_at
  returning id into v_event_id;

  return v_event_id;
end;
$$;

revoke all on function private.record_domain_event(
  text, integer, text, uuid, uuid, uuid, text, uuid, text, uuid, jsonb
) from public;
```

Worker claim loop

Worker queue function

Use `FOR UPDATE SKIP LOCKED` so multiple workers can claim events without processing the same event concurrently.

```
create or replace function private.claim_outbox_events(
  p_worker_id text,
  p_limit integer default 50,
  p_lock_seconds integer default 60
)
returns table (
  id uuid,
  event_type text,
  event_version integer,
  aggregate_type text,
  aggregate_id uuid,
  tenant_id uuid,
  customer_id uuid,
  correlation_id text,
  causation_id uuid,
  actor_type text,
  actor_id uuid,
  payload jsonb,
  occurred_at timestamptz,
  attempts integer
)
language plpgsql
security definer
set search_path = ''
as $$
begin
  if p_worker_id is null or length(trim(p_worker_id)) = 0 then
    raise exception 'worker id is required';
  end if;
```

```
  return query
  with candidates as (
    select event.id
    from private.domain_outbox_events event
    where
      (
        event.status in ('pending', 'failed')
        and event.available_at <= now()
      )
      or (
        event.status = 'processing'
        and event.locked_at < now() - make_interval(secs => p_lock_seconds)
      )
    order by event.occurred_at asc
    limit greatest(1, least(coalesce(p_limit, 50), 250))
    for update skip locked
  ),
  claimed as (
    update private.domain_outbox_events event
    set
      status = 'processing',
      locked_at = now(),
      locked_by = p_worker_id,
      attempts = event.attempts + 1
    from candidates
    where event.id = candidates.id
    returning
      event.id,
      event.event_type,
      event.event_version,
      event.aggregate_type,
      event.aggregate_id,
      event.tenant_id,
      event.customer_id,
      event.correlation_id,
      event.causation_id,
      event.actor_type,
      event.actor_id,
      event.payload,
      event.occurred_at,
      event.attempts
  )
  select * from claimed;
end;
$$;

revoke all on function private.claim_outbox_events(
  text,
  integer,
  integer
) from public;
```

Mark processed

```
create or replace function private.mark_outbox_event_processed(
  p_event_id uuid,
  p_worker_id text
)
returns void
language plpgsql
security definer
set search_path = ''
as $$
begin
  update private.domain_outbox_events
  set
    status = 'processed',
    processed_at = now(),
    locked_at = null,
    locked_by = null,
    last_error = null
  where id = p_event_id
    and status = 'processing'
    and locked_by = p_worker_id;

  if not found then
    raise exception 'event was not owned by worker or is not processing';
  end if;
end;
$$;

revoke all on function private.mark_outbox_event_processed(
  uuid,
  text
) from public;
```

Retry or dead-letter

```
create or replace function private.mark_outbox_event_failed(
  p_event_id uuid,
  p_worker_id text,
  p_error text,
  p_max_attempts integer default 8
)
returns void
language plpgsql
security definer
set search_path = ''
as $$
declare
  v_attempts integer;
  v_delay_seconds integer;
begin
  select attempts
    into v_attempts
  from private.domain_outbox_events
```

```
  where id = p_event_id
    and status = 'processing'
    and locked_by = p_worker_id
  for update;

  if v_attempts is null then
    raise exception 'event was not owned by worker or is not processing';
  end if;

  -- Exponential backoff: 2^attempts seconds, capped at 1 hour.
  v_delay_seconds := least(
    power(2, least(v_attempts, 12))::integer,
    3600
  );

  update private.domain_outbox_events
  set
    status = case
      when v_attempts >= p_max_attempts then 'dead_letter'::private.outbox_status
      else 'failed'::private.outbox_status
    end,
    available_at = case
      when v_attempts >= p_max_attempts then now()
      else now() + make_interval(secs => v_delay_seconds)
    end,
    locked_at = null,
    locked_by = null,
    last_error = left(coalesce(p_error, 'unknown projection error'), 4000)
  where id = p_event_id;
end;
$$;

revoke all on function private.mark_outbox_event_failed(
  uuid,
  text,
  text,
  integer
) from public;
```

Idempotent receipts

Every projection records an event receipt before applying the projection update.

```
create table if not exists private.projection_event_receipts (
  projection_name text not null,
  event_id uuid not null
    references private.domain_outbox_events(id)
    on delete cascade,

  projection_version integer not null default 1,

  processed_at timestamptz not null default now(),
  result_hash text,

  primary key (
```

```
    projection_name,
    event_id
  )
);

create or replace function private.claim_projection_receipt(
  p_projection_name text,
  p_event_id uuid,
  p_projection_version integer
)
returns boolean
language plpgsql
security definer
set search_path = ''
as $$
begin
  insert into private.projection_event_receipts (
    projection_name,
    event_id,
    projection_version
  )
  values (
    p_projection_name,
    p_event_id,
    p_projection_version
  )
  on conflict do nothing;

  return found;
end;
$$;

revoke all on function private.claim_projection_receipt(
  text,
  uuid,
  integer
) from public;
```

Projection worker logic

Run the worker from a server-only environment: Supabase Edge Function, protected Vercel cron/job, dedicated service, or CI-triggered worker. Never execute it from a browser.

```
type OutboxEvent = {
  id: string;
  event_type: string;
  event_version: number;
  aggregate_type: string;
  aggregate_id: string;
  tenant_id: string | null;
  customer_id: string | null;
  correlation_id: string;
  causation_id: string | null;
  actor_type: "user" | "service" | "system" | "worker";
```

```
  actor_id: string | null;
  payload: Record<string, unknown>;
  occurred_at: string;
  attempts: number;
};

const WORKER_ID = `ops-projection-${crypto.randomUUID()}`;

export async function runProjectionWorker() {
  const events = await claimEvents({
    workerId: WORKER_ID,
    limit: 50,
    lockSeconds: 90,
  });

  for (const event of events) {
    try {
      await projectEvent(event);

      await markProcessed({
        eventId: event.id,
        workerId: WORKER_ID,
      });
    } catch (error) {
      await markFailed({
        eventId: event.id,
        workerId: WORKER_ID,
        error: sanitizeError(error),
        maxAttempts: 8,
      });
    }
  }

  return {
    workerId: WORKER_ID,
    claimed: events.length,
  };
}

async function projectEvent(event: OutboxEvent) {
  switch (event.event_type) {
    case "tenant.rollout.transitioned":
    case "tenant.feature.changed":
    case "tenant.plan.changed":
    case "tenant.sso.changed":
    case "integration.sync.completed":
    case "integration.sync.failed":
    case "support.ticket.updated":
    case "pilot.metric.recorded":
    case "renewal.opportunity.updated":
      if (event.tenant_id) {
        await updateTenantHealth(event.tenant_id, event);
        await updateTenantOverview(event.tenant_id, event);
        await invalidateTenantReadModels(event);
      }
      break;
```

```
    case "approval.request.created":
    case "approval.decision.recorded":
      await updateApprovalInbox(event);
      await invalidateApprovalReadModels(event);
      break;

    case "incident.declared":
    case "incident.updated":
    case "incident.resolved":
    case "slo.snapshot.recorded":
      await updateIncidentAndSloReadModels(event);
      await invalidateIncidentReadModels(event);
      break;

    case "compliance.evidence.expired":
    case "trust.artifact.review_due":
      await updateComplianceReadModels(event);
      await invalidateComplianceReadModels(event);
      break;

    default:
      await recordIgnoredEvent(event);
  }
}
```

Versioned read-model schema

Use projection versions so schema changes do not corrupt dashboards or force unsafe in-place rewrites.

```
create table if not exists ops.read_model_registry (
  model_name text not null,
  version integer not null,

  source_event_types text[] not null,
  description text not null,

  refresh_strategy text not null
    check (
      refresh_strategy in (
        'event_driven',
        'scheduled',
        'hybrid',
        'live_query'
      )
    ),

  owner_role text not null,

  current boolean not null default false,

  created_at timestamptz not null default now(),
  retired_at timestamptz,
```

```
  primary key (model_name, version)
);

create unique index if not exists read_model_registry_one_current_idx
  on ops.read_model_registry (model_name)
  where current = true;

create table if not exists ops.projection_watermarks (
  projection_name text primary key,

  projection_version integer not null default 1,

  last_event_id uuid,
  last_event_at timestamptz,

  source_updated_at timestamptz,
  projected_at timestamptz not null default now(),

  status text not null default 'healthy'
    check (
      status in ('healthy', 'delayed', 'failed', 'rebuilding')
    ),

  lag_seconds integer,
  last_error text,

  rebuild_requested_at timestamptz,
  rebuilt_at timestamptz
);

create table if not exists ops.tenant_overview_v1 (
  tenant_id uuid primary key,

  tenant_name text not null,

  rollout_state text,
  plan_key text,
  sso_required boolean,

  active_users_30d integer,
  activation_rate numeric(8,4),
  workflow_completion_rate numeric(8,4),

  open_p0_tickets integer not null default 0,
  open_p1_tickets integer not null default 0,

  integration_health text,
  security_posture text,
  compliance_status text,

  health_score integer,
  health_band text,

  top_drivers jsonb not null default '[]'::jsonb,
  recommended_actions jsonb not null default '[]'::jsonb,
```

```
  source_updated_at timestamptz,
  computed_at timestamptz not null default now(),

  model_version integer not null default 1,

  freshness text not null default 'recent'
    check (
      freshness in ('live', 'recent', 'delayed', 'stale', 'failed')
    )
);

create index if not exists tenant_overview_v1_health_idx
  on ops.tenant_overview_v1 (
    health_band,
    computed_at desc
  );

create index if not exists tenant_overview_v1_rollout_idx
  on ops.tenant_overview_v1 (
    rollout_state,
    computed_at desc
  );

create table if not exists ops.operations_inbox_v1 (
  id uuid primary key default gen_random_uuid(),

  item_type text not null,
  severity text not null
    check (severity in ('p0', 'p1', 'p2', 'p3')),

  status text not null
    check (
      status in (
        'open',
        'acknowledged',
        'blocked',
        'resolved',
        'dismissed'
      )
    ),

  tenant_id uuid,
  customer_id uuid,

  source_type text not null,
  source_id uuid,

  title text not null,
  summary text,

  owner_id uuid,
  due_at timestamptz,

  evidence jsonb not null default '{}'::jsonb,
  recommended_action jsonb not null default '{}'::jsonb,
```

```
  correlation_id text,

  source_updated_at timestamptz,
  computed_at timestamptz not null default now(),

  model_version integer not null default 1
);

create index if not exists operations_inbox_v1_active_idx
  on ops.operations_inbox_v1 (
    severity,
    due_at,
    created_at
  )
  where status in ('open', 'acknowledged', 'blocked');

create index if not exists operations_inbox_v1_tenant_idx
  on ops.operations_inbox_v1 (
    tenant_id,
    status,
    severity
  )
  where tenant_id is not null;

create table if not exists ops.slo_overview_v1 (
  service_key text primary key,

  service_name text not null,
  criticality text not null,

  objective numeric(8,6) not null,
  compliance_ratio numeric(8,6),

  error_budget_remaining numeric(8,6),

  burn_rate_1h numeric(10,4),
  burn_rate_6h numeric(10,4),
  burn_rate_24h numeric(10,4),

  release_policy text not null,
  release_restriction text,

  active_incident_count integer not null default 0,

  source_updated_at timestamptz,
  computed_at timestamptz not null default now(),

  model_version integer not null default 1,

  freshness text not null default 'recent'
    check (
      freshness in ('live', 'recent', 'delayed', 'stale', 'failed')
    )
);
```

```
create table if not exists ops.incident_overview_v1 (
  incident_id uuid primary key,

  severity text not null,
  status text not null,

  title text not null,

  started_at timestamptz not null,
  detected_at timestamptz,

  commander_id uuid,
  technical_lead_id uuid,
  communications_lead_id uuid,

  affected_tenant_count integer not null default 0,
  affected_user_count integer,

  customer_impact_summary text,

  next_update_at timestamptz,

  linked_release_ids jsonb not null default '[]'::jsonb,
  linked_slo_keys text[] not null default '{}',
  linked_runbooks jsonb not null default '[]'::jsonb,

  source_updated_at timestamptz,
  computed_at timestamptz not null default now(),

  model_version integer not null default 1
);
```

Blue/green projection rebuild

For a breaking projection change:

```
tenant_overview_v1 remains live
        ↓
Build tenant_overview_v2 from source events/domain tables
        ↓
Run parity checks:
- row count
- tenant ID coverage
- aggregate comparison
- randomized record comparison
- authorization contract tests
        ↓
Set registry current=true for v2
        ↓
Switch RPC to v2
        ↓
Retain v1 for rollback window
        ↓
Retire v1 after evidence-backed review
```

```
begin;

update ops.read_model_registry
set current = false
where model_name = 'tenant_overview'
  and current = true;

update ops.read_model_registry
set current = true
where model_name = 'tenant_overview'
  and version = 2;

commit;
```

Never point the UI directly at a new projection table. Keep RPC names stable and change the internal implementation after parity verification.

Capability-checked query RPCs

Canonical capability check

Adapt this to your existing role/capability model. Semester already has `app_roles`, `app_capabilities`, `role_grants`, `role_capabilities`, and scope-aware authorization concepts. [4]

```
create or replace function private.require_capability(
  p_capability_key text,
  p_tenant_id uuid default null,
  p_customer_id uuid default null
)
returns void
language plpgsql
security definer
set search_path = ''
as $$
declare
  v_actor_id uuid := auth.uid();
  v_allowed boolean := false;
begin
  if v_actor_id is null then
    raise exception using
      errcode = '42501',
      message = 'Unauthenticated';
  end if;

  select exists (
    select 1
    from public.role_grants grant
    join public.role_capabilities role_capability
      on role_capability.role_id = grant.role_id
    join public.app_capabilities capability
      on capability.id = role_capability.capability_id
    where grant.user_id = v_actor_id
      and capability.key = p_capability_key
```

```
      and grant.revoked_at is null
      and (
        grant.expires_at is null
        or grant.expires_at > now()
      )
      and private.scope_allows(
        grant.scope_type,
        grant.scope_id,
        p_tenant_id,
        p_customer_id
      )
  )
  into v_allowed;

  if not v_allowed then
    raise exception using
      errcode = '42501',
      message = 'Forbidden',
      detail = format(
        'Missing capability "%s" for requested scope',
        p_capability_key
      );
  end if;
end;
$$;

revoke all on function private.require_capability(
  text,
  uuid,
  uuid
) from public;
```

Tenant overview

```
create or replace function public.ops_tenant_overview(
  p_tenant_id uuid
)
returns jsonb
language plpgsql
security definer
set search_path = ''
as $$
declare
  v_result jsonb;
begin
  perform private.require_capability(
    'ops:tenant:read',
    p_tenant_id,
    null
  );

  select jsonb_build_object(
    'data',
    jsonb_build_object(
      'tenantId', overview.tenant_id,
```

```
      'tenantName', overview.tenant_name,
      'rolloutState', overview.rollout_state,
      'planKey', overview.plan_key,
      'ssoRequired', overview.sso_required,

      'adoption', jsonb_build_object(
        'activeUsers30d', overview.active_users_30d,
        'activationRate', overview.activation_rate,
        'workflowCompletionRate', overview.workflow_completion_rate
      ),

      'support', jsonb_build_object(
        'openP0Tickets', overview.open_p0_tickets,
        'openP1Tickets', overview.open_p1_tickets
      ),

      'health', jsonb_build_object(
        'score', overview.health_score,
        'band', overview.health_band,
        'topDrivers', overview.top_drivers,
        'recommendedActions', overview.recommended_actions
      )
    ),

    'meta',
    jsonb_build_object(
      'generatedAt', now(),
      'sourceUpdatedAt', overview.source_updated_at,
      'computedAt', overview.computed_at,
      'freshness', overview.freshness,
      'authority', 'derived',
      'modelVersion', overview.model_version,
      'correlationId', gen_random_uuid()::text
    ),

    'permissions',
    jsonb_build_object(
      'canView', true,
      'canExport', private.has_capability(
        auth.uid(),
        'ops:tenant:export',
        p_tenant_id
      ),
      'allowedActions', private.allowed_ops_actions(
        auth.uid(),
        p_tenant_id
      )
    )
  )
  into v_result
  from ops.tenant_overview_v1 overview
  where overview.tenant_id = p_tenant_id;

  if v_result is null then
    raise exception using
      errcode = 'P0002',
```

```
      message = 'Tenant read model not found';
  end if;

  return v_result;
end;
$$;

revoke all on function public.ops_tenant_overview(uuid) from public;
grant execute on function public.ops_tenant_overview(uuid) to authenticated;
```

Projection dashboard RPC

```
create or replace function public.ops_projection_dashboard()
returns jsonb
language plpgsql
security definer
set search_path = ''
as $$
begin
  perform private.require_capability('ops:projection:read');

  return jsonb_build_object(
    'data',
    coalesce(
      (
        select jsonb_agg(
          jsonb_build_object(
            'projectionName', watermark.projection_name,
            'version', watermark.projection_version,
            'status', watermark.status,
            'lagSeconds', watermark.lag_seconds,
            'sourceUpdatedAt', watermark.source_updated_at,
            'projectedAt', watermark.projected_at,
            'lastError', watermark.last_error,
            'rebuildRequestedAt', watermark.rebuild_requested_at,
            'rebuiltAt', watermark.rebuilt_at
          )
          order by watermark.projection_name
        )
        from ops.projection_watermarks watermark
      ),
      '[]'::jsonb
    ),
    'meta',
    jsonb_build_object(
      'generatedAt', now(),
      'authority', 'derived',
      'freshness', 'recent'
    )
  );
end;
$$;
```

```
revoke all on function public.ops_projection_dashboard() from public;
grant execute on function public.ops_projection_dashboard() to authenticated;
```

Projection rebuild request

This does not rebuild immediately. It creates an approval-aware request.

```
create or replace function public.ops_request_projection_rebuild(
  p_projection_name text,
  p_mode text,
  p_reason text,
  p_correlation_id text
)
returns jsonb
language plpgsql
security definer
set search_path = ''
as $$
declare
  v_actor_id uuid := auth.uid();
  v_request_id uuid;
begin
  perform private.require_capability('ops:projection:rebuild:request');

  if p_mode not in ('incremental', 'full', 'blue_green') then
    raise exception 'unsupported rebuild mode';
  end if;

  if length(trim(coalesce(p_reason, ''))) < 10 then
    raise exception 'rebuild reason must explain the operational need';
  end if;

  insert into ops.projection_rebuild_runs (
    projection_name,
    projection_version,
    requested_by,
    requested_reason,
    mode,
    status,
    correlation_id
  )
  values (
    p_projection_name,
    coalesce(
      (
        select projection_version
        from ops.projection_watermarks
        where projection_name = p_projection_name
      ),
      1
    ),
    v_actor_id,
    p_reason,
    p_mode,
    'queued',
```

```
    p_correlation_id
  )
  returning id into v_request_id;

  perform private.record_domain_event(
    'ops.projection.rebuild_requested',
    1,
    'projection_rebuild',
    v_request_id,
    null,
    null,
    p_correlation_id,
    null,
    'user',
    v_actor_id,
    jsonb_build_object(
      'projection_name', p_projection_name,
      'mode', p_mode,
      'reason', p_reason
    )
  );

  return jsonb_build_object(
    'requestId', v_request_id,
    'status', 'queued',
    'correlationId', p_correlation_id
  );
end;
$$;

revoke all on function public.ops_request_projection_rebuild(
  text,
  text,
  text,
  text
) from public;

grant execute on function public.ops_request_projection_rebuild(
  text,
  text,
  text,
  text
) to authenticated;
```

React: handling lag and optimistic updates

Lag-aware query state

```
type Freshness =
  | "live"
  | "recent"
  | "delayed"
  | "stale"
  | "failed";
```

```
function getLagPresentation(
  freshness: Freshness,
  lagSeconds?: number | null
) {
  if (freshness === "failed") {
    return {
      tone: "critical",
      label: "Projection failed",
      message: "Showing last known data. Do not use this view for high-risk decisions
    };
  }

  if (freshness === "stale") {
    return {
      tone: "warning",
      label: "Data is stale",
      message: `Projection is ${lagSeconds ?? "significantly"} seconds behind source
    };
  }

  if (freshness === "delayed") {
    return {
      tone: "warning",
      label: "Data delayed",
      message: `Projection lag: ${lagSeconds ?? "unknown"} seconds.`,
    };
  }

  return {
    tone: "normal",
    label: freshness === "live" ? "Live" : "Recently updated",
    message: "Read model is within its expected freshness window.",
  };
}
```

UI policy by freshness

Projection state	Read actions	Low-risk writes	High-risk writes
Live	Normal	Allowed	Allowed if approvals satisfied
Recent	Normal	Allowed	Allowed with normal guardrails
Delayed	Normal with badge	Allowed if direct command confirmation exists	Require live source preflight
Stale	Show last known state	Avoid if action depends on current state	Disable or force authoritative preflight
Failed	Show last known state + error	Disable unless independent direct command is safe	Disable
Rebuilding	Show version/freshness notice	Usually avoid	Disable unless break-glass procedure exists

High-risk action preflight

```
async function preflightHighRiskAction(input: {
  action: string;
  tenantId: string;
  expectedProjectionVersion?: number;
}) {
  const { data, error } = await supabase.rpc(
    "ops_preflight_controlled_action",
    {
      p_action: input.action,
      p_tenant_id: input.tenantId,
      p_expected_projection_version: input.expectedProjectionVersion ?? null,
    }
  );

  if (error) {
    throw normalizeOpsError(error);
  }

  return data as {
    allowed: boolean;
    requiresApproval: boolean;
    projectionFresh: boolean;
    sourceStateVersion: string;
    blockers: string[];
    impactPreview: Record<string, unknown>;
  };
}
```

Before enabling a destructive or high-impact button:

```
1. Fetch current authoritative preflight.
2. Verify the caller capability.
3. Verify approval request/decision state.
4. Verify source state version—not merely projection version.
5. Show exact impact and rollback plan.
6. Submit the controlled server command.
7. Wait for the server response.
8. Invalidate projections after the resulting event arrives.
```

Offline fallback boundaries

Operation	Offline support	Behavior
View cached dashboard	Yes, clearly labeled	Show timestamp, stale badge, no false “live” state
Save local filter/view	Yes	Store user preference locally or sync when online
Draft internal note	Yes, queued	Mark pending; sync with conflict detection
Acknowledge inbox item	Optional queued action	Only if backend supports idempotency and action is low risk
Approve/reject a high-risk request	No	Require fresh authenticated connection

Operation	Offline support	Behavior
Feature rollout	No	Require authoritative preflight
Refund/credit/billing action	No	Require fresh connection and approval
Break-glass access	No	Require live identity, reason, and audit write
Tenant offboarding/data export	No	Require live approval and legal-hold validation

CQRS simulator implementation prompt

Use this with Claude Code or a coding agent:

```
Build a local-first CQRS Projection Simulator inside the existing Semester app.

Purpose:
Let authorized internal operators understand how transactional writes become
read-model updates, how event backlogs create projection lag, how retries and
dead-letter events behave, how cache invalidations target specific read models,
and how UI freshness affects safe operational actions.

Do not connect the simulator to production data in its first version.

Build:
1. Transactional outbox simulation
2. Projection worker simulation
3. Projection watermark and lag simulation
4. Cache namespace/key invalidation simulation
5. Retry and exponential-backoff simulation
6. Dead-letter queue simulation
7. Full rebuild, incremental rebuild, and blue/green rebuild simulation
8. Read-model version switching simulation
9. Optimistic update and rollback simulation
10. Offline state/fallback simulation

Simulator controls:
- Events per minute
- Worker capacity per minute
- Worker count
- Failure rate
- Maximum retries
- Retry delay
- Cache TTL
- Invalidation enabled/disabled
- Source writes
- Projection rebuild mode
- Force stale projection
- Force worker failure
- Replay dead-letter event
- Reset state

Simulator output:
- Pending events
- Processing events
- Processed events
```

```
- Failed events
- Dead-letter events
- Projection lag
- Source version versus projection version
- Projection freshness
- Cache hit/miss rate
- Invalidations emitted
- Worker throughput
- Recommended operational action
- Whether low-risk and high-risk console actions are allowed

UI:
- Premium operations dashboard
- Timeline of source write → outbox → worker → projection → cache invalidation → UI r
- Sortable event queue
- Projection-watermark table
- Cache-key inspector
- Read-model version table
- Optimistic mutation panel with success/rollback states
- Offline fallback panel
- Export simulation state as JSON and Markdown

Add tests for:
- Duplicate event delivery
- Idempotent receipts
- Retry behavior
- Dead-letter behavior
- Cache invalidation targeting
- Projection freshness transitions
- Rebuild parity
- Optimistic rollback
- High-risk action disabled when projection is stale or failed
```

Build order

1. Add outbox and projection receipts only after reconciling existing private event/outbox structures.
2. Build `ops.projection_watermarks`.
3. Build the tenant-health projection first.
4. Build `ops_tenant_overview()` and `ops_projection_dashboard()` RPCs.
5. Build targeted cache invalidation.
6. Build the React freshness components and tenant screen.
7. Build the local simulator.
8. Add operations inbox projection.
9. Add incident/SLO projections.
10. Add blue/green rebuild and access-review UI.

This approach gives Semester fast operational dashboards without turning the read side into an unsafe second source of truth.

⁂

1. https://learn.microsoft.com/en-us/azure/architecture/patterns/cqrs
2. https://microservices.io/patterns/data/cqrs.html
3. https://academy.claude.com/courses/claude-101/creating-with-artifacts
4. https://info.lse.ac.uk/staff/divisions/Eden-Centre/Assets-EC/Documents/Claude-Guides-Dec-2025/Guide-Creating-Claude-Artifacts-V1.0-011225.pdf